

IDS Sentinel Terminal - Full Line-by-Line Code Explanation Report

IDS Sentinel Terminal - Code Explanation Report

1. Architectural Overview

The **IDS Sentinel Terminal** is a comprehensive Intrusion Detection System (IDS) suite that combines Classical Machine Learning algorithms with Deep Neural Networks (DNN) to identify and classify network intrusions. The project is split into several major components:

1. **Classical ML** (`classical/`, `all.py`): Utilizes scikit-learn for models like Logistic Regression, Naive Bayes, KNN, Decision Trees, AdaBoost, Random Forest, and SVM.
2. **Deep Neural Networks** (`dnn/`): Implements sequential neural networks, including RNNs, LSTMs, and GRUs, using Keras/TensorFlow.
3. **API & Backend** (`ids_app/`): A modern FastAPI-based backend that orchestrates the execution of both classical and deep learning jobs, manages storage, and exposes an HTTP API for the frontend and terminal interfaces.
4. **Frontend** (`frontend/`): An HTML/JS/CSS based user interface for interacting with the API.
5. **CLI/Terminal Tools** (`tools/`): Launchers for a terminal-based interface.

Due to the size of the repository (~6000 lines), this report provides a detailed, line-by-line and block-by-block explanation of the **core files and paradigms** that power the system, representing the logical execution flow of the entire application.

2. API Backend (`ids_app/api.py`)

This file is the primary entry point for the modern FastAPI backend. It handles incoming requests, orchestrates model training, and returns results.

```
# Line 1-9: Imports
from __future__ import annotations
from pathlib import Path
from typing import Literal
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field
```

Explanation: - The file imports essential type hinting tools (`annotations`, `Literal`) to ensure code safety. - It leverages **FastAPI** as the core web framework. - It uses **Pydantic** (`BaseModel`, `Field`) for robust data validation on incoming API payloads.

```
# Line 11-17: Internal Module Imports
from .classical import train_classical_suite
from .config import CLASSICAL_PROFILES, DNN_PROFILES, FRONTEND_DIR
from .data import dataset_summary
from .dnn import train_dnn_suite
from .jobs import job_manager
from .legacy import evaluate_legacy_predictions
from .storage import ensure_directories, list_run_summaries, read_json, run_summary_path
```

Explanation: - Imports functional modules corresponding to different system capabilities. For example, `train_classical_suite` triggers classical model evaluation, and `job_manager` handles asynchronous job execution.

```
# Line 20-26: Request Schemas
class ClassicalJobRequest(BaseModel):
    profile: Literal["fast", "balanced", "full"] = "fast"
    models: list[str] | None = None
    train_sample: int | None = Field(default=None, ge=1000)
    test_sample: int | None = Field(default=None, ge=1000)
    random_state: int = 42
```

Explanation: - Defines a Pydantic schema for requesting a Classical ML job. - It restricts `profile` to three specific strings, defaults `random_state` to 42 for reproducibility, and ensures samples are at least 1000 records.

```
# Line 37-39: App Initialization
app = FastAPI(title="IDS Automation Console", version="1.0.0")
ensure_directories()
app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")
```

Explanation: - Instantiates the FastAPI application. - Creates required directories on the filesystem (`ensure_directories()`).
- Mounts the frontend directory to serve static assets via the `/static` endpoint.

```
# Line 49-60: Overview API Route
@app.get("/api/overview")
def overview() -> dict[str, object]:
    return {
        "datasets": dataset_summary(),
        "legacy": evaluate_legacy_predictions(),
        "jobs": job_manager.list(limit=12),
        "runs": list_run_summaries(limit=12),
        "profiles": {"classical": CLASSICAL_PROFILES, "dnn": DNN_PROFILES},
    }
```

Explanation: - A critical endpoint that aggregates and returns a snapshot of the entire system state: datasets loaded, legacy prediction statuses, active/recent jobs, completed run summaries, and available configuration profiles.

3. Classical Machine Learning Runner (`all.py`)

This script acts as the legacy monolithic test suite, running multiple scikit-learn models sequentially.

```
# Line 1-21: Standard Library and ML Imports
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
...
from sklearn.preprocessing import Normalizer
```

Explanation: - Imports `pandas` for CSV parsing, `numpy` for matrix operations, and an array of `sklearn` models ranging from basic (`LogisticRegression`) to complex ensembles (`RandomForestClassifier`, `AdaBoostClassifier`). Also imports metrics and preprocessing tools like `Normalizer`.

```
# Line 23-27: Data Loading
traindata = pd.read_csv('kddtrain.csv', header=None)
testdata = pd.read_csv('kddtest.csv', header=None)
X = traindata.iloc[:,1:42]
Y = traindata.iloc[:,0]
C = testdata.iloc[:,0]
T = testdata.iloc[:,1:42]
```

Explanation: - Reads the NSL-KDD (or similar KDD-cup derived) train and test datasets from CSV format. - `iloc[:, 1:42]` extracts the 41 feature columns into X (train features) and T (test features). - `iloc[:, 0]` extracts the 1st column, representing the ground truth labels, into Y (train labels) and C (test labels).

```
# Line 29-33: Data Normalization
scaler = Normalizer().fit(X)
trainX = scaler.transform(X)
scaler = Normalizer().fit(T)
testT = scaler.transform(T)
```

Explanation: - Applies L2 normalization individually to both the training and test sets. Note: Fitting a new scaler on the test data T is generally considered an anti-pattern (data leakage/inconsistency), but it is implemented here sequentially.

```
# Line 47-73: Model Execution Block (Logistic Regression Example)
print("-----LR-----")
model = LogisticRegression()
model.fit(traindata, trainlabel)
expected = testlabel
predicted = model.predict(testdata)
proba = model.predict_proba(testdata)
...
np.savetxt('classical/predictedlabelLR.txt', predicted, fmt='%01d')
```

Explanation: - This block is repeated for every algorithm in the script. - It instantiates the model (LogisticRegression). - Fits it using the normalized feature matrix and label vector. - Computes predictions (predicted) and probability scores (proba). - Uses `np.savetxt` to dump the raw prediction outputs to text files in the `classical/` directory for legacy analysis. - Following these lines, metrics (accuracy, recall, precision, f1) are calculated using sklearn's metrics package and printed to stdout.

4. Deep Neural Network Models (dnn/dnn1.py)

This section utilizes Keras to build deep learning sequences for the same anomaly detection task.

```
# Line 1-17: Imports
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Embedding
from keras.layers import LSTM, SimpleRNN, GRU
from keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLRonPlateau, CSVLogger
```

Explanation: - Imports the Keras Sequential API to build layer-by-layer neural networks. - Brings in layers like Dense (fully connected), Dropout (regularization), and Recurrent layers (LSTM, GRU) suitable for sequential/time-series network traffic analysis. - Configures callbacks to manage the training process (e.g., EarlyStopping prevents overfitting).

```
# Line 19-38: Data Ingestion (Similar to Classical)
traindata = pd.read_csv('kdd/binary/Training.csv', header=None)
# ... data splitting and Normalizer() usage ...
```

Explanation: - The data ingestion process is identical to `all.py`, loading the KDD dataset, splitting labels and features, and normalizing them, preparing them as float arrays for Keras.

```
# Line 47-52: DNN Architecture Definition
batch_size = 64
model = Sequential()
# Followed generally by layer additions:
# model.add(Dense(1024, input_dim=41, activation='relu'))
# model.add(Dropout(0.01))
# model.add(Dense(1, activation='sigmoid'))
```

Explanation: - Sets up the training mini-batch size to 64. - Instantiates an empty Sequential computational graph. - While omitted for brevity in this exact snippet, `dnn1.py` goes on to stack fully connected Dense layers (often starting with the 41 input dimensions) combined with Dropout to handle non-linear relationships in the network traffic data, outputting to a final prediction layer.

Summary Conclusion

The project is structurally split into a **Legacy Evaluation System** consisting of raw scripts like `all.py` and `dnn/*.py` which directly load data, run scikit-learn or Keras algorithms, and spit out `.txt` prediction files. Alongside this, the **Modern Application Suite** (`ids_app/`) wraps these capabilities into a coherent, job-based web architecture via FastAPI, providing RESTful endpoints, asynchronous task processing, and Pydantic validation, offering a cleaner interface for automated intrusion detection monitoring.

Complete Source Code Analysis

This section contains an automated line-by-line block breakdown of every Python file in the repository.

Module: `./ipynb_checkpoints/all-checkpoint.py`

Overview

Total Lines: 298

Module: `./all.py`

Overview

Total Lines: 279

Module: `./classical/accuclassical.py`

Overview

Total Lines: 190

Module: `./dnn/dnn1.py`

Overview

Total Lines: 60

Module: `./dnn/dnn1acc.py`

Overview

Total Lines: 117

Module: `./dnn/dnn1test.py`

Overview

Total Lines: 143

Module: ./dnn/dnn2.py

Overview

Total Lines: 55

Module: ./dnn/dnn2test.py

Overview

Total Lines: 118

Module: ./dnn/dnn3.py

Overview

Total Lines: 57

Module: ./dnn/dnn3test.py

Overview

Total Lines: 114

Module: ./dnn/dnn4.py

Overview

Total Lines: 70

Module: ./dnn/dnn4test.py

Overview

Total Lines: 113

Module: ./dnn/dnn5.py

Overview

Total Lines: 63

Module: ./dnn/dnn5test.py

Overview

Total Lines: 107

Module: ./dnn/fix_optimizer_warning.py

Overview

Total Lines: 51

Function: load_weights_only

Lines: 1 to 8

Description: Analyzes and executes load_weights_only logic.

```
0001 | def load_weights_only(model, weights_path):
0002 |     if weights_path.endswith('.keras'):
0003 |         from keras.models import load_model
0004 |         loaded_model = load_model(weights_path)
0005 |         model.set_weights(loaded_model.get_weights())
0006 |     else:
0007 |         model.load_weights(weights_path, skip_mismatch=True, by_name=True)
0008 |     return model
```

Function: load_weights_no_warning

Lines: 17 to 21

Description: Analyzes and executes load_weights_no_warning logic.

```
0017 | def load_weights_no_warning(model, weights_path):
0018 |     with warnings.catch_warnings():
0019 |         warnings.filterwarnings('ignore', message='Skipping variable loading for optimizer')
0020 |         model.load_weights(weights_path)
0021 |     return model
```

Function: build_model_matching_saved_state

Lines: 24 to 29

Description: Analyzes and executes build_model_matching_saved_state logic.

```
0024 | def build_model_matching_saved_state(weights_path):
0025 |     from keras.models import load_model
0026 |
0027 |     model = load_model(weights_path)
0028 |
0029 |     return model
```

Function: modified_dnn3test_loading

Lines: 32 to 43

Description: Analyzes and executes modified_dnn3test_loading logic.

```
0032 | def modified_dnn3test_loading():
0033 |     import warnings
0034 |
0035 |     score = []
0036 |     name = []
0037 |
0038 |     for file in os.listdir("kddresults/dnn3layer/"):
0039 |         with warnings.catch_warnings():
0040 |             warnings.filterwarnings('ignore', message='Skipping variable loading for optimizer')
0041 |             model.load_weights("kddresults/dnn3layer/" + file)
0042 |
0043 |             y_pred = (model.predict(X_test) > 0.5).astype(int).flatten()
```

Function: load_weights_silent

Lines: 46 to 51

Description: Analyzes and executes load_weights_silent logic.

```
0046 | def load_weights_silent(model, path):  
0047 |     import warnings  
0048 |     with warnings.catch_warnings():  
0049 |         warnings.filterwarnings('ignore', category=UserWarning, module='keras.src.saving.saving_lib')  
0050 |         model.load_weights(path)  
0051 |     return model
```

Module: ./dnn/tempCodeRunnerFile.py

Overview

Total Lines: 63

Module: ./generate_code_report.py

Overview

Total Lines: 100

Function: generate_markdown

Lines: 6 to 61

Description: Analyzes and executes generate_markdown logic.

Module: ./ids_app/api.py

Overview

Total Lines: 106

Class: ClassicalJobRequest

Lines: 20 to 25

Description: Analyzes and executes ClassicalJobRequest logic.

```
0020 | class ClassicalJobRequest(BaseModel):
0021 |     profile: Literal["fast", "balanced", "full"] = "fast"
0022 |     models: list[str] | None = None
0023 |     train_sample: int | None = Field(default=None, ge=1000)
0024 |     test_sample: int | None = Field(default=None, ge=1000)
0025 |     random_state: int = 42
```

Class: DnnJobRequest

Lines: 28 to 35

Description: Analyzes and executes DnnJobRequest logic.

```
0028 | class DnnJobRequest(BaseModel):
0029 |     profile: Literal["fast", "balanced", "full"] = "fast"
0030 |     architectures: list[int] | None = None
0031 |     epochs: int | None = Field(default=None, ge=1, le=100)
0032 |     batch_size: int | None = Field(default=None, ge=16, le=512)
0033 |     train_sample: int | None = Field(default=None, ge=1000)
0034 |     test_sample: int | None = Field(default=None, ge=1000)
0035 |     random_state: int = 42
```

Function: index

Lines: 44 to 45

Description: Analyzes and executes index logic.

```
0044 | def index() -> FileResponse:
0045 |     return FileResponse(FRONTEND_DIR / "index.html")
```

Function: health

Lines: 49 to 50

Description: Analyzes and executes health logic.

```
0049 | def health() -> dict[str, str]:
0050 |     return {"status": "ok"}
```

Function: overview

Lines: 54 to 64

Description: Analyzes and executes overview logic.

```
0054 | def overview() -> dict[str, object]:
0055 |     return {
0056 |         "datasets": dataset_summary(),
0057 |         "legacy": evaluate_legacy_predictions(),
0058 |         "jobs": job_manager.list(limit=12),
0059 |         "runs": list_run_summaries(limit=12),
0060 |         "profiles": {
0061 |             "classical": CLASSICAL_PROFILES,
0062 |             "dnn": DNN_PROFILES,
0063 |         },
0064 |     }
```

Function: list_jobs

Lines: 68 to 69

Description: Analyzes and executes list_jobs logic.

```
0068 | def list_jobs() -> list[dict[str, object]]:
0069 |     return job_manager.list(limit=40)
```

Function: get_job

Lines: 73 to 77

Description: Analyzes and executes get_job logic.

```
0073 | def get_job(job_id: str) -> dict[str, object]:
0074 |     payload = job_manager.get(job_id)
0075 |     if not payload:
0076 |         raise HTTPException(status_code=404, detail="Job not found")
0077 |     return payload
```

Function: list_runs

Lines: 81 to 82

Description: Analyzes and executes list_runs logic.

```
0081 | def list_runs() -> list[dict[str, object]]:
0082 |     return list_run_summaries(limit=40)
```

Function: get_run

Lines: 86 to 90

Description: Analyzes and executes get_run logic.

```
0086 | def get_run(run_id: str) -> dict[str, object]:
0087 |     payload = read_json(run_summary_path(run_id))
0088 |     if not payload:
0089 |         raise HTTPException(status_code=404, detail="Run not found")
0090 |     return payload
```

Function: launch_legacy_evaluation

Lines: 94 to 95

Description: Analyzes and executes launch_legacy_evaluation logic.

```
0094 | def launch_legacy_evaluation() -> dict[str, object]:
0095 |     return job_manager.submit("legacy_evaluation", {}, lambda _job_id, _config: {"run_id": "legacy-snapshot", "kind":
```

Function: launch_classical

Lines: 99 to 100

Description: Analyzes and executes launch_classical logic.

```
0099 | def launch_classical(request: ClassicalJobRequest) -> dict[str, object]:
0100 |     return job_manager.submit("classical_train", request.model_dump(exclude_none=True), train_classical_suite)
```

Function: `launch_dnn`

Lines: 104 to 105

Description: Analyzes and executes launch_dnn logic.

```
0104 | def launch_dnn(request: DnnJobRequest) -> dict[str, object]:
0105 |     return job_manager.submit("dnn_train", request.model_dump(exclude_none=True), train_dnn_suite)
```

Module: `./ids_app/classical.py`

Overview

Total Lines: 113

Function: resolve_classical_config

Lines: 32 to 47

Description: Analyzes and executes resolve_classical_config logic.

```
0032 | def resolve_classical_config(config: dict[str, object] | None) -> dict[str, object]:
0033 |     payload = dict(config or {})
0034 |     profile = str(payload.get("profile", "fast"))
0035 |     if profile not in CLASSICAL_PROFILES:
0036 |         profile = "fast"
0037 |     merged = dict(CLASSICAL_PROFILES[profile])
0038 |     merged["profile"] = profile
0039 |     if payload.get("models"):
0040 |         selected = [name for name in payload["models"] if name in MODEL_BUILDERS]
0041 |         if selected:
0042 |             merged["models"] = selected
0043 |     for key in ("train_sample", "test_sample", "random_state"):
0044 |         if key in payload:
0045 |             merged[key] = payload.get(key)
0046 |     merged.setdefault("random_state", 42)
0047 |     return merged
```

Function: `train_classical_suite`

Lines: 50 to 113

Description: Analyzes and executes train_classical_suite logic.

```

0050 | def train_classical_suite(job_id: str, config: dict[str, object] | None = None) -> dict[str, object]:
0051 |     resolved = resolve_classical_config(config)
0052 |     split = load_classical_split(
0053 |         train_sample=resolved.get("train_sample"),
0054 |         test_sample=resolved.get("test_sample"),
0055 |         random_state=int(resolved.get("random_state", 42)),
0056 |     )
0057 |
0058 |     generated_run_id = f"classical-{{uuid4().hex[:12]}}"
0059 |     target_dir = run_dir(generated_run_id)
0060 |     models_dir = target_dir / "models"
0061 |     predictions_dir = target_dir / "predictions"
0062 |     models_dir.mkdir(parents=True, exist_ok=True)
0063 |     predictions_dir.mkdir(parents=True, exist_ok=True)
0064 |
0065 |     y_test = split["y_test"]
0066 |     results = []
0067 |
0068 |     for model_name in resolved["models"]:
0069 |         builder = MODEL_BUILDERS[model_name]
0070 |         estimator = builder()
0071 |         started = time.perf_counter()
0072 |         estimator.fit(split["X_train"], split["y_train"])
0073 |         training_seconds = round(time.perf_counter() - started, 3)
0074 |
0075 |         predicted = estimator.predict(split["X_test"]).astype(int)
0076 |         probabilities = None
0077 |         if hasattr(estimator, "predict_proba"):
0078 |             probabilities = estimator.predict_proba(split["X_test"])[:, 1]
0079 |
0080 |         metrics = binary_metrics(y_test, predicted)
0081 |         joblib.dump(estimator, models_dir / f"{{model_name}}.joblib")
0082 |         np.savetxt(predictions_dir / f"{{model_name}}_labels.txt", predicted, fmt="%01d")
0083 |         if probabilities is not None:
0084 |             np.savetxt(predictions_dir / f"{{model_name}}_probabilities.txt", probabilities)
0085 |
0086 |         results.append(
0087 |             {
0088 |                 "id": model_name,
0089 |                 "label": CLASSICAL_MODEL_LABELS[model_name],
0090 |                 "training_seconds": training_seconds,
0091 |                 "metrics": metrics,
0092 |                 "probability_summary": probability_summary(probabilities),
0093 |                 "model_path": str((models_dir / f"{{model_name}}.joblib").relative_to(target_dir.parent.parent)),
0094 |             }
0095 |         )
0096 |
0097 |     results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
0098 |     summary = {
0099 |         "run_id": generated_run_id,
0100 |         "job_id": job_id,
0101 |         "kind": "classical_train",
0102 |         "created_at": datetime.now(timezone.utc).isoformat(),
0103 |         "config": resolved,
0104 |         "dataset": {
0105 |             "train_rows": split["train_rows"],
0106 |             "test_rows": split["test_rows"],
0107 |             "feature_count": split["feature_count"],
0108 |         },
0109 |         "results": results,
0110 |         "best_model": results[0]["id"] if results else None,
0111 |     }
0112 |     write_json(run_summary_path(generated_run_id), summary)
0113 |     return summary

```

Module: ./ids_app/config.py

Overview

Total Lines: 96

Module: ./ids_app/data.py

Overview

Total Lines: 123

Function: _sample_frame

Lines: 16 to 20

Description: Analyzes and executes _sample_frame logic.

```
0016 | def _sample_frame(frame: pd.DataFrame, sample_size: int | None, random_state: int) -> pd.DataFrame:
0017 |     if sample_size is None or sample_size >= len(frame):
0018 |         return frame
0019 |     sampled, _ = train_test_split(frame, train_size=sample_size, stratify=frame.iloc[:, 0], random_state=random_state)
0020 |     return sampled.reset_index(drop=True)
```

Function: load_dataset_split

Lines: 23 to 53

Description: Analyzes and executes load_dataset_split logic.

```
0023 | def load_dataset_split(
0024 |     train_path: Path,
0025 |     test_path: Path,
0026 |     train_sample: int | None = None,
0027 |     test_sample: int | None = None,
0028 |     random_state: int = 42,
0029 | ) -> dict[str, np.ndarray | int | str]:
0030 |     train_frame = pd.read_csv(train_path, header=None)
0031 |     test_frame = pd.read_csv(test_path, header=None)
0032 |
0033 |     train_frame = _sample_frame(train_frame, train_sample, random_state)
0034 |     test_frame = _sample_frame(test_frame, test_sample, random_state)
0035 |
0036 |     X_train = train_frame.iloc[:, 1:42].to_numpy(dtype=np.float32)
0037 |     y_train = train_frame.iloc[:, 0].to_numpy(dtype=np.int32)
0038 |     X_test = test_frame.iloc[:, 1:42].to_numpy(dtype=np.float32)
0039 |     y_test = test_frame.iloc[:, 0].to_numpy(dtype=np.int32)
0040 |
0041 |     scaler = Normalizer()
0042 |     X_train = scaler.fit_transform(X_train)
0043 |     X_test = scaler.transform(X_test)
0044 |
0045 |     return {
0046 |         "X_train": X_train,
0047 |         "y_train": y_train,
0048 |         "X_test": X_test,
0049 |         "y_test": y_test,
0050 |         "train_rows": int(X_train.shape[0]),
0051 |         "test_rows": int(X_test.shape[0]),
0052 |         "feature_count": int(X_train.shape[1]),
0053 |     }
```

Function: load_classical_split

Lines: 56 to 57

Description: Analyzes and executes load_classical_split logic.

```
0056 | def load_classical_split(train_sample: int | None = None, test_sample: int | None = None, random_state: int = 42) -> dict[str, object]:
0057 |     return load_dataset_split(CLASSICAL_TRAIN_DATA, CLASSICAL_TEST_DATA, train_sample, test_sample, random_state)
```

Function: load_dnn_split

Lines: 60 to 61

Description: Analyzes and executes load_dnn_split logic.

```
0060 | def load_dnn_split(train_sample: int | None = None, test_sample: int | None = None, random_state: int = 42) -> dict[str, object]:
0061 |     return load_dataset_split(DNN_TRAIN_DATA, DNN_TEST_DATA, train_sample, test_sample, random_state)
```

Function: _dataset_file_summary

Lines: 64 to 92

Description: Analyzes and executes _dataset_file_summary logic.

```
0064 | def _dataset_file_summary(path: Path) -> dict[str, object]:
0065 |     sha256 = hashlib.sha256()
0066 |     row_count = 0
0067 |     columns = 0
0068 |     label_counts: dict[str, int] = {}
0069 |
0070 |     with path.open("rb") as raw_handle:
0071 |         for chunk in iter(lambda: raw_handle.read(1024 * 1024), b''):
0072 |             sha256.update(chunk)
0073 |
0074 |     with path.open("r", encoding="utf-8", errors="ignore", newline='') as handle:
0075 |         reader = csv.reader(handle)
0076 |         for row in reader:
0077 |             if not row:
0078 |                 continue
0079 |             row_count += 1
0080 |             if columns == 0:
0081 |                 columns = len(row)
0082 |             label = row[0]
0083 |             label_counts[label] = label_counts.get(label, 0) + 1
0084 |
0085 |     return {
0086 |         "path": _relative_path(path),
0087 |         "rows": row_count,
0088 |         "columns": columns,
0089 |         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
0090 |         "sha256": sha256.hexdigest(),
0091 |         "label_counts": label_counts,
0092 |     }
```

Function: _relative_path

Lines: 95 to 99

Description: Analyzes and executes _relative_path logic.

```
0095 | def _relative_path(path: Path) -> str:
0096 |     try:
0097 |         return str(path.relative_to(ROOT_DIR))
0098 |     except ValueError:
0099 |         return str(path)
```

Function: dataset_summary

Lines: 103 to 123

Description: Analyzes and executes dataset_summary logic.

```
0103 | def dataset_summary() -> dict[str, object]:
0104 |     classical_train = _dataset_file_summary(CLASSICAL_TRAIN_DATA)
0105 |     classical_test = _dataset_file_summary(CLASSICAL_TEST_DATA)
0106 |     dnn_train = _dataset_file_summary(DNN_TRAIN_DATA)
0107 |     dnn_test = _dataset_file_summary(DNN_TEST_DATA)
0108 |
0109 |     classical_train["path"] = _relative_path(CLASSICAL_TRAIN_DATA)
0110 |     classical_test["path"] = _relative_path(CLASSICAL_TEST_DATA)
0111 |     dnn_train["path"] = _relative_path(DNN_TRAIN_DATA)
0112 |     dnn_test["path"] = _relative_path(DNN_TEST_DATA)
0113 |
0114 |     return {
0115 |         "classical_train": classical_train,
0116 |         "classical_test": classical_test,
0117 |         "dnn_train": dnn_train,
0118 |         "dnn_test": dnn_test,
0119 |         "duplicates": {
0120 |             "train_files_match": classical_train["sha256"] == dnn_train["sha256"],
0121 |             "test_files_match": classical_test["sha256"] == dnn_test["sha256"],
0122 |         },
0123 |     }
```

Module: ./ids_app/dnn.py

Overview

Total Lines: 145

Function: _tensorflow

Lines: 16 to 21

Description: Analyzes and executes _tensorflow logic.

```
0016 | def _tensorflow():
0017 |     os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1")
0018 |     os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "3")
0019 |     import tensorflow as tf
0020 |
0021 |     return tf
```

Function: resolve_dnn_config

Lines: 24 to 39

Description: Analyzes and executes resolve_dnn_config logic.

```

0024 | def resolve_dnn_config(config: dict[str, object] | None) -> dict[str, object]:
0025 |     payload = dict(config or {})
0026 |     profile = str(payload.get("profile", "fast"))
0027 |     if profile not in DNN_PROFILES:
0028 |         profile = "fast"
0029 |     merged = dict(DNN_PROFILES[profile])
0030 |     merged["profile"] = profile
0031 |     if payload.get("architectures"):
0032 |         selected = [int(value) for value in payload["architectures"] if int(value) in DNN_LAYER_SPECS]
0033 |         if selected:
0034 |             merged["architectures"] = selected
0035 |     for key in ("train_sample", "test_sample", "epochs", "batch_size", "random_state"):
0036 |         if key in payload:
0037 |             merged[key] = payload.get(key)
0038 |     merged.setdefault("random_state", 42)
0039 |     return merged

```

Function: `_build_model`

Lines: 42 to 53

Description: Analyzes and executes `_build_model` logic.

```

0042 | def _build_model(layer_count: int, dropout_rate: float = 0.01):
0043 |     tf = _tensorflow()
0044 |
0045 |     tf.keras.backend.clear_session()
0046 |     model = tf.keras.Sequential()
0047 |     for index, units in enumerate(DNN_LAYER_SPECS[layer_count]):
0048 |         kwargs = {"input_dim": 41} if index == 0 else {}
0049 |         model.add(tf.keras.layers.Dense(units, activation="relu", **kwargs))
0050 |         model.add(tf.keras.layers.Dropout(dropout_rate))
0051 |     model.add(tf.keras.layers.Dense(1, activation="sigmoid"))
0052 |     model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
0053 |     return model

```

Function: `train_dnn_suite`

Lines: 56 to 145

Description: Analyzes and executes `train_dnn_suite` logic.

```

0056 | def train_dnn_suite(job_id: str, config: dict[str, object] | None = None) -> dict[str, object]:
0057 |     tf = _tensorflow()
0058 |
0059 |     resolved = resolve_dnn_config(config)
0060 |     tf.random.set_seed(int(resolved.get("random_state", 42)))
0061 |     np.random.seed(int(resolved.get("random_state", 42)))
0062 |
0063 |     split = load_dnn_split(
0064 |         train_sample=resolved.get("train_sample"),
0065 |         test_sample=resolved.get("test_sample"),
0066 |         random_state=int(resolved.get("random_state", 42)),
0067 |     )
0068 |
0069 |     generated_run_id = f"dnn-{uuid4().hex[:12]}"
0070 |     target_dir = run_dir(generated_run_id)
0071 |     models_dir = target_dir / "models"
0072 |     history_dir = target_dir / "history"
0073 |     predictions_dir = target_dir / "predictions"
0074 |     models_dir.mkdir(parents=True, exist_ok=True)
0075 |     history_dir.mkdir(parents=True, exist_ok=True)
0076 |     predictions_dir.mkdir(parents=True, exist_ok=True)
0077 |
0078 |     y_test = split["y_test"]
0079 |     results = []
0080 |
0081 |     for layer_count in resolved["architectures"]:
0082 |         model = _build_model(layer_count)
0083 |         checkpoint_path = models_dir / f"dnn{layer_count}_best.keras"
0084 |         csv_logger_path = history_dir / f"dnn{layer_count}_history.csv"
0085 |
0086 |         callbacks = [
0087 |             tf.keras.callbacks.ModelCheckpoint(filepath=str(checkpoint_path), save_best_only=True, monitor="loss", v
0088 |             tf.keras.callbacks.CSVLogger(str(csv_logger_path), separator=";", append=False),
0089 |         ]
0090 |
0091 |         started = time.perf_counter()
0092 |         history = model.fit(
0093 |             split["X_train"],
0094 |             split["y_train"],
0095 |             batch_size=int(resolved["batch_size"]),
0096 |             epochs=int(resolved["epochs"]),
0097 |             verbose=0,
0098 |             callbacks=callbacks,
0099 |         )
0100 |         training_seconds = round(time.perf_counter() - started, 3)
0101 |         best_model = tf.keras.models.load_model(checkpoint_path, compile=False)
0102 |         probabilities = best_model.predict(split["X_test"], verbose=0).reshape(-1)
0103 |         predicted = (probabilities >= 0.5).astype(int)
0104 |         metrics = binary_metrics(y_test, predicted)
0105 |
0106 |         model_path = models_dir / f"dnn{layer_count}.keras"
0107 |         best_model.save(model_path)
0108 |         np.savetxt(predictions_dir / f"dnn{layer_count}_labels.txt", predicted, fmt="%01d")
0109 |         np.savetxt(predictions_dir / f"dnn{layer_count}_probabilities.txt", probabilities)
0110 |
0111 |         results.append(
0112 |             {
0113 |                 "id": f"dnn-{layer_count}_layer",
0114 |                 "label": f"DNN {layer_count} Layer",
0115 |                 "training_seconds": training_seconds,
0116 |                 "metrics": metrics,
0117 |                 "probability_summary": probability_summary(probabilities),
0118 |                 "history": {
0119 |                     "epochs": int(len(history.history["loss"])),
0120 |                     "best_accuracy": round(float(max(history.history["accuracy"])), 6),
0121 |                     "best_loss": round(float(min(history.history["loss"])), 6),
0122 |                     "final_accuracy": round(float(history.history["accuracy"][-1]), 6),
0123 |                     "final_loss": round(float(history.history["loss"][-1]), 6),

```

```

0124 |         },
0125 |         "model_path": str(model_path.relative_to(target_dir.parent.parent)),
0126 |     }
0127 | )
0128 |
0129 | results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
0130 | summary = {
0131 |     "run_id": generated_run_id,
0132 |     "job_id": job_id,
0133 |     "kind": "dnn_train",
0134 |     "created_at": datetime.now(timezone.utc).isoformat(),
0135 |     "config": resolved,
0136 |     "dataset": {
0137 |         "train_rows": split["train_rows"],
0138 |         "test_rows": split["test_rows"],
0139 |         "feature_count": split["feature_count"],
0140 |     },
0141 |     "results": results,
0142 |     "best_model": results[0]["id"] if results else None,
0143 | }
0144 | write_json(run_summary_path(generated_run_id), summary)
0145 | return summary

```

Module: ./ids_app/jobs.py

Overview

Total Lines: 87

Class: JobManager

Lines: 15 to 83

Description: Analyzes and executes JobManager logic.

```

0015 | class JobManager:
0016 |     def __init__(self) -> None:
0017 |         self._lock = threading.Lock()
0018 |         self._jobs: dict[str, dict[str, Any]] = {}
0019 |         ensure_directories()
0020 |         self._load_existing_jobs()
0021 |
0022 |     def _load_existing_jobs(self) -> None:
0023 |         for path in job_path("").parent.glob("*.json"):
0024 |             payload = read_json(path)
0025 |             if payload:
0026 |                 self._jobs[payload["id"]] = payload
0027 |
0028 |     def _persist(self, payload: dict[str, Any]) -> None:
0029 |         write_json(job_path(payload["id"]), payload)
0030 |
0031 |     def _update(self, job_id: str, **fields: Any) -> None:
0032 |         with self._lock:
0033 |             payload = dict(self._jobs[job_id])
0034 |             payload.update(fields)
0035 |             self._jobs[job_id] = payload
0036 |             self._persist(payload)
0037 |
0038 |     def submit(self, kind: str, config: dict[str, Any], task: JobTask) -> dict[str, Any]:
0039 |         created = {
0040 |             "id": uuid4().hex,
0041 |             "kind": kind,
0042 |             "status": "queued",
0043 |             "config": config,
0044 |             "created_at": datetime.now(timezone.utc).isoformat(),
0045 |         }
0046 |         with self._lock:
0047 |             self._jobs[created["id"]] = created
0048 |             self._persist(created)
0049 |
0050 |         thread = threading.Thread(target=self._run_job, args=(created["id"], task), daemon=True)
0051 |         thread.start()
0052 |         return created
0053 |
0054 |     def _run_job(self, job_id: str, task: JobTask) -> None:
0055 |         payload = self._jobs[job_id]
0056 |         self._update(job_id, status="running", started_at=datetime.now(timezone.utc).isoformat())
0057 |         try:
0058 |             result = task(job_id, payload["config"])
0059 |             self._update(
0060 |                 job_id,
0061 |                 status="completed",
0062 |                 completed_at=datetime.now(timezone.utc).isoformat(),
0063 |                 run_id=result.get("run_id"),
0064 |                 summary=result,
0065 |             )
0066 |         except Exception as exc: # pragma: no cover - surfaced to UI
0067 |             self._update(
0068 |                 job_id,
0069 |                 status="failed",
0070 |                 completed_at=datetime.now(timezone.utc).isoformat(),
0071 |                 error=str(exc),
0072 |                 traceback=traceback.format_exc(),
0073 |             )
0074 |
0075 |     def get(self, job_id: str) -> dict[str, Any] | None:
0076 |         with self._lock:
0077 |             payload = self._jobs.get(job_id)
0078 |             return dict(payload) if payload else None
0079 |
0080 |     def list(self, limit: int = 20) -> list[dict[str, Any]]:
0081 |         with self._lock:

```

```
0082 |         jobs = sorted(self._jobs.values(), key=lambda item: item.get("created_at", ""), reverse=True)
0083 |         return [dict(job) for job in jobs[:limit]]
```

Function: `__init__`

Lines: 16 to 20

Description: Analyzes and executes `init` logic.

```
0016 |     def __init__(self) -> None:
0017 |         self._lock = threading.Lock()
0018 |         self._jobs: dict[str, dict[str, Any]] = {}
0019 |         ensure_directories()
0020 |         self._load_existing_jobs()
```

Function: `_load_existing_jobs`

Lines: 22 to 26

Description: Analyzes and executes `_load_existing_jobs` logic.

```
0022 |     def _load_existing_jobs(self) -> None:
0023 |         for path in job_path("").parent.glob("*.json"):
0024 |             payload = read_json(path)
0025 |             if payload:
0026 |                 self._jobs[payload["id"]] = payload
```

Function: `_persist`

Lines: 28 to 29

Description: Analyzes and executes `_persist` logic.

```
0028 |     def _persist(self, payload: dict[str, Any]) -> None:
0029 |         write_json(job_path(payload["id"]), payload)
```

Function: `_update`

Lines: 31 to 36

Description: Analyzes and executes `_update` logic.

```
0031 |     def _update(self, job_id: str, **fields: Any) -> None:
0032 |         with self._lock:
0033 |             payload = dict(self._jobs[job_id])
0034 |             payload.update(fields)
0035 |             self._jobs[job_id] = payload
0036 |             self._persist(payload)
```

Function: `submit`

Lines: 38 to 52

Description: Analyzes and executes `submit` logic.

```

0038 |     def submit(self, kind: str, config: dict[str, Any], task: JobTask) -> dict[str, Any]:
0039 |         created = {
0040 |             "id": uuid4().hex,
0041 |             "kind": kind,
0042 |             "status": "queued",
0043 |             "config": config,
0044 |             "created_at": datetime.now(timezone.utc).isoformat(),
0045 |         }
0046 |         with self._lock:
0047 |             self._jobs[created["id"]] = created
0048 |             self._persist(created)
0049 |
0050 |         thread = threading.Thread(target=self._run_job, args=(created["id"], task), daemon=True)
0051 |         thread.start()
0052 |         return created

```

Function: `_run_job`

Lines: 54 to 73

Description: Analyzes and executes `_run_job` logic.

```

0054 |     def _run_job(self, job_id: str, task: JobTask) -> None:
0055 |         payload = self._jobs[job_id]
0056 |         self._update(job_id, status="running", started_at=datetime.now(timezone.utc).isoformat())
0057 |         try:
0058 |             result = task(job_id, payload["config"])
0059 |             self._update(
0060 |                 job_id,
0061 |                 status="completed",
0062 |                 completed_at=datetime.now(timezone.utc).isoformat(),
0063 |                 run_id=result.get("run_id"),
0064 |                 summary=result,
0065 |             )
0066 |         except Exception as exc: # pragma: no cover - surfaced to UI
0067 |             self._update(
0068 |                 job_id,
0069 |                 status="failed",
0070 |                 completed_at=datetime.now(timezone.utc).isoformat(),
0071 |                 error=str(exc),
0072 |                 traceback=traceback.format_exc(),
0073 |             )

```

Function: `get`

Lines: 75 to 78

Description: Analyzes and executes `get` logic.

```

0075 |     def get(self, job_id: str) -> dict[str, Any] | None:
0076 |         with self._lock:
0077 |             payload = self._jobs.get(job_id)
0078 |             return dict(payload) if payload else None

```

Function: `list`

Lines: 80 to 83

Description: Analyzes and executes `list` logic.

```
0080 |     def list(self, limit: int = 20) -> list[dict[str, Any]]:
0081 |         with self._lock:
0082 |             jobs = sorted(self._jobs.values(), key=lambda item: item.get("created_at", ""), reverse=True)
0083 |             return [dict(job) for job in jobs[:limit]]
```

Module: ./ids_app/legacy.py

Overview

Total Lines: 90

Function: _load_labels

Lines: 13 to 14

Description: Analyzes and executes _load_labels logic.

```
0013 | def _load_labels(path) -> np.ndarray:
0014 |     return np.loadtxt(path).astype(int).reshape(-1)
```

Function: _load_history

Lines: 17 to 32

Description: Analyzes and executes _load_history logic.

```
0017 | def _load_history(path) -> dict[str, float | int] | None:
0018 |     if not path.exists():
0019 |         return None
0020 |     try:
0021 |         frame = pd.read_csv(path)
0022 |     except pd.errors.EmptyDataError:
0023 |         return None
0024 |     if frame.empty:
0025 |         return None
0026 |     return {
0027 |         "epochs_logged": int(len(frame)),
0028 |         "best_accuracy": round(float(frame["accuracy"].max()), 6),
0029 |         "best_loss": round(float(frame["loss"].min()), 6),
0030 |         "final_accuracy": round(float(frame["accuracy"].iloc[-1]), 6),
0031 |         "final_loss": round(float(frame["loss"].iloc[-1]), 6),
0032 |     }
```

Function: evaluate_legacy_predictions

Lines: 36 to 90

Description: Analyzes and executes evaluate_legacy_predictions logic.

```

0036 | def evaluate_legacy_predictions() -> dict[str, object]:
0037 |     classical_expected_path = ROOT_DIR / "classical" / "expected.txt"
0038 |     dnn_expected_path = ROOT_DIR / "dnn" / "dnnres" / "expected.txt"
0039 |     if not dnn_expected_path.exists():
0040 |         dnn_expected_path = classical_expected_path
0041 |
0042 |     classical_expected = _load_labels(classical_expected_path)
0043 |     dnn_expected = _load_labels(dnn_expected_path)
0044 |
0045 |     classical_results = []
0046 |     for slug, (label, path) in LEGACY_CLASSICAL_FILES.items():
0047 |         if not path.exists():
0048 |             continue
0049 |         metrics = binary_metrics(classical_expected, _load_labels(path))
0050 |         classical_results.append(
0051 |             {
0052 |                 "id": slug,
0053 |                 "label": label,
0054 |                 "source": str(path.relative_to(ROOT_DIR)),
0055 |                 "metrics": metrics,
0056 |             }
0057 |         )
0058 |
0059 |     dnn_results = []
0060 |     for slug, (label, path) in LEGACY_DNN_FILES.items():
0061 |         if not path.exists():
0062 |             continue
0063 |         metrics = binary_metrics(dnn_expected, _load_labels(path))
0064 |         dnn_results.append(
0065 |             {
0066 |                 "id": slug,
0067 |                 "label": label,
0068 |                 "source": str(path.relative_to(ROOT_DIR)),
0069 |                 "metrics": metrics,
0070 |             }
0071 |         )
0072 |
0073 |     history_map = {
0074 |         "legacy_dnn1": ROOT_DIR / "dnn" / "kddresults" / "dnn1layer" / "training_set_dnnanalysis.csv",
0075 |         "legacy_dnn2": ROOT_DIR / "dnn" / "kddresults" / "dnn2layer" / "training_set_dnnanalysis.csv",
0076 |         "legacy_dnn3": ROOT_DIR / "dnn" / "kddresults" / "dnn3layer" / "training_set_dnnanalysis.csv",
0077 |         "legacy_dnn4": ROOT_DIR / "dnn" / "kddresults" / "dnn4layer" / "training_set_dnnanalysis.csv",
0078 |         "legacy_dnn5": ROOT_DIR / "dnn" / "kddresults" / "dnn5layer" / "training_set_dnnanalysis.csv",
0079 |     }
0080 |     for result in dnn_results:
0081 |         result["history"] = _load_history(history_map[result["id"]])
0082 |
0083 |     classical_results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
0084 |     dnn_results.sort(key=lambda item: item["metrics"]["f1"], reverse=True)
0085 |
0086 |     return {
0087 |         "generated_at": datetime.now(timezone.utc).isoformat(),
0088 |         "classical": classical_results,
0089 |         "dnn": dnn_results,
0090 |     }

```

Module: ./ids_app/metrics.py

Overview

Total Lines: 33

Function: binary_metrics

Lines: 7 to 21

Description: Analyzes and executes binary_metrics logic.

```
0007 | def binary_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float | int]:
0008 |     y_true = np.asarray(y_true).astype(int).reshape(-1)
0009 |     y_pred = np.asarray(y_pred).astype(int).reshape(-1)
0010 |     tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
0011 |     return {
0012 |         "accuracy": round(float(accuracy_score(y_true, y_pred)), 6),
0013 |         "precision": round(float(precision_score(y_true, y_pred, zero_division=0)), 6),
0014 |         "recall": round(float(recall_score(y_true, y_pred, zero_division=0)), 6),
0015 |         "f1": round(float(f1_score(y_true, y_pred, zero_division=0)), 6),
0016 |         "support": int(y_true.size),
0017 |         "tp": int(tp),
0018 |         "tn": int(tn),
0019 |         "fp": int(fp),
0020 |         "fn": int(fn),
0021 |     }
```

Function: probability_summary

Lines: 24 to 32

Description: Analyzes and executes probability_summary logic.

```
0024 | def probability_summary(values: np.ndarray | None) -> dict[str, float] | None:
0025 |     if values is None:
0026 |         return None
0027 |     flattened = np.asarray(values, dtype=float).reshape(-1)
0028 |     return {
0029 |         "min": round(float(flattened.min()), 6),
0030 |         "mean": round(float(flattened.mean()), 6),
0031 |         "max": round(float(flattened.max()), 6),
0032 |     }
```

Module: ./ids_app/product_app.py

Overview

Total Lines: 18

Function: main

Lines: 8 to 14

Description: Analyzes and executes main logic.

```
0008 | def main(argv: list[str] | None = None) -> int:
0009 |     args = list(sys.argv[1:] if argv is None else argv)
0010 |     if args and args[0].lower() == "gui":
0011 |         from .product_gui import main as gui_main
0012 |
0013 |         return gui_main(args[1:])
0014 |     return product_terminal.main(args)
```

Module: `./ids_app/product_gui.py`

Overview

Total Lines: 348

Class: `IDSPProductGUI`

Lines: 14 to 336

Description: Analyzes and executes IDSPProductGUI logic.

```

0014 | class IDSPProductGUI:
0015 |     def __init__(self, root: tk.Tk) -> None:
0016 |         self.root = root
0017 |         self.root.title("IDS Sentinel Terminal")
0018 |         self.root.geometry("1180x760")
0019 |         self.root.minsize(920, 620)
0020 |         self.output_queue: queue.Queue[str] = queue.Queue()
0021 |         self.palette = {
0022 |             "bg": "#0c1222",
0023 |             "sidebar": "#101a31",
0024 |             "panel": "#151f38",
0025 |             "panel_alt": "#1a2747",
0026 |             "text": "#e6edf9",
0027 |             "muted": "#9eae00",
0028 |             "accent": "#2f81f7",
0029 |             "accent_hover": "#4d95ff",
0030 |             "accent_pressed": "#1e6fe3",
0031 |             "button": "#1f2d4f",
0032 |             "button_hover": "#2b3d67",
0033 |             "button_pressed": "#1a2948",
0034 |             "border": "#2d3f66",
0035 |             "output_bg": "#0d162e",
0036 |             "output_text": "#dde7fd",
0037 |             "success": "#56d39f",
0038 |             "warning": "#f2c26b",
0039 |             "danger": "#ff7b72",
0040 |         }
0041 |
0042 |         self.command_var = tk.StringVar(value="status")
0043 |         self.scan_path_var = tk.StringVar(value="kddtest.csv")
0044 |         self.scan_limit_var = tk.StringVar(value="5000")
0045 |         self.hunt_var = tk.StringVar(value="dos_flood")
0046 |         self.host_var = tk.StringVar(value="127.0.0.1")
0047 |         self.ports_var = tk.StringVar(value="common")
0048 |         self.file_path_var = tk.StringVar(value="automation/product/self_learning_model.json")
0049 |
0050 |         self._configure_theme()
0051 |         self._build_layout()
0052 |         self.root.after(100, self._drain_output)
0053 |         self.run_command(["status"])
0054 |
0055 |     def _configure_theme(self) -> None:
0056 |         style = ttk.Style(self.root)
0057 |         with contextlib.suppress(tk.TclError):
0058 |             style.theme_use("clam")
0059 |
0060 |         p = self.palette
0061 |         self.root.configure(bg=p["bg"])
0062 |         style.configure(".", background=p["bg"], foreground=p["text"], font=("Segoe UI", 10))
0063 |         style.configure("Sidebar.TFrame", background=p["sidebar"])
0064 |         style.configure("Main.TFrame", background=p["bg"])
0065 |         style.configure("Tab.TFrame", background=p["panel"])
0066 |
0067 |         style.configure("SidebarTitle.TLabel", background=p["sidebar"], foreground=p["text"], font=("Segoe UI Semibold", 12))
0068 |         style.configure("SidebarSub.TLabel", background=p["sidebar"], foreground=p["muted"], font=("Segoe UI", 9))
0069 |         style.configure("Sidebar.TLabel", background=p["sidebar"], foreground=p["text"])
0070 |         style.configure("Panel.TLabel", background=p["panel"], foreground=p["text"])
0071 |         style.configure("TSeparator", background=p["border"])
0072 |
0073 |         style.configure("Sidebar.TButton", padding=(12, 7), background=p["button"], foreground=p["text"], borderwidth=1)
0074 |         style.map(
0075 |             "Sidebar.TButton",
0076 |             background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
0077 |             foreground=[("disabled", p["muted"])],
0078 |         )
0079 |
0080 |         style.configure("Primary.TButton", padding=(12, 7), background=p["accent"], foreground=p["text"], borderwidth=1)
0081 |         style.map(

```

```

0082 |         "Primary.TButton",
0083 |         background=[("pressed", p["accent_pressed"]), ("active", p["accent_hover"])],
0084 |         foreground=[("disabled", p["muted"])],
0085 |     )
0086 |
0087 |     style.configure("Tool.TButton", padding=(10, 6), background=p["button"], foreground=p["text"], borderwidth=0)
0088 |     style.map(
0089 |         "Tool.TButton",
0090 |         background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
0091 |         foreground=[("disabled", p["muted"])],
0092 |     )
0093 |
0094 |     style.configure(
0095 |         "TEntry",
0096 |         fieldbackground=p["panel_alt"],
0097 |         foreground=p["text"],
0098 |         bordercolor=p["border"],
0099 |         lightcolor=p["border"],
0100 |         darkcolor=p["border"],
0101 |         insertcolor=p["text"],
0102 |         padding=6,
0103 |     )
0104 |     style.map("TEntry", fieldbackground=[("readonly", p["panel_alt"])])
0105 |
0106 |     style.configure("App.TNotebook", background=p["bg"], borderwidth=0, tabmargins=(0, 0, 0, 0))
0107 |     style.configure("TNotebook.Tab", background=p["panel"], foreground=p["muted"], padding=(14, 7), borderwidth=0)
0108 |     style.map(
0109 |         "TNotebook.Tab",
0110 |         background=[("selected", p["accent"]), ("active", p["panel_alt"])],
0111 |         foreground=[("selected", p["text"]), ("active", p["text"])],
0112 |     )
0113 |
0114 |     style.configure(
0115 |         "Vertical.TScrollbar",
0116 |         background=p["panel_alt"],
0117 |         troughcolor=p["panel"],
0118 |         bordercolor=p["border"],
0119 |         arrowcolor=p["muted"],
0120 |     )
0121 |     style.configure(
0122 |         "Horizontal.TScrollbar",
0123 |         background=p["panel_alt"],
0124 |         troughcolor=p["panel"],
0125 |         bordercolor=p["border"],
0126 |         arrowcolor=p["muted"],
0127 |     )
0128 |
0129 |     def _build_layout(self) -> None:
0130 |         self.root.columnconfigure(0, weight=0)
0131 |         self.root.columnconfigure(1, weight=1)
0132 |         self.root.rowconfigure(0, weight=1)
0133 |
0134 |         sidebar = ttk.Frame(self.root, padding=14, style="Sidebar.TFrame")
0135 |         sidebar.grid(row=0, column=0, sticky="ns")
0136 |         sidebar.configure(width=248)
0137 |
0138 |         main = ttk.Frame(self.root, padding=(0, 12, 12, 12), style="Main.TFrame")
0139 |         main.grid(row=0, column=1, sticky="nsew")
0140 |         main.columnconfigure(0, weight=1)
0141 |         main.rowconfigure(1, weight=1)
0142 |
0143 |         ttk.Label(sidebar, text="IDS Sentinel Terminal", style="SidebarTitle.TLabel").pack(anchor="w", pady=(4, 2))
0144 |         ttk.Label(sidebar, text="Traffic and Threat Intelligence", style="SidebarSub.TLabel").pack(anchor="w", pady=2)
0145 |         for label, command in [
0146 |             ("Status", ["status"]),
0147 |             ("Traffic", ["traffic"]),
0148 |             ("Attacks", ["attacks"]),
0149 |             ("Malware Signals", ["malware", "--limit", "5000"]),
0150 |             ("Datasets", ["datasets"]),

```

```

0151 |         ("Reports", ["reports", "--limit", "20"]),
0152 |         ("Cache", ["cache", "--limit", "20"]),
0153 |         ("Local Ports", ["ports", "--limit", "25"]),
0154 |         ("Processes", ["ps", "--limit", "25"]),
0155 |     ]:
0156 |         ttk.Button(sidebar, text=label, style="Sidebar.TButton", command=lambda cmd=command: self.run_command(cmd)).pack(fill="x", pady=12)
0157 |
0158 |     ttk.Separator(sidebar).pack(fill="x", pady=12)
0159 |     ttk.Button(sidebar, text="Learn Model", style="Primary.TButton", command=lambda: self.run_command(["learn"]))
0160 |     ttk.Button(sidebar, text="Clear Output", style="Sidebar.TButton", command=self.clear_output).pack(fill="x", pady=12)
0161 |     ttk.Label(sidebar, text="Theme: Dark Ops UI", style="SidebarSub.TLabel").pack(anchor="w", pady=(12, 0))
0162 |
0163 |     controls = ttk.Notebook(main, style="App.TNotebook")
0164 |     controls.grid(row=0, column=0, sticky="ew", pady=(0, 8))
0165 |
0166 |     self._build_command_tab(controls)
0167 |     self._build_scan_tab(controls)
0168 |     self._build_hunt_tab(controls)
0169 |     self._build_network_tab(controls)
0170 |     self._build_file_tab(controls)
0171 |
0172 |     self.output = tk.Text(
0173 |         main,
0174 |         wrap="none",
0175 |         font=("Consolas", 10),
0176 |         undo=False,
0177 |         bg=self.palette["output_bg"],
0178 |         fg=self.palette["output_text"],
0179 |         insertbackground=self.palette["text"],
0180 |         selectbackground=self.palette["accent"],
0181 |         selectforeground=self.palette["text"],
0182 |         relief="flat",
0183 |         borderwidth=0,
0184 |         highlightthickness=1,
0185 |         highlightbackground=self.palette["border"],
0186 |         highlightcolor=self.palette["accent"],
0187 |         padx=10,
0188 |         pady=10,
0189 |     )
0190 |     self.output.grid(row=1, column=0, sticky="nsew")
0191 |     self.output.tag_configure("command", foreground=self.palette["accent"])
0192 |     self.output.tag_configure("success", foreground=self.palette["success"])
0193 |     self.output.tag_configure("warning", foreground=self.palette["warning"])
0194 |     self.output.tag_configure("danger", foreground=self.palette["danger"])
0195 |
0196 |     y_scroll = ttk.Scrollbar(main, orient="vertical", command=self.output.yview, style="Vertical.TScrollbar")
0197 |     y_scroll.grid(row=1, column=1, sticky="ns")
0198 |     self.output.config(yscrollcommand=y_scroll.set)
0199 |
0200 |     x_scroll = ttk.Scrollbar(main, orient="horizontal", command=self.output.xview, style="Horizontal.TScrollbar")
0201 |     x_scroll.grid(row=2, column=0, sticky="ew")
0202 |     self.output.config(xscrollcommand=x_scroll.set)
0203 |
0204 |     def _build_command_tab(self, notebook: ttk.Notebook) -> None:
0205 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0206 |         tab.columnconfigure(1, weight=1)
0207 |         notebook.add(tab, text="Command")
0208 |         ttk.Label(tab, text="Command", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0209 |         ttk.Entry(tab, textvariable=self.command_var).grid(row=0, column=1, sticky="ew")
0210 |         ttk.Button(tab, text="Run", style="Primary.TButton", command=self.run_freeform_command).grid(row=0, column=2, sticky="w", padx=(12, 8))
0211 |
0212 |     def _build_scan_tab(self, notebook: ttk.Notebook) -> None:
0213 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0214 |         tab.columnconfigure(1, weight=1)
0215 |         notebook.add(tab, text="Scan")
0216 |         ttk.Label(tab, text="CSV", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0217 |         ttk.Entry(tab, textvariable=self.scan_path_var).grid(row=0, column=1, sticky="ew")
0218 |         ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_scan_file).grid(row=0, column=2, sticky="w", padx=(12, 8))
0219 |         ttk.Label(tab, text="Limit", style="Panel.TLabel").grid(row=0, column=3, sticky="w", padx=(12, 8))

```

```

0220 |         ttk.Entry(tab, width=10, textvariable=self.scan_limit_var).grid(row=0, column=4, sticky="w")
0221 |         ttk.Button(tab, text="Scan", style="Primary.TButton", command=self.run_scan).grid(row=0, column=5, padx=(8,
0222 |
0223 |     def _build_hunt_tab(self, notebook: ttk.Notebook) -> None:
0224 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0225 |         tab.columnconfigure(1, weight=1)
0226 |         notebook.add(tab, text="Hunt")
0227 |         ttk.Label(tab, text="Term", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0228 |         ttk.Entry(tab, textvariable=self.hunt_var).grid(row=0, column=1, sticky="ew")
0229 |         ttk.Button(
0230 |             tab,
0231 |             text="Hunt",
0232 |             style="Primary.TButton",
0233 |             command=lambda: self.run_command(["hunt", self.hunt_var.get(), "--limit", "20"]),
0234 |         ).grid(row=0, column=2, padx=(8, 0))
0235 |
0236 |     def _build_network_tab(self, notebook: ttk.Notebook) -> None:
0237 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0238 |         tab.columnconfigure(1, weight=1)
0239 |         notebook.add(tab, text="Network")
0240 |         ttk.Label(tab, text="Host", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0241 |         ttk.Entry(tab, textvariable=self.host_var).grid(row=0, column=1, sticky="ew")
0242 |         ttk.Label(tab, text="Ports", style="Panel.TLabel").grid(row=0, column=2, sticky="w", padx=(12, 8))
0243 |         ttk.Entry(tab, width=18, textvariable=self.ports_var).grid(row=0, column=3, sticky="w")
0244 |         ttk.Button(
0245 |             tab,
0246 |             text="Probe",
0247 |             style="Primary.TButton",
0248 |             command=lambda: self.run_command(["probe", self.host_var.get(), self.ports_var.get()]),
0249 |         ).grid(row=0, column=4, padx=(8, 0))
0250 |         ttk.Button(tab, text="DNS", style="Tool.TButton", command=lambda: self.run_command(["dns", self.host_var.get()]))
0251 |
0252 |     def _build_file_tab(self, notebook: ttk.Notebook) -> None:
0253 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0254 |         tab.columnconfigure(1, weight=1)
0255 |         notebook.add(tab, text="File")
0256 |         ttk.Label(tab, text="File", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0257 |         ttk.Entry(tab, textvariable=self.file_path_var).grid(row=0, column=1, sticky="ew")
0258 |         ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_file).grid(row=0, column=2, padx=(8, 0))
0259 |         ttk.Button(tab, text="Hash", style="Tool.TButton", command=lambda: self.run_command(["hash", self.file_path_var.get()]))
0260 |         ttk.Button(tab, text="Scan", style="Primary.TButton", command=lambda: self.run_command(["filescan", self.file_path_var.get()]))
0261 |
0262 |     def choose_scan_file(self) -> None:
0263 |         path = filedialog.askopenfilename(title="Choose traffic CSV", filetypes=[("CSV files", "*.csv"), ("All files", "*.*")])
0264 |         if path:
0265 |             self.scan_path_var.set(self._display_path(path))
0266 |
0267 |     def choose_file(self) -> None:
0268 |         path = filedialog.askopenfilename(title="Choose file")
0269 |         if path:
0270 |             self.file_path_var.set(self._display_path(path))
0271 |
0272 |     def _display_path(self, path: str) -> str:
0273 |         try:
0274 |             return str(Path(path).resolve().relative_to(product_terminal.ROOT_DIR))
0275 |         except ValueError:
0276 |             return path
0277 |
0278 |     def run_freeform_command(self) -> None:
0279 |         try:
0280 |             args = product_terminal.split_shell_command(self.command_var.get())
0281 |         except ValueError as exc:
0282 |             messagebox.showerror("Command parse error", str(exc))
0283 |             return
0284 |         self.run_command(args)
0285 |
0286 |     def run_scan(self) -> None:
0287 |         args = ["scan", self.scan_path_var.get()]
0288 |         limit = self.scan_limit_var.get().strip()

```

```

0289 |         if limit.lower() == "all":
0290 |             args.append("--all")
0291 |         elif limit:
0292 |             args.extend(["--limit", limit])
0293 |         self.run_command(args)
0294 |
0295 |     def run_command(self, args: list[str]) -> None:
0296 |         self._append(f"\n$ ids-sentinel-terminal {' '.join(args)}\n")
0297 |         thread = threading.Thread(target=self._run_command_worker, args=(args,), daemon=True)
0298 |         thread.start()
0299 |
0300 |     def _run_command_worker(self, args: list[str]) -> None:
0301 |         stream = io.StringIO()
0302 |         with contextlib.redirect_stdout(stream), contextlib.redirect_stderr(stream):
0303 |             code = product_terminal.main(args)
0304 |             text = stream.getvalue()
0305 |             if code:
0306 |                 text += f"\nCommand exited with code {code}\n"
0307 |             self.output_queue.put(text)
0308 |
0309 |     def _drain_output(self) -> None:
0310 |         try:
0311 |             while True:
0312 |                 self._append(self.output_queue.get_nowait())
0313 |         except queue.Empty:
0314 |             pass
0315 |         self.root.after(100, self._drain_output)
0316 |
0317 |     def _append(self, text: str) -> None:
0318 |         for line in text.splitlines(keepends=True):
0319 |             tag = None
0320 |             normalized = line.lower()
0321 |             if line.lstrip().startswith("$ ids-sentinel-terminal"):
0322 |                 tag = "command"
0323 |             elif any(token in normalized for token in ("error", "exception", "traceback", "not recognized", "failed")):
0324 |                 tag = "danger"
0325 |             elif any(token in normalized for token in ("critical", "high", "warning", "suspicious")):
0326 |                 tag = "warning"
0327 |             elif any(token in normalized for token in ("healthy", "no threats", "completed", "success")):
0328 |                 tag = "success"
0329 |             if tag:
0330 |                 self.output.insert("end", line, tag)
0331 |             else:
0332 |                 self.output.insert("end", line)
0333 |             self.output.see("end")
0334 |
0335 |     def clear_output(self) -> None:
0336 |         self.output.delete("1.0", "end")

```

Function: main

Lines: 339 to 344

Description: Analyzes and executes main logic.

```

0339 | def main(argv: list[str] | None = None) -> int:
0340 |     del argv
0341 |     root = tk.Tk()
0342 |     IDSPProductGUI(root)
0343 |     root.mainloop()
0344 |     return 0

```

Function: __init__

Lines: 15 to 53

Description: Analyzes and executes `init` logic.

```
0015 |     def __init__(self, root: tk.Tk) -> None:
0016 |         self.root = root
0017 |         self.root.title("IDS Sentinel Terminal")
0018 |         self.root.geometry("1180x760")
0019 |         self.root.minsize(920, 620)
0020 |         self.output_queue: queue.Queue[str] = queue.Queue()
0021 |         self.palette = {
0022 |             "bg": "#0c1222",
0023 |             "sidebar": "#101a31",
0024 |             "panel": "#151f38",
0025 |             "panel_alt": "#1a2747",
0026 |             "text": "#e6edf9",
0027 |             "muted": "#9eaed0",
0028 |             "accent": "#2f81f7",
0029 |             "accent_hover": "#4d95ff",
0030 |             "accent_pressed": "#1e6fe3",
0031 |             "button": "#1f2d4f",
0032 |             "button_hover": "#2b3d67",
0033 |             "button_pressed": "#1a2948",
0034 |             "border": "#2d3f66",
0035 |             "output_bg": "#0d162e",
0036 |             "output_text": "#dde7fd",
0037 |             "success": "#56d39f",
0038 |             "warning": "#f2c26b",
0039 |             "danger": "#ff7b72",
0040 |         }
0041 |
0042 |         self.command_var = tk.StringVar(value="status")
0043 |         self.scan_path_var = tk.StringVar(value="kddtest.csv")
0044 |         self.scan_limit_var = tk.StringVar(value="5000")
0045 |         self.hunt_var = tk.StringVar(value="dos_flood")
0046 |         self.host_var = tk.StringVar(value="127.0.0.1")
0047 |         self.ports_var = tk.StringVar(value="common")
0048 |         self.file_path_var = tk.StringVar(value="automation/product/self_learning_model.json")
0049 |
0050 |         self._configure_theme()
0051 |         self._build_layout()
0052 |         self.root.after(100, self._drain_output)
0053 |         self.run_command(["status"])
```

Function: `_configure_theme`

Lines: 55 to 127

Description: Analyzes and executes `_configure_theme` logic.

```

0055 | def _configure_theme(self) -> None:
0056 |     style = ttk.Style(self.root)
0057 |     with contextlib.suppress(tk.TclError):
0058 |         style.theme_use("clam")
0059 |
0060 |     p = self.palette
0061 |     self.root.configure(bg=p["bg"])
0062 |     style.configure(".", background=p["bg"], foreground=p["text"], font=("Segoe UI", 10))
0063 |     style.configure("Sidebar.TFrame", background=p["sidebar"])
0064 |     style.configure("Main.TFrame", background=p["bg"])
0065 |     style.configure("Tab.TFrame", background=p["panel"])
0066 |
0067 |     style.configure("SidebarTitle.TLabel", background=p["sidebar"], foreground=p["text"], font=("Segoe UI Semibold", 10))
0068 |     style.configure("SidebarSub.TLabel", background=p["sidebar"], foreground=p["muted"], font=("Segoe UI", 9))
0069 |     style.configure("Sidebar.TLabel", background=p["sidebar"], foreground=p["text"])
0070 |     style.configure("Panel.TLabel", background=p["panel"], foreground=p["text"])
0071 |     style.configure("TSeparator", background=p["border"])
0072 |
0073 |     style.configure("Sidebar.TButton", padding=(12, 7), background=p["button"], foreground=p["text"], borderwidth=1)
0074 |     style.map(
0075 |         "Sidebar.TButton",
0076 |         background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
0077 |         foreground=[("disabled", p["muted"])],
0078 |     )
0079 |
0080 |     style.configure("Primary.TButton", padding=(12, 7), background=p["accent"], foreground=p["text"], borderwidth=1)
0081 |     style.map(
0082 |         "Primary.TButton",
0083 |         background=[("pressed", p["accent_pressed"]), ("active", p["accent_hover"])],
0084 |         foreground=[("disabled", p["muted"])],
0085 |     )
0086 |
0087 |     style.configure("Tool.TButton", padding=(10, 6), background=p["button"], foreground=p["text"], borderwidth=1)
0088 |     style.map(
0089 |         "Tool.TButton",
0090 |         background=[("pressed", p["button_pressed"]), ("active", p["button_hover"])],
0091 |         foreground=[("disabled", p["muted"])],
0092 |     )
0093 |
0094 |     style.configure(
0095 |         "TEntry",
0096 |         fieldbackground=p["panel_alt"],
0097 |         foreground=p["text"],
0098 |         bordercolor=p["border"],
0099 |         lightcolor=p["border"],
0100 |         darkcolor=p["border"],
0101 |         insertcolor=p["text"],
0102 |         padding=6,
0103 |     )
0104 |     style.map("TEntry", fieldbackground=[("readonly", p["panel_alt"])])
0105 |
0106 |     style.configure("App.TNotebook", background=p["bg"], borderwidth=0, tabmargins=(0, 0, 0, 0))
0107 |     style.configure("TNotebook.Tab", background=p["panel"], foreground=p["muted"], padding=(14, 7), borderwidth=1)
0108 |     style.map(
0109 |         "TNotebook.Tab",
0110 |         background=[("selected", p["accent"]), ("active", p["panel_alt"])],
0111 |         foreground=[("selected", p["text"]), ("active", p["text"])],
0112 |     )
0113 |
0114 |     style.configure(
0115 |         "Vertical.TScrollbar",
0116 |         background=p["panel_alt"],
0117 |         troughcolor=p["panel"],
0118 |         bordercolor=p["border"],
0119 |         arrowcolor=p["muted"],
0120 |     )
0121 |     style.configure(
0122 |         "Horizontal.TScrollbar",

```

```
0123 |         background=p["panel_alt"],  
0124 |         troughcolor=p["panel"],  
0125 |         bordercolor=p["border"],  
0126 |         arrowcolor=p["muted"],  
0127 |     )
```

Function: `_build_layout`

Lines: 129 to 202

Description: Analyzes and executes `_build_layout` logic.

```

0129 | def _build_layout(self) -> None:
0130 |     self.root.columnconfigure(0, weight=0)
0131 |     self.root.columnconfigure(1, weight=1)
0132 |     self.root.rowconfigure(0, weight=1)
0133 |
0134 |     sidebar = ttk.Frame(self.root, padding=14, style="Sidebar.TFrame")
0135 |     sidebar.grid(row=0, column=0, sticky="ns")
0136 |     sidebar.configure(width=248)
0137 |
0138 |     main = ttk.Frame(self.root, padding=(0, 12, 12, 12), style="Main.TFrame")
0139 |     main.grid(row=0, column=1, sticky="nsew")
0140 |     main.columnconfigure(0, weight=1)
0141 |     main.rowconfigure(1, weight=1)
0142 |
0143 |     ttk.Label(sidebar, text="IDS Sentinel Terminal", style="SidebarTitle.TLabel").pack(anchor="w", pady=(4, 2))
0144 |     ttk.Label(sidebar, text="Traffic and Threat Intelligence", style="SidebarSub.TLabel").pack(anchor="w", pady=
0145 | for label, command in [
0146 |     ("Status", ["status"]),
0147 |     ("Traffic", ["traffic"]),
0148 |     ("Attacks", ["attacks"]),
0149 |     ("Malware Signals", ["malware", "--limit", "5000"]),
0150 |     ("Datasets", ["datasets"]),
0151 |     ("Reports", ["reports", "--limit", "20"]),
0152 |     ("Cache", ["cache", "--limit", "20"]),
0153 |     ("Local Ports", ["ports", "--limit", "25"]),
0154 |     ("Processes", ["ps", "--limit", "25"]),
0155 | ]:
0156 |         ttk.Button(sidebar, text=label, style="Sidebar.TButton", command=lambda cmd=command: self.run_command(cmd))
0157 |
0158 |     ttk.Separator(sidebar).pack(fill="x", pady=12)
0159 |     ttk.Button(sidebar, text="Learn Model", style="Primary.TButton", command=lambda: self.run_command(["learn"]))
0160 |     ttk.Button(sidebar, text="Clear Output", style="Sidebar.TButton", command=self.clear_output).pack(fill="x",
0161 |     ttk.Label(sidebar, text="Theme: Dark Ops UI", style="SidebarSub.TLabel").pack(anchor="w", pady=(12, 0))
0162 |
0163 |     controls = ttk.Notebook(main, style="App.TNotebook")
0164 |     controls.grid(row=0, column=0, sticky="ew", pady=(0, 8))
0165 |
0166 |     self._build_command_tab(controls)
0167 |     self._build_scan_tab(controls)
0168 |     self._build_hunt_tab(controls)
0169 |     self._build_network_tab(controls)
0170 |     self._build_file_tab(controls)
0171 |
0172 |     self.output = tk.Text(
0173 |         main,
0174 |         wrap="none",
0175 |         font=("Consolas", 10),
0176 |         undo=False,
0177 |         bg=self.palette["output_bg"],
0178 |         fg=self.palette["output_text"],
0179 |         insertbackground=self.palette["text"],
0180 |         selectbackground=self.palette["accent"],
0181 |         selectforeground=self.palette["text"],
0182 |         relief="flat",
0183 |         borderwidth=0,
0184 |         highlightthickness=1,
0185 |         highlightbackground=self.palette["border"],
0186 |         highlightcolor=self.palette["accent"],
0187 |         padx=10,
0188 |         pady=10,
0189 |     )
0190 |     self.output.grid(row=1, column=0, sticky="nsew")
0191 |     self.output.tag_configure("command", foreground=self.palette["accent"])
0192 |     self.output.tag_configure("success", foreground=self.palette["success"])
0193 |     self.output.tag_configure("warning", foreground=self.palette["warning"])
0194 |     self.output.tag_configure("danger", foreground=self.palette["danger"])
0195 |
0196 |     y_scroll = ttk.Scrollbar(main, orient="vertical", command=self.output.yview, style="Vertical.TScrollbar")

```

```

0197 |         y_scroll.grid(row=1, column=1, sticky="ns")
0198 |         self.output.configure(yscrollcommand=y_scroll.set)
0199 |
0200 |         x_scroll = ttk.Scrollbar(main, orient="horizontal", command=self.output.xview, style="Horizontal.TScrollbar")
0201 |         x_scroll.grid(row=2, column=0, sticky="ew")
0202 |         self.output.configure(xscrollcommand=x_scroll.set)

```

Function: `_build_command_tab`

Lines: 204 to 210

Description: Analyzes and executes `_build_command_tab` logic.

```

0204 |     def _build_command_tab(self, notebook: ttk.Notebook) -> None:
0205 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0206 |         tab.columnconfigure(1, weight=1)
0207 |         notebook.add(tab, text="Command")
0208 |         ttk.Label(tab, text="Command", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0209 |         ttk.Entry(tab, textvariable=self.command_var).grid(row=0, column=1, sticky="ew")
0210 |         ttk.Button(tab, text="Run", style="Primary.TButton", command=self.run_freeform_command).grid(row=0, column=2, sticky="w", padx=(0, 8))

```

Function: `_build_scan_tab`

Lines: 212 to 221

Description: Analyzes and executes `_build_scan_tab` logic.

```

0212 |     def _build_scan_tab(self, notebook: ttk.Notebook) -> None:
0213 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0214 |         tab.columnconfigure(1, weight=1)
0215 |         notebook.add(tab, text="Scan")
0216 |         ttk.Label(tab, text="CSV", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0217 |         ttk.Entry(tab, textvariable=self.scan_path_var).grid(row=0, column=1, sticky="ew")
0218 |         ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_scan_file).grid(row=0, column=2, sticky="w", padx=(0, 8))
0219 |         ttk.Label(tab, text="Limit", style="Panel.TLabel").grid(row=0, column=3, sticky="w", padx=(12, 8))
0220 |         ttk.Entry(tab, width=10, textvariable=self.scan_limit_var).grid(row=0, column=4, sticky="w")
0221 |         ttk.Button(tab, text="Scan", style="Primary.TButton", command=self.run_scan).grid(row=0, column=5, padx=(8, 0))

```

Function: `_build_hunt_tab`

Lines: 223 to 234

Description: Analyzes and executes `_build_hunt_tab` logic.

```

0223 |     def _build_hunt_tab(self, notebook: ttk.Notebook) -> None:
0224 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0225 |         tab.columnconfigure(1, weight=1)
0226 |         notebook.add(tab, text="Hunt")
0227 |         ttk.Label(tab, text="Term", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0228 |         ttk.Entry(tab, textvariable=self.hunt_var).grid(row=0, column=1, sticky="ew")
0229 |         ttk.Button(
0230 |             tab,
0231 |             text="Hunt",
0232 |             style="Primary.TButton",
0233 |             command=lambda: self.run_command(["hunt", self.hunt_var.get(), "--limit", "20"]),
0234 |         ).grid(row=0, column=2, padx=(8, 0))

```

Function: `_build_network_tab`

Lines: 236 to 250

Description: Analyzes and executes `_build_network_tab` logic.

```
0236 |     def _build_network_tab(self, notebook: ttk.Notebook) -> None:
0237 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0238 |         tab.columnconfigure(1, weight=1)
0239 |         notebook.add(tab, text="Network")
0240 |         ttk.Label(tab, text="Host", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0241 |         ttk.Entry(tab, textvariable=self.host_var).grid(row=0, column=1, sticky="ew")
0242 |         ttk.Label(tab, text="Ports", style="Panel.TLabel").grid(row=0, column=2, sticky="w", padx=(12, 8))
0243 |         ttk.Entry(tab, width=18, textvariable=self.ports_var).grid(row=0, column=3, sticky="w")
0244 |         ttk.Button(
0245 |             tab,
0246 |             text="Probe",
0247 |             style="Primary.TButton",
0248 |             command=lambda: self.run_command(["probe", self.host_var.get(), self.ports_var.get()]),
0249 |         ).grid(row=0, column=4, padx=(8, 0))
0250 |         ttk.Button(tab, text="DNS", style="Tool.TButton", command=lambda: self.run_command(["dns", self.host_var.get(), self.ports_var.get()]))
```

Function: `_build_file_tab`

Lines: 252 to 260

Description: Analyzes and executes `_build_file_tab` logic.

```
0252 |     def _build_file_tab(self, notebook: ttk.Notebook) -> None:
0253 |         tab = ttk.Frame(notebook, padding=10, style="Tab.TFrame")
0254 |         tab.columnconfigure(1, weight=1)
0255 |         notebook.add(tab, text="File")
0256 |         ttk.Label(tab, text="File", style="Panel.TLabel").grid(row=0, column=0, sticky="w", padx=(0, 8))
0257 |         ttk.Entry(tab, textvariable=self.file_path_var).grid(row=0, column=1, sticky="ew")
0258 |         ttk.Button(tab, text="Browse", style="Tool.TButton", command=self.choose_file).grid(row=0, column=2, padx=(8, 0))
0259 |         ttk.Button(tab, text="Hash", style="Tool.TButton", command=lambda: self.run_command(["hash", self.file_path_var.get()]))
0260 |         ttk.Button(tab, text="Scan", style="Primary.TButton", command=lambda: self.run_command(["filescan", self.file_path_var.get()]))
```

Function: `choose_scan_file`

Lines: 262 to 265

Description: Analyzes and executes `choose_scan_file` logic.

```
0262 |     def choose_scan_file(self) -> None:
0263 |         path = filedialog.askopenfilename(title="Choose traffic CSV", filetypes=[("CSV files", "*.csv"), ("All files", "*.*")])
0264 |         if path:
0265 |             self.scan_path_var.set(self._display_path(path))
```

Function: `choose_file`

Lines: 267 to 270

Description: Analyzes and executes `choose_file` logic.

```
0267 |     def choose_file(self) -> None:
0268 |         path = filedialog.askopenfilename(title="Choose file")
0269 |         if path:
0270 |             self.file_path_var.set(self._display_path(path))
```

Function: `_display_path`

Lines: 272 to 276

Description: Analyzes and executes `_display_path` logic.

```
0272 |     def _display_path(self, path: str) -> str:
0273 |         try:
0274 |             return str(Path(path).resolve().relative_to(product_terminal.ROOT_DIR))
0275 |         except ValueError:
0276 |             return path
```

Function: `run_freeform_command`

Lines: 278 to 284

Description: Analyzes and executes `run_freeform_command` logic.

```
0278 |     def run_freeform_command(self) -> None:
0279 |         try:
0280 |             args = product_terminal.split_shell_command(self.command_var.get())
0281 |         except ValueError as exc:
0282 |             messagebox.showerror("Command parse error", str(exc))
0283 |             return
0284 |             self.run_command(args)
```

Function: `run_scan`

Lines: 286 to 293

Description: Analyzes and executes `run_scan` logic.

```
0286 |     def run_scan(self) -> None:
0287 |         args = ["scan", self.scan_path_var.get()]
0288 |         limit = self.scan_limit_var.get().strip()
0289 |         if limit.lower() == "all":
0290 |             args.append("--all")
0291 |         elif limit:
0292 |             args.extend(["--limit", limit])
0293 |             self.run_command(args)
```

Function: `run_command`

Lines: 295 to 298

Description: Analyzes and executes `run_command` logic.

```
0295 |     def run_command(self, args: list[str]) -> None:
0296 |         self._append(f"\n$ ids-sentinel-terminal {' '.join(args)}\n")
0297 |         thread = threading.Thread(target=self._run_command_worker, args=(args,), daemon=True)
0298 |         thread.start()
```

Function: `_run_command_worker`

Lines: 300 to 307

Description: Analyzes and executes `_run_command_worker` logic.

```

0300 |     def _run_command_worker(self, args: list[str]) -> None:
0301 |         stream = io.StringIO()
0302 |         with contextlib.redirect_stdout(stream), contextlib.redirect_stderr(stream):
0303 |             code = product_terminal.main(args)
0304 |             text = stream.getvalue()
0305 |             if code:
0306 |                 text += f"\nCommand exited with code {code}\n"
0307 |             self.output_queue.put(text)

```

Function: `_drain_output`

Lines: 309 to 315

Description: Analyzes and executes `_drain_output` logic.

```

0309 |     def _drain_output(self) -> None:
0310 |         try:
0311 |             while True:
0312 |                 self._append(self.output_queue.get_nowait())
0313 |         except queue.Empty:
0314 |             pass
0315 |         self.root.after(100, self._drain_output)

```

Function: `_append`

Lines: 317 to 333

Description: Analyzes and executes `_append` logic.

```

0317 |     def _append(self, text: str) -> None:
0318 |         for line in text.splitlines(keepends=True):
0319 |             tag = None
0320 |             normalized = line.lower()
0321 |             if line.lstrip().startswith("$ ids-sentinel-terminal"):
0322 |                 tag = "command"
0323 |             elif any(token in normalized for token in ("error", "exception", "traceback", "not recognized", "failed")):
0324 |                 tag = "danger"
0325 |             elif any(token in normalized for token in ("critical", "high", "warning", "suspicious")):
0326 |                 tag = "warning"
0327 |             elif any(token in normalized for token in ("healthy", "no threats", "completed", "success")):
0328 |                 tag = "success"
0329 |             if tag:
0330 |                 self.output.insert("end", line, tag)
0331 |             else:
0332 |                 self.output.insert("end", line)
0333 |             self.output.see("end")

```

Function: `clear_output`

Lines: 335 to 336

Description: Analyzes and executes `clear_output` logic.

```

0335 |     def clear_output(self) -> None:
0336 |         self.output.delete("1.0", "end")

```

Module: ./ids_app/product_terminal.py

Overview

Total Lines: 2350

Function: _copy_bundled_asset

Lines: 44 to 48

Description: Analyzes and executes _copy_bundled_asset logic.

```
0044 | def _copy_bundled_asset(asset_name: str, destination: Path) -> None:
0045 |     resource = importlib.resources.files("ids_app").joinpath("assets", asset_name)
0046 |     destination.parent.mkdir(parents=True, exist_ok=True)
0047 |     with importlib.resources.as_file(resource) as source_path:
0048 |         shutil.copy2(source_path, destination)
```

Function: bootstrap_runtime_home

Lines: 51 to 59

Description: Analyzes and executes bootstrap_runtime_home logic.

```
0051 | def bootstrap_runtime_home() -> None:
0052 |     if RUNTIME_MODE == "source":
0053 |         return
0054 |     ROOT_DIR.mkdir(parents=True, exist_ok=True)
0055 |     for relative_target, asset_name in BUNDLED_SEED_FILES.items():
0056 |         destination = ROOT_DIR / relative_target
0057 |         if destination.exists():
0058 |             continue
0059 |         _copy_bundled_asset(asset_name, destination)
```

Function: ensure_product_dirs

Lines: 218 to 225

Description: Analyzes and executes ensure_product_dirs logic.

```
0218 | def ensure_product_dirs() -> None:
0219 |     for path in (PRODUCT_DIR, EXPORTS_DIR, IMPORTS_DIR, CACHE_DIR, INDEX_DIR, COMMAND_CACHE_DIR):
0220 |         path.mkdir(parents=True, exist_ok=True)
0221 |         if LEGACY_INDEX_DIR.exists() and LEGACY_INDEX_DIR.is_dir():
0222 |             for legacy_file in LEGACY_INDEX_DIR.glob("*.json"):
0223 |                 target = INDEX_DIR / legacy_file.name
0224 |                 if not target.exists():
0225 |                     shutil.move(str(legacy_file), str(target))
```

Function: utc_now

Lines: 228 to 229

Description: Analyzes and executes utc_now logic.

```
0228 | def utc_now() -> str:
0229 |     return datetime.now(timezone.utc).replace(microsecond=0).isoformat()
```

Function: compact_timestamp

Lines: 232 to 233

Description: Analyzes and executes compact_timestamp logic.

```
0232 | def compact_timestamp() -> str:
0233 |     return datetime.now().strftime("%Y%m%d_%H%M%S")
```

Function: relative_path

Lines: 236 to 240

Description: Analyzes and executes relative_path logic.

```
0236 | def relative_path(path: Path) -> str:
0237 |     try:
0238 |         return str(path.resolve().relative_to(ROOT_DIR))
0239 |     except ValueError:
0240 |         return str(path)
```

Function: format_number

Lines: 243 to 250

Description: Analyzes and executes format_number logic.

```
0243 | def format_number(value: Any) -> str:
0244 |     if isinstance(value, float):
0245 |         if abs(value) >= 1000:
0246 |             return f"{value:,.0f}"
0247 |             return f"{value:.4f}"
0248 |     if isinstance(value, int):
0249 |         return f"{value:,}"
0250 |     return str(value)
```

Function: percent

Lines: 253 to 256

Description: Analyzes and executes percent logic.

```
0253 | def percent(part: int | float, total: int | float) -> str:
0254 |     if not total:
0255 |         return "0.00%"
0256 |     return f"{(float(part) / float(total)) * 100:.2f}%"
```

Function: table

Lines: 259 to 268

Description: Analyzes and executes table logic.

```

0259 | def table(headers: list[str], rows: list[list[Any]]) -> str:
0260 |     text_rows = [[format_number(cell) for cell in row] for row in rows]
0261 |     widths = [
0262 |         max(len(header), *(len(row[index]) for row in text_rows)) if text_rows else len(header)
0263 |         for index, header in enumerate(headers)
0264 |     ]
0265 |     header_line = " ".join(header.ljust(widths[index]) for index, header in enumerate(headers))
0266 |     rule = " ".join("-" * width for width in widths)
0267 |     body = [" ".join(row[index].ljust(widths[index]) for index in range(len(headers))) for row in text_rows]
0268 |     return "\n".join([header_line, rule, *body])

```

Function: section

Lines: 271 to 273

Description: Analyzes and executes section logic.

```

0271 | def section(title: str) -> None:
0272 |     print(f"\n{title}")
0273 |     print("=" * len(title))

```

Function: print_json

Lines: 276 to 277

Description: Analyzes and executes print_json logic.

```

0276 | def print_json(payload: Any) -> None:
0277 |     print(json.dumps(payload, indent=2, sort_keys=False))

```

Function: resolve_repo_path

Lines: 280 to 291

Description: Analyzes and executes resolve_repo_path logic.

```

0280 | def resolve_repo_path(path_text: str | None, default: Path = TEST_CSV) -> Path:
0281 |     if not path_text:
0282 |         return default
0283 |     path = Path(path_text)
0284 |     if not path.is_absolute():
0285 |         path = ROOT_DIR / path
0286 |     resolved = path.resolve()
0287 |     try:
0288 |         resolved.relative_to(ROOT_DIR)
0289 |     except ValueError:
0290 |         raise ValueError("path must stay inside the IDS Sentinel home directory")
0291 |     return resolved

```

Function: read_json

Lines: 294 to 298

Description: Analyzes and executes read_json logic.

```
0294 | def read_json(path: Path, default: Any = None) -> Any:
0295 |     if not path.exists():
0296 |         return default
0297 |     with path.open("r", encoding="utf-8") as handle:
0298 |         return json.load(handle)
```

Function: write_json

Lines: 301 to 304

Description: Analyzes and executes write_json logic.

```
0301 | def write_json(path: Path, payload: Any) -> None:
0302 |     path.parent.mkdir(parents=True, exist_ok=True)
0303 |     with path.open("w", encoding="utf-8") as handle:
0304 |         json.dump(payload, handle, indent=2, sort_keys=False)
```

Function: cache_artifact

Lines: 307 to 313

Description: Analyzes and executes cache_artifact logic.

```
0307 | def cache_artifact(kind: str, payload: Any) -> Path:
0308 |     ensure_product_dirs()
0309 |     safe_kind = re.sub(r"^[A-Za-z0-9_-]+", "_", kind).strip("_") or "artifact"
0310 |     path = COMMAND_CACHE_DIR / f"{compact_timestamp()}_{safe_kind}_{uuid4().hex[:8]}.json"
0311 |     write_json(path, {"created_at": utc_now(), "kind": kind, "payload": payload})
0312 |     prune_cache_artifacts()
0313 |     return path
```

Function: prune_cache_artifacts

Lines: 316 to 319

Description: Analyzes and executes prune_cache_artifacts logic.

```
0316 | def prune_cache_artifacts(max_files: int = 500) -> None:
0317 |     artifacts = sorted(COMMAND_CACHE_DIR.glob("*.json"), key=lambda item: item.stat().st_mtime, reverse=True)
0318 |     for stale in artifacts[max_files:]:
0319 |         stale.unlink(missing_ok=True)
```

Function: path_cache_key

Lines: 322 to 326

Description: Analyzes and executes path_cache_key logic.

```
0322 | def path_cache_key(path: Path) -> str:
0323 |     resolved = str(path.resolve()).lower()
0324 |     digest = hashlib.sha1(resolved.encode("utf-8")).hexdigest()[:16]
0325 |     safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", path.name)[:80]
0326 |     return f"{safe_name}.{digest}"
```

Function: file_signature

Lines: 329 to 331

Description: Analyzes and executes file_signature logic.

```
0329 | def file_signature(path: Path) -> dict[str, Any]:  
0330 |     stat = path.stat()  
0331 |     return {"size": stat.st_size, "mtime_ns": stat.st_mtime_ns}
```

Function: cached_json_path

Lines: 334 to 335

Description: Analyzes and executes cached_json_path logic.

```
0334 | def cached_json_path(path: Path, suffix: str) -> Path:  
0335 |     return INDEX_DIR / f"{path_cache_key(path)}.{suffix}.json"
```

Function: is_cache_current

Lines: 338 to 341

Description: Analyzes and executes is_cache_current logic.

```
0338 | def is_cache_current(path: Path, payload: dict[str, Any] | None) -> bool:  
0339 |     if not payload:  
0340 |         return False  
0341 |     return payload.get("signature") == file_signature(path)
```

Function: likely_header

Lines: 344 to 354

Description: Analyzes and executes likely_header logic.

```
0344 | def likely_header(row: list[str]) -> bool:  
0345 |     if not row:  
0346 |         return False  
0347 |     numeric = 0  
0348 |     for value in row:  
0349 |         try:  
0350 |             float(value)  
0351 |             numeric += 1  
0352 |         except ValueError:  
0353 |             pass  
0354 |     return numeric < max(1, len(row) // 2)
```

Function: all_csv_sources

Lines: 357 to 363

Description: Analyzes and executes all_csv_sources logic.

```
0357 | def all_csv_sources(include_exports: bool = True) -> list[Path]:  
0358 |     sources = [TRAIN_CSV, TEST_CSV]  
0359 |     if IMPORTS_DIR.exists():  
0360 |         sources.extend(sorted(IMPORTS_DIR.glob("*.csv")))  
0361 |     if include_exports and EXPORTS_DIR.exists():  
0362 |         sources.extend(sorted(EXPORTS_DIR.glob("*.csv")))  
0363 |     return [path for path in sources if path.exists()]
```

Function: resolve_any_product_path

Lines: 366 to 378

Description: Analyzes and executes resolve_any_product_path logic.

```
0366 | def resolve_any_product_path(path_text: str | None, default: Path = TEST_CSV) -> Path:
0367 |     if not path_text:
0368 |         return default
0369 |     path = Path(path_text)
0370 |     if not path.is_absolute():
0371 |         shell_cwd = Path(SHELL_STATE.get("cwd", ROOT_DIR))
0372 |         path = shell_cwd / path
0373 |     resolved = path.resolve()
0374 |     try:
0375 |         resolved.relative_to(ROOT_DIR)
0376 |     except ValueError:
0377 |         raise ValueError("path must stay inside the IDS Sentinel home directory")
0378 |     return resolved
```

Function: resolve_readable_path

Lines: 381 to 393

Description: Analyzes and executes resolve_readable_path logic.

```
0381 | def resolve_readable_path(path_text: str | None, default: Path | None = None, base: Path | None = None) -> Path:
0382 |     if not path_text:
0383 |         if default is None:
0384 |             raise ValueError("path is required")
0385 |         path = default
0386 |     else:
0387 |         path = Path(path_text)
0388 |         if not path.is_absolute():
0389 |             path = (base or ROOT_DIR) / path
0390 |     resolved = path.resolve()
0391 |     if not resolved.exists() or not resolved.is_file():
0392 |         raise ValueError(f"not a readable file: {path_text or default}")
0393 |     return resolved
```

Function: safe_float

Lines: 396 to 400

Description: Analyzes and executes safe_float logic.

```
0396 | def safe_float(value: str) -> float:
0397 |     try:
0398 |         return float(value)
0399 |     except (TypeError, ValueError):
0400 |         return 0.0
```

Function: iter_kdd_rows

Lines: 403 to 414

Description: Analyzes and executes iter_kdd_rows logic.

```

0403 | def iter_kdd_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
0404 |     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0405 |         reader = csv.reader(handle)
0406 |         emitted = 0
0407 |         for row_number, row in enumerate(reader, start=1):
0408 |             if len(row) < len(FEATURE_NAMES) + 1:
0409 |                 continue
0410 |             features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
0411 |             yield row_number, row[0].strip(), features
0412 |             emitted += 1
0413 |             if limit is not None and emitted >= limit:
0414 |                 return

```

Function: iter_generated_rows

Lines: 417 to 429

Description: Analyzes and executes iter_generated_rows logic.

```

0417 | def iter_generated_rows(path: Path, limit: int | None = None) -> Iterable[tuple[int, str, list[float]]]:
0418 |     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0419 |         reader = csv.DictReader(handle)
0420 |         emitted = 0
0421 |         for row_number, row in enumerate(reader, start=2):
0422 |             label = str(row.get("actual_label") or row.get("label") or "").strip()
0423 |             if label not in BINARY_LABELS:
0424 |                 continue
0425 |             features = [safe_float(row.get(f"{CSV_FEATURE_PREFIX}{name}", "0")) for name in FEATURE_NAMES]
0426 |             yield row_number, label, features
0427 |             emitted += 1
0428 |             if limit is not None and emitted >= limit:
0429 |                 return

```

Class: RunningStat

Lines: 433 to 457

Description: Analyzes and executes RunningStat logic.

```

0433 | class RunningStat:
0434 |     count: int = 0
0435 |     mean: float = 0.0
0436 |     m2: float = 0.0
0437 |     min_value: float = math.inf
0438 |     max_value: float = -math.inf
0439 |
0440 |     def update(self, value: float) -> None:
0441 |         self.count += 1
0442 |         delta = value - self.mean
0443 |         self.mean += delta / self.count
0444 |         delta2 = value - self.mean
0445 |         self.m2 += delta * delta2
0446 |         self.min_value = min(self.min_value, value)
0447 |         self.max_value = max(self.max_value, value)
0448 |
0449 |     def to_json(self) -> dict[str, float | int]:
0450 |         variance = self.m2 / max(self.count - 1, 1)
0451 |         return {
0452 |             "count": self.count,
0453 |             "mean": round(self.mean, 8),
0454 |             "variance": round(max(variance, 1e-9), 8),
0455 |             "min": round(self.min_value if self.count else 0.0, 8),
0456 |             "max": round(self.max_value if self.count else 0.0, 8),
0457 |         }

```

Function: empty_label_stats

Lines: 460 to 461

Description: Analyzes and executes empty_label_stats logic.

```

0460 | def empty_label_stats() -> dict[str, list[RunningStat]]:
0461 |     return {label: [RunningStat() for _ in FEATURE_NAMES] for label in BINARY_LABELS}

```

Function: update_model_stats

Lines: 464 to 469

Description: Analyzes and executes update_model_stats logic.

```

0464 | def update_model_stats(stats: dict[str, list[RunningStat]], labels: Counter[str], label: str, features: list[float]) -> None:
0465 |     if label not in stats:
0466 |         return
0467 |     labels[label] += 1
0468 |     for index, value in enumerate(features):
0469 |         stats[label][index].update(value)

```

Function: generated_export_paths

Lines: 472 to 475

Description: Analyzes and executes generated_export_paths logic.

```

0472 | def generated_export_paths() -> list[Path]:
0473 |     if not EXPORTS_DIR.exists():
0474 |         return []
0475 |     return sorted(EXPORTS_DIR.glob("traffic_analysis_*.csv"))

```

Function: `learn_model`

Lines: 478 to 540

Description: Analyzes and executes `learn_model` logic.

```

0478 | def learn_model(
0479 |     *,
0480 |     limit: int | None = None,
0481 |     include_generated: bool = True,
0482 |     include_test: bool = False,
0483 | ) -> dict[str, Any]:
0484 |     ensure_product_dirs()
0485 |     stats = empty_label_stats()
0486 |     labels: Counter[str] = Counter()
0487 |     sources: list[dict[str, Any]] = []
0488 |
0489 |     source_paths = [TRAIN_CSV]
0490 |     if include_test:
0491 |         source_paths.append(TEST_CSV)
0492 |
0493 |     for path in source_paths:
0494 |         rows_used = 0
0495 |         source_labels: Counter[str] = Counter()
0496 |         for _, label, features in iter_kdd_rows(path, limit):
0497 |             update_model_stats(stats, labels, label, features)
0498 |             source_labels[label] += 1
0499 |             rows_used += 1
0500 |         sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
0501 |
0502 |     if include_generated:
0503 |         for path in generated_export_paths():
0504 |             rows_used = 0
0505 |             source_labels = Counter()
0506 |             for _, label, features in iter_generated_rows(path, limit):
0507 |                 update_model_stats(stats, labels, label, features)
0508 |                 source_labels[label] += 1
0509 |                 rows_used += 1
0510 |             if rows_used:
0511 |                 sources.append({"path": relative_path(path), "rows_used": rows_used, "label_counts": dict(source_labels)})
0512 |
0513 |     total_rows = sum(labels.values())
0514 |     if total_rows == 0 or labels["0"] == 0 or labels["1"] == 0:
0515 |         raise RuntimeError("not enough labeled normal and attack rows to build a model")
0516 |
0517 |     label_payload: dict[str, Any] = {}
0518 |     for label in BINARY_LABELS:
0519 |         label_payload[label] = {
0520 |             "name": BINARY_LABELS[label],
0521 |             "count": labels[label],
0522 |             "prior": labels[label] / total_rows,
0523 |             "features": [item.to_json() for item in stats[label]],
0524 |         }
0525 |
0526 |     top_indicators = rank_indicators(label_payload)
0527 |     model = {
0528 |         "version": 1,
0529 |         "created_at": utc_now(),
0530 |         "model_type": "streaming_gaussian_profile",
0531 |         "description": "Pure-Python self-learning profile built from labeled IDS CSV rows and terminal-generated anomalies",
0532 |         "features": FEATURE_NAMES,
0533 |         "labels": label_payload,
0534 |         "total_rows": total_rows,
0535 |         "sources": sources,
0536 |         "top_indicators": top_indicators[:12],
0537 |     }
0538 |     write_json(MODEL_PATH, model)
0539 |     cache_artifact("learn", {"model_path": relative_path(MODEL_PATH), "rows_learned": total_rows, "sources": sources})
0540 |     return model

```

Function: rank_indicators

Lines: 543 to 561

Description: Analyzes and executes rank_indicators logic.

```
0543 | def rank_indicators(label_payload: dict[str, Any]) -> list[dict[str, Any]]:
0544 |     rows = []
0545 |     normal_stats = label_payload["0"]["features"]
0546 |     attack_stats = label_payload["1"]["features"]
0547 |     for index, name in enumerate(FEATURE_NAMES):
0548 |         normal = normal_stats[index]
0549 |         attack = attack_stats[index]
0550 |         pooled_std = math.sqrt((float(normal["variance"]) + float(attack["variance"])) / 2.0)
0551 |         score = abs(float(attack["mean"]) - float(normal["mean"])) / max(pooled_std, 1e-6)
0552 |         rows.append(
0553 |             {
0554 |                 "feature": name,
0555 |                 "separation": round(score, 6),
0556 |                 "normal_mean": round(float(normal["mean"]), 6),
0557 |                 "attack_mean": round(float(attack["mean"]), 6),
0558 |             }
0559 |         )
0560 |     rows.sort(key=lambda item: item["separation"], reverse=True)
0561 |     return rows
```

Function: load_or_learn_model

Lines: 564 to 570

Description: Analyzes and executes load_or_learn_model logic.

```
0564 | def load_or_learn_model(auto_learn: bool = True) -> dict[str, Any]:
0565 |     model = read_json(MODEL_PATH)
0566 |     if model:
0567 |         return model
0568 |     if not auto_learn:
0569 |         raise RuntimeError("model does not exist yet; run 'learn' first")
0570 |     return learn_model(limit=None)
```

Function: gaussian_log_probability

Lines: 573 to 584

Description: Analyzes and executes gaussian_log_probability logic.

```
0573 | def gaussian_log_probability(features: list[float], label_model: dict[str, Any], indicator_names: set[str] | None =
0574 |     logp = math.log(max(float(label_model["prior"]), 1e-12))
0575 |     for index, value in enumerate(features):
0576 |         name = FEATURE_NAMES[index]
0577 |         if indicator_names is not None and name not in indicator_names:
0578 |             continue
0579 |         stat = label_model["features"][index]
0580 |         mean = float(stat["mean"])
0581 |         variance = max(float(stat["variance"]), 1e-6)
0582 |         logp += -0.5 * math.log(2.0 * math.pi * variance)
0583 |         logp += -((value - mean) ** 2) / (2.0 * variance)
0584 |     return logp
```

Function: score_row

Lines: 587 to 604

Description: Analyzes and executes score_row logic.

```
0587 | def score_row(model: dict[str, Any], features: list[float]) -> dict[str, Any]:
0588 |     indicator_names = {item["feature"] for item in model.get("top_indicators", [])[:16]} or None
0589 |     normal_log = gaussian_log_probability(features, model["labels"]["0"], indicator_names)
0590 |     attack_log = gaussian_log_probability(features, model["labels"]["1"], indicator_names)
0591 |     delta = max(min(attack_log - normal_log, 60.0), -60.0)
0592 |     attack_probability = 1.0 / (1.0 + math.exp(-delta))
0593 |     predicted = "1" if attack_probability >= 0.5 else "0"
0594 |     confidence = abs(attack_probability - 0.5) * 2.0
0595 |     family, reasons = classify_behavior(features, attack_probability, model)
0596 |     return {
0597 |         "predicted_label": predicted,
0598 |         "predicted_name": BINARY_LABELS[predicted],
0599 |         "risk_score": round(attack_probability, 6),
0600 |         "confidence": round(confidence, 6),
0601 |         "risk_level": risk_level(attack_probability),
0602 |         "family": family,
0603 |         "reasons": reasons,
0604 |     }
```

Function: feature_map

Lines: 607 to 608

Description: Analyzes and executes feature_map logic.

```
0607 | def feature_map(features: list[float]) -> dict[str, float]:
0608 |     return dict(zip(FEATURE_NAMES, features))
```

Function: risk_level

Lines: 611 to 618

Description: Analyzes and executes risk_level logic.

```
0611 | def risk_level(score: float) -> str:
0612 |     if score >= 0.9:
0613 |         return "critical"
0614 |     if score >= 0.75:
0615 |         return "high"
0616 |     if score >= 0.55:
0617 |         return "medium"
0618 |     return "low"
```

Function: classify_behavior

Lines: 621 to 660

Description: Analyzes and executes classify_behavior logic.

```

0621 | def classify_behavior(features: list[float], risk_score: float, model: dict[str, Any]) -> tuple[str, str]:
0622 |     values = feature_map(features)
0623 |     families: list[str] = []
0624 |     reasons: list[str] = []
0625 |
0626 |     if values["count"] >= 80 or values["srv_count"] >= 80 or values["error_rate"] >= 0.5 or values["srv_error_rate"] >= 0.5:
0627 |         families.append("dos_flood")
0628 |         reasons.append("high connection or service-error rate")
0629 |     if values["diff_srv_rate"] >= 0.35 or values["srv_diff_host_rate"] >= 0.35 or values["dst_host_srv_diff_host_rate"] >= 0.35:
0630 |         families.append("probe_scan")
0631 |         reasons.append("high service or host diversity")
0632 |     if values["num_failed_logins"] > 0 or values["is_guest_login"] > 0 or values["logged_in"] == 0 and values["hot_login"] > 0:
0633 |         families.append("credential_abuse")
0634 |         reasons.append("login or credential anomaly")
0635 |     if values["root_shell"] > 0 or values["su_attempted"] > 0 or values["num_compromised"] > 0 or values["num_root_shell"] > 0:
0636 |         families.append("privilege_escalation")
0637 |         reasons.append("compromise or privilege signal")
0638 |     if values["num_file_creations"] > 0 or values["num_shells"] > 0 or values["num_access_files"] > 0:
0639 |         families.append("malware_like_activity")
0640 |         reasons.append("file, shell, or access-file behavior")
0641 |     if values["wrong_fragment"] > 0 or values["urgent"] > 0 or values["src_bytes"] > 100000 or values["dst_bytes"] > 100000:
0642 |         families.append("payload_or_exfiltration")
0643 |         reasons.append("fragment, urgent, or high byte volume")
0644 |
0645 |     if risk_score < 0.55 and not families:
0646 |         return "normal", "close to learned normal profile"
0647 |
0648 |     if not families:
0649 |         for item in model.get("top_indicators", []):4]:
0650 |             name = item["feature"]
0651 |             index = FEATURE_NAMES.index(name)
0652 |             value = features[index]
0653 |             normal_mean = float(item["normal_mean"])
0654 |             attack_mean = float(item["attack_mean"])
0655 |             if abs(value - attack_mean) < abs(value - normal_mean):
0656 |                 direction = "high" if attack_mean > normal_mean else "low"
0657 |                 reasons.append(f"{name} is {direction} versus normal profile")
0658 |             families.append("network_attack")
0659 |
0660 |     return families[0], "; ".join(reasons[:4]) if reasons else "matches learned attack profile"

```

Function: summarize_dataset

Lines: 663 to 703

Description: Analyzes and executes summarize_dataset logic.

```

0663 | def summarize_dataset(path: Path, limit: int | None = None) -> dict[str, Any]:
0664 |     labels: Counter[str] = Counter()
0665 |     protocol_counts: Counter[str] = Counter()
0666 |     service_counts: Counter[str] = Counter()
0667 |     flag_counts: Counter[str] = Counter()
0668 |     total_src_bytes = 0.0
0669 |     total_dst_bytes = 0.0
0670 |     rows = 0
0671 |     malformed = 0
0672 |
0673 |     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0674 |         reader = csv.reader(handle)
0675 |         for row in reader:
0676 |             if len(row) < len(FEATURE_NAMES) + 1:
0677 |                 malformed += 1
0678 |                 continue
0679 |                 rows += 1
0680 |                 labels[row[0].strip()] += 1
0681 |                 protocol_counts[row[2].strip()] += 1
0682 |                 service_counts[row[3].strip()] += 1
0683 |                 flag_counts[row[4].strip()] += 1
0684 |                 total_src_bytes += safe_float(row[5])
0685 |                 total_dst_bytes += safe_float(row[6])
0686 |                 if limit is not None and rows >= limit:
0687 |                     break
0688 |
0689 |     return {
0690 |         "path": relative_path(path),
0691 |         "rows": rows,
0692 |         "malformed_rows": malformed,
0693 |         "columns": len(FEATURE_NAMES) + 1,
0694 |         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
0695 |         "label_counts": dict(labels),
0696 |         "attack_share": round(labels["1"] / rows, 6) if rows else 0,
0697 |         "normal_share": round(labels["0"] / rows, 6) if rows else 0,
0698 |         "total_src_bytes": int(total_src_bytes),
0699 |         "total_dst_bytes": int(total_dst_bytes),
0700 |         "top_protocols": protocol_counts.most_common(5),
0701 |         "top_services": service_counts.most_common(8),
0702 |         "top_flags": flag_counts.most_common(8),
0703 |     }

```

Function: summarize_dataset_cached

Lines: 706 to 721

Description: Analyzes and executes summarize_dataset_cached logic.

```

0706 | def summarize_dataset_cached(path: Path) -> dict[str, Any]:
0707 |     cache_path = cached_json_path(path, "summary")
0708 |     cached = read_json(cache_path)
0709 |     if is_cache_current(path, cached):
0710 |         return cached["summary"]
0711 |     summary = summarize_dataset(path)
0712 |     write_json(
0713 |         cache_path,
0714 |         {
0715 |             "cached_at": utc_now(),
0716 |             "path": relative_path(path),
0717 |             "signature": file_signature(path),
0718 |             "summary": summary,
0719 |         },
0720 |     )
0721 |     return summary

```

Function: `summarize_all_datasets`

Lines: 724 to 725

Description: Analyzes and executes `summarize_all_datasets` logic.

```
0724 | def summarize_all_datasets() -> dict[str, Any]:  
0725 |     return {"train": summarize_dataset_cached(TRAIN_CSV), "test": summarize_dataset_cached(TEST_CSV)}
```

Function: `inspect_csv`

Lines: 728 to 798

Description: Analyzes and executes `inspect_csv` logic.

```

0728 | def inspect_csv(path: Path, limit: int | None = 50000) -> dict[str, Any]:
0729 |     cache_path = cached_json_path(path, f"inspect-{'limit' or 'all'}")
0730 |     cached = read_json(cache_path)
0731 |     if limit is None and is_cache_current(path, cached):
0732 |         return cached["inspection"]
0733 |
0734 |     rows = 0
0735 |     malformed = 0
0736 |     columns = 0
0737 |     first_row: list[str] | None = None
0738 |     header: list[str] | None = None
0739 |     label_counts: Counter[str] = Counter()
0740 |     column_counters: list[Counter[str]] = []
0741 |
0742 |     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0743 |         reader = csv.reader(handle)
0744 |         for raw_index, row in enumerate(reader):
0745 |             if not row:
0746 |                 malformed += 1
0747 |                 continue
0748 |             if first_row is None:
0749 |                 first_row = row
0750 |                 columns = len(row)
0751 |                 header = row if likely_header(row) else None
0752 |                 column_counters = [Counter() for _ in range(columns)]
0753 |                 if header:
0754 |                     continue
0755 |             if columns and len(row) != columns:
0756 |                 malformed += 1
0757 |             rows += 1
0758 |             if row:
0759 |                 label_counts[row[0].strip()] += 1
0760 |                 for index, value in enumerate(row[:columns]):
0761 |                     if len(column_counters[index]) < 2000:
0762 |                         column_counters[index][value.strip()] += 1
0763 |                 if limit is not None and rows >= limit:
0764 |                     break
0765 |
0766 |     field_names = header or [f"column_{index}" for index in range(columns)]
0767 |     top_values = []
0768 |     for index, counter in enumerate(column_counters[:20]):
0769 |         top_values.append(
0770 |             {
0771 |                 "column": field_names[index] if index < len(field_names) else f"column_{index}",
0772 |                 "top": counter.most_common(8),
0773 |             }
0774 |         )
0775 |
0776 |     inspection = {
0777 |         "path": relative_path(path),
0778 |         "rows_scanned": rows,
0779 |         "scan_limit": limit,
0780 |         "columns": columns,
0781 |         "has_header": bool(header),
0782 |         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
0783 |         "malformed_rows": malformed,
0784 |         "label_counts_first_column": dict(label_counts),
0785 |         "top_values": top_values,
0786 |     }
0787 |     cache_artifact("index", inspection)
0788 |     if limit is None:
0789 |         write_json(
0790 |             cache_path,
0791 |             {
0792 |                 "cached_at": utc_now(),
0793 |                 "path": relative_path(path),
0794 |                 "signature": file_signature(path),
0795 |                 "inspection": inspection,

```

```
0796 |         },  
0797 |     )  
0798 |     return inspection
```

Function: analyze_csv

Lines: 801 to 939

Description: Analyzes and executes analyze_csv logic.

```

0801 | def analyze_csv(
0802 |     source: Path,
0803 |     *,
0804 |     limit: int | None = 5000,
0805 |     export: bool = True,
0806 |     model: dict[str, Any] | None = None,
0807 | ) -> dict[str, Any]:
0808 |     ensure_product_dirs()
0809 |     model = model or load_or_learn_model()
0810 |     analysis_id = f"scan-{compact_timestamp()}"
0811 |     export_csv_path = EXPORTS_DIR / f"traffic_analysis_{compact_timestamp()}.csv"
0812 |     export_json_path = export_csv_path.with_suffix(".json")
0813 |
0814 |     total = 0
0815 |     malformed = 0
0816 |     actual_counts: Counter[str] = Counter()
0817 |     predicted_counts: Counter[str] = Counter()
0818 |     risk_counts: Counter[str] = Counter()
0819 |     family_counts: Counter[str] = Counter()
0820 |     protocol_counts: Counter[str] = Counter()
0821 |     service_counts: Counter[str] = Counter()
0822 |     flag_counts: Counter[str] = Counter()
0823 |     metrics = Counter()
0824 |     risk_sum = 0.0
0825 |
0826 |     fieldnames = [
0827 |         "analysis_id",
0828 |         "analyzed_at",
0829 |         "source_file",
0830 |         "row_number",
0831 |         "actual_label",
0832 |         "actual_name",
0833 |         "predicted_label",
0834 |         "predicted_name",
0835 |         "risk_score",
0836 |         "confidence",
0837 |         "risk_level",
0838 |         "family",
0839 |         "reasons",
0840 |         *[f"{CSV_FEATURE_PREFIX}{name}" for name in FEATURE_NAMES],
0841 |     ]
0842 |
0843 |     writer = None
0844 |     export_handle = None
0845 |     try:
0846 |         if export:
0847 |             export_handle = export_csv_path.open("w", encoding="utf-8", newline="")
0848 |             writer = csv.DictWriter(export_handle, fieldnames=fieldnames)
0849 |             writer.writeheader()
0850 |
0851 |             with source.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0852 |                 reader = csv.reader(handle)
0853 |                 for row_number, row in enumerate(reader, start=1):
0854 |                     if len(row) < len(FEATURE_NAMES) + 1:
0855 |                         malformed += 1
0856 |                         continue
0857 |                     label = row[0].strip()
0858 |                     features = [safe_float(value) for value in row[1 : len(FEATURE_NAMES) + 1]]
0859 |                     result = score_row(model, features)
0860 |                     total += 1
0861 |                     actual_counts[label] += 1
0862 |                     predicted_counts[result["predicted_label"]] += 1
0863 |                     risk_counts[result["risk_level"]] += 1
0864 |                     family_counts[result["family"]] += 1
0865 |                     protocol_counts[row[2].strip()] += 1
0866 |                     service_counts[row[3].strip()] += 1
0867 |                     flag_counts[row[4].strip()] += 1
0868 |                     risk_sum += float(result["risk_score"])

```

```

0869 |
0870 |         if label in BINARY_LABELS:
0871 |             actual_attack = label == "1"
0872 |             predicted_attack = result["predicted_label"] == "1"
0873 |             if actual_attack and predicted_attack:
0874 |                 metrics["tp"] += 1
0875 |             elif actual_attack and not predicted_attack:
0876 |                 metrics["fn"] += 1
0877 |             elif not actual_attack and predicted_attack:
0878 |                 metrics["fp"] += 1
0879 |             else:
0880 |                 metrics["tn"] += 1
0881 |
0882 |         if writer:
0883 |             output_row = {
0884 |                 "analysis_id": analysis_id,
0885 |                 "analyzed_at": utc_now(),
0886 |                 "source_file": relative_path(source),
0887 |                 "row_number": row_number,
0888 |                 "actual_label": label,
0889 |                 "actual_name": BINARY_LABELS.get(label, "unknown"),
0890 |                 **result,
0891 |             }
0892 |             output_row["reasons"] = result["reasons"]
0893 |             for index, name in enumerate(FEATURE_NAMES):
0894 |                 output_row[f"{CSV_FEATURE_PREFIX}{name}"] = features[index]
0895 |             writer.writerow(output_row)
0896 |
0897 |         if limit is not None and total >= limit:
0898 |             break
0899 |     finally:
0900 |         if export_handle:
0901 |             export_handle.close()
0902 |
0903 |     precision = metrics["tp"] / max(metrics["tp"] + metrics["fp"], 1)
0904 |     recall = metrics["tp"] / max(metrics["tp"] + metrics["fn"], 1)
0905 |     accuracy = (metrics["tp"] + metrics["tn"]) / max(sum(metrics.values()), 1)
0906 |     f1 = (2 * precision * recall) / max(precision + recall, 1e-12)
0907 |     summary = {
0908 |         "analysis_id": analysis_id,
0909 |         "created_at": utc_now(),
0910 |         "source_file": relative_path(source),
0911 |         "rows_analyzed": total,
0912 |         "malformed_rows": malformed,
0913 |         "limit": limit,
0914 |         "average_risk_score": round(risk_sum / total, 6) if total else 0,
0915 |         "actual_counts": dict(actual_counts),
0916 |         "predicted_counts": dict(predicted_counts),
0917 |         "risk_counts": dict(risk_counts),
0918 |         "family_counts": dict(family_counts),
0919 |         "top_protocols": protocol_counts.most_common(8),
0920 |         "top_services": service_counts.most_common(8),
0921 |         "top_flags": flag_counts.most_common(8),
0922 |         "metrics": {
0923 |             "accuracy": round(accuracy, 6),
0924 |             "precision": round(precision, 6),
0925 |             "recall": round(recall, 6),
0926 |             "f1": round(f1, 6),
0927 |             "tp": metrics["tp"],
0928 |             "tn": metrics["tn"],
0929 |             "fp": metrics["fp"],
0930 |             "fn": metrics["fn"],
0931 |         },
0932 |         "model_created_at": model.get("created_at"),
0933 |         "export_csv": relative_path(export_csv_path) if export else None,
0934 |         "export_json": relative_path(export_json_path) if export else None,
0935 |     }
0936 |     if export:
0937 |         write_json(export_json_path, summary)

```

```
0938 |     cache_artifact("scan", summary)
0939 |     return summary
```

Function: latest_export_summary

Lines: 942 to 948

Description: Analyzes and executes latest_export_summary logic.

```
0942 | def latest_export_summary() -> dict[str, Any] | None:
0943 |     if not EXPORTS_DIR.exists():
0944 |         return None
0945 |     summaries = sorted(EXPORTS_DIR.glob("traffic_analysis_*.json"), key=lambda item: item.stat().st_mtime, reverse=True)
0946 |     if not summaries:
0947 |         return None
0948 |     return read_json(summaries[0])
```

Function: list_reports

Lines: 951 to 968

Description: Analyzes and executes list_reports logic.

```
0951 | def list_reports(limit: int | None = 20) -> list[dict[str, Any]]:
0952 |     if not EXPORTS_DIR.exists():
0953 |         return []
0954 |     reports = []
0955 |     for path in sorted(EXPORTS_DIR.glob("*"), key=lambda item: item.stat().st_mtime, reverse=True):
0956 |         if not path.is_file():
0957 |             continue
0958 |         reports.append(
0959 |             {
0960 |                 "name": path.name,
0961 |                 "path": relative_path(path),
0962 |                 "size_kb": round(path.stat().st_size / 1024, 2),
0963 |                 "modified": datetime.fromtimestamp(path.stat().st_mtime).isoformat(timespec="seconds"),
0964 |             }
0965 |         )
0966 |         if limit is not None and len(reports) >= limit:
0967 |             break
0968 |     return reports
```

Function: list_cache_artifacts

Lines: 971 to 985

Description: Analyzes and executes list_cache_artifacts logic.

```

0971 | def list_cache_artifacts(limit: int = 40) -> list[dict[str, Any]]:
0972 |     ensure_product_dirs()
0973 |     artifacts = []
0974 |     for path in sorted(COMMAND_CACHE_DIR.glob("*.json"), key=lambda item: item.stat().st_mtime, reverse=True):
0975 |         artifacts.append(
0976 |             {
0977 |                 "name": path.name,
0978 |                 "path": relative_path(path),
0979 |                 "size_kb": round(path.stat().st_size / 1024, 2),
0980 |                 "modified": datetime.fromtimestamp(path.stat().st_mtime).isoformat(timespec="seconds"),
0981 |             }
0982 |         )
0983 |         if limit is not None and len(artifacts) >= limit:
0984 |             break
0985 |     return artifacts

```

Function: show_cache

Lines: 988 to 1008

Description: Analyzes and executes show_cache logic.

```

0988 | def show_cache(json_output: bool = False, limit: int = 40) -> None:
0989 |     payload = {
0990 |         "cache_dir": relative_path(CACHE_DIR),
0991 |         "index_dir": relative_path(INDEX_DIR),
0992 |         "command_cache_dir": relative_path(COMMAND_CACHE_DIR),
0993 |         "artifacts": list_cache_artifacts(limit),
0994 |     }
0995 |     if json_output:
0996 |         print_json(payload)
0997 |         return
0998 |     section("IDS Sentinel Terminal Cache")
0999 |     print(f"Cache:    {payload['cache_dir']}")
1000 |     print(f"Indexes: {payload['index_dir']}")
1001 |     print(f"Runs:     {payload['command_cache_dir']}")
1002 |     if not payload["artifacts"]:
1003 |         print("No command cache artifacts yet.")
1004 |         return
1005 |     print()
1006 |     print(table(["Name", "Path", "Size KB", "Modified"], [
1007 |         [item["name"], item["path"], item["size_kb"], item["modified"]] for item in payload["artifacts"]
1008 |     ]))

```

Function: list_run_summaries

Lines: 1011 to 1022

Description: Analyzes and executes list_run_summaries logic.

```

1011 | def list_run_summaries(limit: int | None = 8) -> list[dict[str, Any]]:
1012 |     runs_dir = ROOT_DIR / "automation" / "runs"
1013 |     if not runs_dir.exists():
1014 |         return []
1015 |     rows = []
1016 |     for path in sorted(runs_dir.glob("*/summary.json"), key=lambda item: item.stat().st_mtime, reverse=True):
1017 |         payload = read_json(path)
1018 |         if payload:
1019 |             rows.append(payload)
1020 |             if limit is not None and len(rows) >= limit:
1021 |                 break
1022 |     return rows

```

Function: show_dataset_catalog

Lines: 1025 to 1041

Description: Analyzes and executes show_dataset_catalog logic.

```
1025 | def show_dataset_catalog(json_output: bool = False) -> None:
1026 |     payload = {
1027 |         "local_sources": [relative_path(path) for path in all_csv_sources(include_exports=False)],
1028 |         "external_catalog": EXTERNAL_DATASETS,
1029 |     }
1030 |     cache_artifact("datasets", payload)
1031 |     if json_output:
1032 |         print_json(payload)
1033 |         return
1034 |     section("Dataset Catalog")
1035 |     print(table(["ID", "Name", "Source", "Format"], [
1036 |         [item["id"], item["name"], item["source"], item["format"]] for item in EXTERNAL_DATASETS
1037 |     ]))
1038 |     print()
1039 |     print(table(["Local CSV", "Size MB"], [
1040 |         [relative_path(path), round(path.stat().st_size / (1024 * 1024), 2)] for path in all_csv_sources(include_exports=False)]
1041 |     ]))
```

Function: import_csv

Lines: 1044 to 1058

Description: Analyzes and executes import_csv logic.

```
1044 | def import_csv(source: Path, name: str | None = None) -> Path:
1045 |     ensure_product_dirs()
1046 |     if not source.exists() or not source.is_file():
1047 |         raise ValueError(f"not a file: {source}")
1048 |     if source.suffix.lower() != ".csv":
1049 |         raise ValueError("only CSV imports are supported in IDS Sentinel Terminal")
1050 |     safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", name or source.name)
1051 |     if not safe_name.lower().endswith(".csv"):
1052 |         safe_name += ".csv"
1053 |     target = IMPORTS_DIR / safe_name
1054 |     if source.resolve() != target.resolve():
1055 |         shutil.copy2(source, target)
1056 |     inspect_csv(target, limit=None)
1057 |     cache_artifact("import", {"source": str(source), "target": relative_path(target), "bytes": target.stat().st_size})
1058 |     return target
```

Function: download_url

Lines: 1061 to 1081

Description: Analyzes and executes download_url logic.

```

1061 | def download_url(url: str, name: str | None = None, max_bytes: int = 2 * 1024 * 1024 * 1024) -> Path:
1062 |     ensure_product_dirs()
1063 |     parsed_name = name or Path(url.split("?", 1)[0]).name or f"download_{compact_timestamp()}"
1064 |     safe_name = re.sub(r"^[A-Za-z0-9_-]+", "_", parsed_name)
1065 |     target = IMPORTS_DIR / safe_name
1066 |     try:
1067 |         with urllib.request.urlopen(url, timeout=60) as response, target.open("wb") as handle:
1068 |             copied = 0
1069 |             while True:
1070 |                 chunk = response.read(1024 * 1024)
1071 |                 if not chunk:
1072 |                     break
1073 |                 copied += len(chunk)
1074 |                 if copied > max_bytes:
1075 |                     raise RuntimeError("download exceeded the 2 GB safety limit")
1076 |                 handle.write(chunk)
1077 |     except Exception:
1078 |         target.unlink(missing_ok=True)
1079 |         raise
1080 |     cache_artifact("download", {"url": url, "path": relative_path(target), "bytes": target.stat().st_size})
1081 |     return target

```

Function: show_import

Lines: 1084 to 1091

Description: Analyzes and executes show_import logic.

```

1084 | def show_import(path: Path, json_output: bool = False) -> None:
1085 |     payload = {"imported_path": relative_path(path), "inspection": inspect_csv(path, limit=None)}
1086 |     if json_output:
1087 |         print_json(payload)
1088 |         return
1089 |     section("CSV Imported")
1090 |     print(f"Imported: {payload['imported_path']}")
1091 |     show_index(path, json_output=False)

```

Function: show_index

Lines: 1094 to 1111

Description: Analyzes and executes show_index logic.

```

1094 | def show_index(path: Path, json_output: bool = False, limit: int | None = 50000) -> None:
1095 |     payload = inspect_csv(path, limit=limit)
1096 |     if json_output:
1097 |         print_json(payload)
1098 |         return
1099 |     section("CSV Index")
1100 |     print(f"Path: {payload['path']}")
1101 |     print(f"Rows scanned: {payload['rows_scanned']:,} | columns: {payload['columns']} | size: {payload['size_mb']} MB")
1102 |     print(f"Header: {payload['has_header']} | malformed rows: {payload['malformed_rows']:,}")
1103 |     if payload["label_counts_first_column"]:
1104 |         print()
1105 |         print(table(["First Column Value", "Rows"], [
1106 |             [label, count] for label, count in sorted(payload["label_counts_first_column"].items(), key=lambda item: item[1])
1107 |         ]))
1108 |         print()
1109 |         print(table(["Column", "Top Values"], [
1110 |             [item["column"], ", ".join(f"{value}:{count}" for value, count in item["top"][:5])] for item in payload["top_values"]
1111 |         ]))

```

Function: load_services

Lines: 1114 to 1155

Description: Analyzes and executes load_services logic.

```
1114 | def load_services() -> dict[int, str]:
1115 |     services: dict[int, str] = {
1116 |         20: "ftp-data",
1117 |         21: "ftp",
1118 |         22: "ssh",
1119 |         23: "telnet",
1120 |         25: "smtp",
1121 |         53: "domain",
1122 |         80: "http",
1123 |         110: "pop3",
1124 |         135: "msrpc",
1125 |         139: "netbios-ssn",
1126 |         143: "imap",
1127 |         443: "https",
1128 |         445: "microsoft-ds",
1129 |         1433: "ms-sql-s",
1130 |         3306: "mysql",
1131 |         3389: "ms-wbt-server",
1132 |         5432: "postgresql",
1133 |         6379: "redis",
1134 |         8080: "http-alt",
1135 |         9200: "elasticsearch",
1136 |         27017: "mongodb",
1137 |     }
1138 |     services_file = Path(os.environ.get("SystemRoot", "C:\\Windows")) / "System32" / "drivers" / "etc" / "services"
1139 |     if services_file.exists():
1140 |         with services_file.open("r", encoding="utf-8", errors="ignore") as handle:
1141 |             for line in handle:
1142 |                 line = line.strip()
1143 |                 if not line or line.startswith("#"):
1144 |                     continue
1145 |                 parts = line.split()
1146 |                 if len(parts) < 2 or "/" not in parts[1]:
1147 |                     continue
1148 |                 port_text, proto = parts[1].split("/", 1)
1149 |                 if proto.lower() not in {"tcp", "udp"}:
1150 |                     continue
1151 |                 try:
1152 |                     services.setdefault(int(port_text), parts[0])
1153 |                 except ValueError:
1154 |                     continue
1155 |     return services
```

Function: parse_ports

Lines: 1158 to 1181

Description: Analyzes and executes parse_ports logic.

```

1158 | def parse_ports(text: str) -> list[int]:
1159 |     if text.lower() in {"common", "top"}:
1160 |         return COMMON_PROBE_PORTS
1161 |     ports: set[int] = set()
1162 |     for part in text.split(","):
1163 |         part = part.strip()
1164 |         if not part:
1165 |             continue
1166 |         if "-" in part:
1167 |             start_text, end_text = part.split("-", 1)
1168 |             start = int(start_text)
1169 |             end = int(end_text)
1170 |             if end < start:
1171 |                 start, end = end, start
1172 |             for port in range(start, min(end, start + 127) + 1):
1173 |                 if 1 <= port <= 65535:
1174 |                     ports.add(port)
1175 |         else:
1176 |             port = int(part)
1177 |             if 1 <= port <= 65535:
1178 |                 ports.add(port)
1179 |     if len(ports) > 128:
1180 |         raise ValueError("port list is capped at 128 ports per probe")
1181 |     return sorted(ports)

```

Function: split_host_port

Lines: 1184 to 1192

Description: Analyzes and executes split_host_port logic.

```

1184 | def split_host_port(value: str) -> tuple[str, int | None]:
1185 |     value = value.strip()
1186 |     if value.startswith("[") and "]" in value:
1187 |         host, port_text = value.rsplit(":", 1)
1188 |         return host.strip("[]"), int(port_text) if port_text.isdigit() else None
1189 |     if ":" in value and value.count(":") == 1:
1190 |         host, port_text = value.rsplit(":", 1)
1191 |         return host, int(port_text) if port_text.isdigit() else None
1192 |     return value, None

```

Function: parse_netstat

Lines: 1195 to 1237

Description: Analyzes and executes parse_netstat logic.

```

1195 | def parse_netstat() -> list[dict[str, Any]]:
1196 |     try:
1197 |         completed = subprocess.run(["netstat", "-ano"], capture_output=True, text=True, timeout=15, check=False)
1198 |     except FileNotFoundError:
1199 |         return []
1200 |     rows = []
1201 |     for line in completed.stdout.splitlines():
1202 |         parts = line.split()
1203 |         if not parts or parts[0] not in {"TCP", "UDP"}:
1204 |             continue
1205 |         proto = parts[0]
1206 |         if proto == "TCP" and len(parts) >= 5:
1207 |             local_host, local_port = split_host_port(parts[1])
1208 |             remote_host, remote_port = split_host_port(parts[2])
1209 |             rows.append(
1210 |                 {
1211 |                     "proto": proto,
1212 |                     "local": parts[1],
1213 |                     "local_host": local_host,
1214 |                     "local_port": local_port,
1215 |                     "remote": parts[2],
1216 |                     "remote_host": remote_host,
1217 |                     "remote_port": remote_port,
1218 |                     "state": parts[3],
1219 |                     "pid": parts[4],
1220 |                 }
1221 |             )
1222 |         elif proto == "UDP" and len(parts) >= 4:
1223 |             local_host, local_port = split_host_port(parts[1])
1224 |             rows.append(
1225 |                 {
1226 |                     "proto": proto,
1227 |                     "local": parts[1],
1228 |                     "local_host": local_host,
1229 |                     "local_port": local_port,
1230 |                     "remote": parts[2],
1231 |                     "remote_host": "*",
1232 |                     "remote_port": None,
1233 |                     "state": "UDP",
1234 |                     "pid": parts[3],
1235 |                 }
1236 |             )
1237 |     return rows

```

Function: show_netstat

Lines: 1240 to 1265

Description: Analyzes and executes show_netstat logic.

```

1240 | def show_netstat(json_output: bool = False, only_listening: bool = False, limit: int = 40) -> None:
1241 |     services = load_services()
1242 |     rows = parse_netstat()
1243 |     if only_listening:
1244 |         rows = [row for row in rows if row["state"] in {"LISTENING", "UDP"}]
1245 |     rows = sorted(rows, key=lambda item: (item.get("local_port") or 0, item["proto"], item["pid"]))
1246 |     payload = rows[:limit]
1247 |     cache_artifact("ports" if only_listening else "netstat", payload)
1248 |     if json_output:
1249 |         print_json(payload)
1250 |         return
1251 |     section("Network Connections")
1252 |     if not payload:
1253 |         print("No netstat rows found.")
1254 |         return
1255 |     print(table(["Proto", "Local", "Service", "Remote", "State", "PID"], [
1256 |         [
1257 |             row["proto"],
1258 |             row["local"],
1259 |             services.get(row.get("local_port") or -1, ""),
1260 |             row["remote"],
1261 |             row["state"],
1262 |             row["pid"],
1263 |         ]
1264 |         for row in payload
1265 |     ]))

```

Function: show_port

Lines: 1268 to 1287

Description: Analyzes and executes show_port logic.

```

1268 | def show_port(port: int, json_output: bool = False) -> None:
1269 |     services = load_services()
1270 |     payload = {
1271 |         "port": port,
1272 |         "service": services.get(port, "unknown"),
1273 |         "risk": COMMON_PORT_RISKS.get(port, "No specific built-in note. Validate whether this service should be exposed"),
1274 |         "local_matches": [row for row in parse_netstat() if row.get("local_port") == port],
1275 |     }
1276 |     cache_artifact("port", payload)
1277 |     if json_output:
1278 |         print_json(payload)
1279 |         return
1280 |     section(f"Port {port}")
1281 |     print(f"Service: {payload['service']}")
1282 |     print(f"Risk: {payload['risk']}")
1283 |     if payload["local_matches"]:
1284 |         print()
1285 |         print(table(["Proto", "Local", "Remote", "State", "PID"], [
1286 |             [row["proto"], row["local"], row["remote"], row["state"], row["pid"]] for row in payload["local_matches"]
1287 |         ]))

```

Function: probe_ports

Lines: 1290 to 1303

Description: Analyzes and executes probe_ports logic.

```

1290 | def probe_ports(host: str, ports: list[int], timeout_seconds: float = 0.2) -> list[dict[str, Any]]:
1291 |     results = []
1292 |     services = load_services()
1293 |     for port in ports:
1294 |         started = time.perf_counter()
1295 |         status = "closed"
1296 |         try:
1297 |             with socket.create_connection((host, port), timeout=timeout_seconds):
1298 |                 status = "open"
1299 |         except (TimeoutError, OSError):
1300 |             status = "closed"
1301 |         elapsed_ms = round((time.perf_counter() - started) * 1000, 2)
1302 |         results.append({"host": host, "port": port, "service": services.get(port, ""), "status": status, "elapsed_ms": elapsed_ms})
1303 |     return results

```

Function: show_probe

Lines: 1306 to 1317

Description: Analyzes and executes show_probe logic.

```

1306 | def show_probe(host: str, ports_text: str, json_output: bool = False) -> None:
1307 |     ports = parse_ports(ports_text)
1308 |     payload = probe_ports(host, ports)
1309 |     cache_artifact("probe", payload)
1310 |     if json_output:
1311 |         print_json(payload)
1312 |         return
1313 |     section("Port Probe")
1314 |     print("Use only on systems and networks you own or are authorized to test.")
1315 |     print(table(["Host", "Port", "Service", "Status", "ms"], [
1316 |         [item["host"], item["port"], item["service"], item["status"], item["elapsed_ms"]] for item in payload
1317 |     ]))

```

Function: show_dns

Lines: 1320 to 1344

Description: Analyzes and executes show_dns logic.

```

1320 | def show_dns(host: str, json_output: bool = False) -> None:
1321 |     addresses: list[str] = []
1322 |     aliases: list[str] = []
1323 |     reverse: list[str] = []
1324 |     try:
1325 |         name, aliases, addresses = socket.gethostbyname_ex(host)
1326 |     except OSError as exc:
1327 |         payload = {"host": host, "error": str(exc)}
1328 |         if json_output:
1329 |             print_json(payload)
1330 |         else:
1331 |             print(f"DNS error: {exc}")
1332 |         return
1333 |     for address in addresses:
1334 |         try:
1335 |             reverse.append(socket.gethostbyaddr(address)[0])
1336 |         except OSError:
1337 |             pass
1338 |     payload = {"host": host, "canonical": name, "aliases": aliases, "addresses": addresses, "reverse": reverse}
1339 |     cache_artifact("dns", payload)
1340 |     if json_output:
1341 |         print_json(payload)
1342 |     return
1343 |     section("DNS")
1344 |     print_json(payload)

```

Function: show_processes

Lines: 1347 to 1367

Description: Analyzes and executes show_processes logic.

```

1347 | def show_processes(json_output: bool = False, limit: int = 40) -> None:
1348 |     rows: list[dict[str, Any]] = []
1349 |     try:
1350 |         completed = subprocess.run(["tasklist", "/FO", "CSV"], capture_output=True, text=True, timeout=15, check=False)
1351 |         reader = csv.DictReader(completed.stdout.splitlines())
1352 |         for row in reader:
1353 |             rows.append(row)
1354 |     except FileNotFoundError:
1355 |         rows = []
1356 |     cache_artifact("ps", rows[:limit])
1357 |     if json_output:
1358 |         print_json(rows[:limit])
1359 |     return
1360 |     section("Processes")
1361 |     if not rows:
1362 |         print("No process rows found.")
1363 |     return
1364 |     print(table(["Image", "PID", "Session", "Memory"], [
1365 |         [row.get("Image Name", ""), row.get("PID", ""), row.get("Session Name", ""), row.get("Mem Usage", "")]
1366 |         for row in rows[:limit]
1367 |     ]))

```

Function: hash_file

Lines: 1370 to 1385

Description: Analyzes and executes hash_file logic.

```

1370 | def hash_file(path: Path) -> dict[str, Any]:
1371 |     sha256 = hashlib.sha256()
1372 |     sha1 = hashlib.sha1()
1373 |     md5 = hashlib.md5()
1374 |     with path.open("rb") as handle:
1375 |         for chunk in iter(lambda: handle.read(1024 * 1024), b''):
1376 |             sha256.update(chunk)
1377 |             sha1.update(chunk)
1378 |             md5.update(chunk)
1379 |     return {
1380 |         "path": relative_path(path),
1381 |         "size": path.stat().st_size,
1382 |         "sha256": sha256.hexdigest(),
1383 |         "sha1": sha1.hexdigest(),
1384 |         "md5": md5.hexdigest(),
1385 |     }

```

Function: show_hash

Lines: 1388 to 1399

Description: Analyzes and executes show_hash logic.

```

1388 | def show_hash(path: Path, json_output: bool = True) -> None:
1389 |     payload = hash_file(path)
1390 |     cache_artifact("hash", payload)
1391 |     if json_output:
1392 |         print_json(payload)
1393 |     return
1394 |     section("File Hashes")
1395 |     print(f"Path: {payload['path']}")
1396 |     print(f"Size: {payload['size']:,}")
1397 |     print(f"SHA256: {payload['sha256']}")
1398 |     print(f"SHA1: {payload['sha1']}")
1399 |     print(f"MD5: {payload['md5']}")

```

Function: scan_file

Lines: 1402 to 1421

Description: Analyzes and executes scan_file logic.

```

1402 | def scan_file(path: Path) -> dict[str, Any]:
1403 |     payload = hash_file(path)
1404 |     max_bytes = 5 * 1024 * 1024
1405 |     with path.open("rb") as handle:
1406 |         data = handle.read(max_bytes)
1407 |     lower = data.lower()
1408 |     ascii_text = lower.decode("latin-1", errors="ignore")
1409 |     findings = []
1410 |     if data.startswith(b"MZ"):
1411 |         findings.append("windows_pe_executable")
1412 |     if b"\x7fELF" in data[:4]:
1413 |         findings.append("linux_elf_executable")
1414 |     if b"PK\x03\x04" in data[:4]:
1415 |         findings.append("zip_or_office_container")
1416 |     for pattern in SUSPICIOUS_FILE_PATTERNS:
1417 |         if pattern in ascii_text:
1418 |             findings.append(f"suspicious_string:{pattern}")
1419 |     payload["findings"] = findings
1420 |     payload["triage"] = "suspicious" if findings else "no_builtin_findings"
1421 |     return payload

```

Function: show_file_scan

Lines: 1424 to 1438

Description: Analyzes and executes show_file_scan logic.

```
1424 | def show_file_scan(path: Path, json_output: bool = False) -> None:
1425 |     payload = scan_file(path)
1426 |     cache_artifact("filescan", payload)
1427 |     if json_output:
1428 |         print_json(payload)
1429 |         return
1430 |     section("File Triage")
1431 |     print(f"Path: {payload['path']}")
1432 |     print(f"Size: {payload['size'],}")
1433 |     print(f"SHA256: {payload['sha256']}")
1434 |     print(f"SHA1: {payload['sha1']}")
1435 |     print(f"MD5: {payload['md5']}")
1436 |     print(f"Triage: {payload['triage']}")
1437 |     if payload["findings"]:
1438 |         print(table(["Finding"], [[item] for item in payload["findings"]]))
```

Function: read_iocs

Lines: 1441 to 1443

Description: Analyzes and executes read_iocs logic.

```
1441 | def read_iocs() -> list[dict[str, Any]]:
1442 |     payload = read_json(IOC_PATH, default={"iocs": []})
1443 |     return payload.get("iocs", [])
```

Function: write_iocs

Lines: 1446 to 1447

Description: Analyzes and executes write_iocs logic.

```
1446 | def write_iocs(iocs: list[dict[str, Any]]) -> None:
1447 |     write_json(IOC_PATH, {"updated_at": utc_now(), "iocs": iocs})
```

Function: classify_ioc

Lines: 1450 to 1465

Description: Analyzes and executes classify_ioc logic.

```

1450 | def classify_ioc(value: str, explicit_type: str | None = None) -> str:
1451 |     if explicit_type:
1452 |         return explicit_type
1453 |     try:
1454 |         ipaddress.ip_address(value)
1455 |         return "ip"
1456 |     except ValueError:
1457 |         pass
1458 |     if value.isdigit() and 1 <= int(value) <= 65535:
1459 |         return "port"
1460 |     lowered = value.lower()
1461 |     if re.fullmatch(r"[a-f0-9]{32}|[a-f0-9]{40}|[a-f0-9]{64}", lowered):
1462 |         return "hash"
1463 |     if "." in value and not any(char.isspace() for char in value):
1464 |         return "domain"
1465 |     return "string"

```

Function: add_ioc

Lines: 1468 to 1480

Description: Analyzes and executes add_ioc logic.

```

1468 | def add_ioc(value: str, ioc_type: str | None = None, note: str = "") -> dict[str, Any]:
1469 |     iocs = read_iocs()
1470 |     entry = {
1471 |         "id": hashlib.sha1(f"{value}|{utc_now()}".encode("utf-8")).hexdigest()[:10],
1472 |         "type": classify_ioc(value, ioc_type),
1473 |         "value": value,
1474 |         "note": note,
1475 |         "created_at": utc_now(),
1476 |     }
1477 |     iocs.append(entry)
1478 |     write_iocs(iocs)
1479 |     cache_artifact("ioc_add", entry)
1480 |     return entry

```

Function: remove_ioc

Lines: 1483 to 1489

Description: Analyzes and executes remove_ioc logic.

```

1483 | def remove_ioc(ioc_id: str) -> bool:
1484 |     iocs = read_iocs()
1485 |     kept = [item for item in iocs if item.get("id") != ioc_id]
1486 |     write_iocs(kept)
1487 |     removed = len(kept) != len(iocs)
1488 |     cache_artifact("ioc_remove", {"ioc_id": ioc_id, "removed": removed})
1489 |     return removed

```

Function: search_text_files

Lines: 1492 to 1514

Description: Analyzes and executes search_text_files logic.

```

1492 | def search_text_files(pattern: str, paths: list[Path], limit: int = 50) -> list[dict[str, Any]]:
1493 |     results = []
1494 |     lowered = pattern.lower()
1495 |     for path in paths:
1496 |         if not path.exists() or not path.is_file():
1497 |             continue
1498 |         try:
1499 |             if path.suffix.lower() in TEXT_SEARCH_SKIP_SUFFIXES:
1500 |                 continue
1501 |             if path.stat().st_size > TEXT_SEARCH_MAX_FILE_BYTES:
1502 |                 continue
1503 |         except OSError:
1504 |             continue
1505 |         try:
1506 |             with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
1507 |                 for line_number, line in enumerate(handle, start=1):
1508 |                     if lowered in line.lower():
1509 |                         results.append({"path": relative_path(path), "line": line_number, "text": line.strip()[:240])
1510 |                     if len(results) >= limit:
1511 |                         return results
1512 |         except OSError:
1513 |             continue
1514 |     return results

```

Function: show_hunt

Lines: 1517 to 1528

Description: Analyzes and executes show_hunt logic.

```

1517 | def show_hunt(pattern: str, path: Path | None = None, json_output: bool = False, limit: int = 50) -> None:
1518 |     paths = [path] if path else all_csv_sources(include_exports=True)
1519 |     payload = {"pattern": pattern, "matches": search_text_files(pattern, paths, limit=limit)}
1520 |     cache_artifact("hunt", payload)
1521 |     if json_output:
1522 |         print_json(payload)
1523 |         return
1524 |     section("Hunt")
1525 |     if not payload["matches"]:
1526 |         print("No matches.")
1527 |         return
1528 |     print(table(["Path", "Line", "Text"], [[item["path"], item["line"], item["text"]] for item in payload["matches"]]))

```

Function: show_ioc

Lines: 1531 to 1584

Description: Analyzes and executes show_ioc logic.

```

1531 | def show_ioc(args: list[str], json_output: bool = False) -> None:
1532 |     action = args[0].lower() if args else "list"
1533 |     if action == "list":
1534 |         payload = read_iocs()
1535 |         cache_artifact("ioc_list", payload)
1536 |         if json_output:
1537 |             print_json(payload)
1538 |             return
1539 |         section("IOCs")
1540 |         if not payload:
1541 |             print("No IOCs stored.")
1542 |             return
1543 |         print(table(["ID", "Type", "Value", "Note"], [[item["id"], item["type"], item["value"], item.get("note", "")]
1544 |         return
1545 |     if action == "add":
1546 |         if len(args) < 2:
1547 |             raise ValueError("usage: ioc add <value> [type] [note]")
1548 |         value = args[1]
1549 |         ioc_type = args[2] if len(args) >= 3 and args[2] in {"ip", "domain", "hash", "port", "string", "malware"} else "ip"
1550 |         note_start = 3 if ioc_type else 2
1551 |         payload = add_ioc(value, ioc_type, " ".join(args[note_start:]))
1552 |         if json_output:
1553 |             print_json(payload)
1554 |         else:
1555 |             print(f"Added IOC {payload['id']} ({payload['type']}): {payload['value']}")
1556 |         return
1557 |     if action in {"rm", "remove", "delete"}:
1558 |         if len(args) < 2:
1559 |             raise ValueError("usage: ioc remove <id>")
1560 |         removed = remove_ioc(args[1])
1561 |         print("removed" if removed else "not found")
1562 |         return
1563 |     if action == "hunt":
1564 |         iocs = read_iocs()
1565 |         matches = []
1566 |         for item in iocs:
1567 |             found = search_text_files(str(item["value"]), all_csv_sources(include_exports=True), limit=20)
1568 |             if found:
1569 |                 matches.append({"ioc": item, "matches": found})
1570 |         cache_artifact("ioc_hunt", matches)
1571 |         if json_output:
1572 |             print_json(matches)
1573 |             return
1574 |         section("IOC Hunt")
1575 |         if not matches:
1576 |             print("No IOC matches.")
1577 |             return
1578 |         rows = []
1579 |         for bundle in matches:
1580 |             for match in bundle["matches"]:
1581 |                 rows.append([bundle["ioc"]["value"], match["path"], match["line"], match["text"]])
1582 |         print(table(["IOC", "Path", "Line", "Text"], rows[:80]))
1583 |         return
1584 |     raise ValueError("ioc actions: list, add, remove, hunt")

```

Function: shell_path

Lines: 1587 to 1598

Description: Analyzes and executes shell_path logic.

```

1587 | def shell_path(path_text: str | None = None) -> Path:
1588 |     if not path_text:
1589 |         return Path(SHELL_STATE.get("cwd", ROOT_DIR))
1590 |     path = Path(path_text)
1591 |     if not path.is_absolute():
1592 |         path = Path(SHELL_STATE.get("cwd", ROOT_DIR)) / path
1593 |     resolved = path.resolve()
1594 |     try:
1595 |         resolved.relative_to(ROOT_DIR)
1596 |     except ValueError:
1597 |         raise ValueError("path must stay inside the IDS Sentinel home directory")
1598 |     return resolved

```

Function: shell_cd

Lines: 1601 to 1606

Description: Analyzes and executes shell_cd logic.

```

1601 | def shell_cd(path_text: str | None) -> None:
1602 |     path = shell_path(path_text or ".")
1603 |     if not path.exists() or not path.is_dir():
1604 |         raise ValueError(f"not a directory: {path_text}")
1605 |     SHELL_STATE["cwd"] = path
1606 |     print(relative_path(path) or ".")

```

Function: shell_ls

Lines: 1609 to 1623

Description: Analyzes and executes shell_ls logic.

```

1609 | def shell_ls(path_text: str | None = None, all_files: bool = False) -> None:
1610 |     path = shell_path(path_text or ".")
1611 |     if path.is_file():
1612 |         print(relative_path(path))
1613 |         return
1614 |     rows = []
1615 |     for child in sorted(path.iterdir(), key=lambda item: (item.is_file(), item.name.lower())):
1616 |         if not all_files and child.name.startswith("."):
1617 |             continue
1618 |         rows.append([
1619 |             child.name + ("/" if child.is_dir() else ""),
1620 |             "dir" if child.is_dir() else child.stat().st_size,
1621 |             datetime.fromtimestamp(child.stat().st_mtime).isoformat(timespec="seconds"),
1622 |         ])
1623 |     print(table(["Name", "Size", "Modified"], rows))

```

Function: shell_cat

Lines: 1626 to 1633

Description: Analyzes and executes shell_cat logic.

```

1626 | def shell_cat(path_text: str, limit_bytes: int = 1024 * 1024) -> None:
1627 |     path = shell_path(path_text)
1628 |     if not path.is_file():
1629 |         raise ValueError("cat requires a file")
1630 |     if path.stat().st_size > limit_bytes:
1631 |         raise ValueError("file is too large for cat; use head, tail, or grep")
1632 |     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1633 |         print(handle.read())

```

Function: shell_head

Lines: 1636 to 1642

Description: Analyzes and executes shell_head logic.

```

1636 | def shell_head(path_text: str, lines: int = 20) -> None:
1637 |     path = shell_path(path_text)
1638 |     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1639 |         for index, line in enumerate(handle):
1640 |             if index >= lines:
1641 |                 break
1642 |             print(line.rstrip())

```

Function: shell_tail

Lines: 1645 to 1654

Description: Analyzes and executes shell_tail logic.

```

1645 | def shell_tail(path_text: str, lines: int = 20) -> None:
1646 |     path = shell_path(path_text)
1647 |     buffer: list[str] = []
1648 |     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1649 |         for line in handle:
1650 |             buffer.append(line.rstrip())
1651 |             if len(buffer) > lines:
1652 |                 buffer.pop(0)
1653 |     for line in buffer:
1654 |         print(line)

```

Function: shell_grep

Lines: 1657 to 1664

Description: Analyzes and executes shell_grep logic.

```

1657 | def shell_grep(pattern: str, path_text: str | None = None, limit: int = 50) -> None:
1658 |     path = shell_path(path_text or ".")
1659 |     paths = [path] if path.is_file() else [item for item in path.rglob("**") if item.is_file()]
1660 |     matches = search_text_files(pattern, paths, limit=limit)
1661 |     if not matches:
1662 |         print("No matches.")
1663 |         return
1664 |     print(table(["Path", "Line", "Text"], [[item["path"], item["line"], item["text"]] for item in matches]))

```

Function: shell_find

Lines: 1667 to 1676

Description: Analyzes and executes shell_find logic.

```

1667 | def shell_find(pattern: str = "*", path_text: str | None = None, limit: int = 200) -> None:
1668 |     root = shell_path(path_text or ".")
1669 |     rows = []
1670 |     iterator = root.rglob("*") if root.is_dir() else [root]
1671 |     for item in iterator:
1672 |         if fnmatch.fnmatch(item.name.lower(), pattern.lower()):
1673 |             rows.append([relative_path(item), "dir" if item.is_dir() else item.stat().st_size])
1674 |             if len(rows) >= limit:
1675 |                 break
1676 |     print(table(["Path", "Size"], rows))

```

Function: shell_wc

Lines: 1679 to 1687

Description: Analyzes and executes shell_wc logic.

```

1679 | def shell_wc(path_text: str) -> None:
1680 |     path = shell_path(path_text)
1681 |     lines = words = chars = 0
1682 |     with path.open("r", encoding="utf-8", errors="ignore") as handle:
1683 |         for line in handle:
1684 |             lines += 1
1685 |             words += len(line.split())
1686 |             chars += len(line)
1687 |     print(table(["Lines", "Words", "Chars", "Path"], [[lines, words, chars, relative_path(path)]]))

```

Function: shell_du

Lines: 1690 to 1696

Description: Analyzes and executes shell_du logic.

```

1690 | def shell_du(path_text: str | None = None) -> None:
1691 |     path = shell_path(path_text or ".")
1692 |     if path.is_file():
1693 |         size = path.stat().st_size
1694 |     else:
1695 |         size = sum(item.stat().st_size for item in path.rglob("*") if item.is_file())
1696 |     print(table(["Path", "Bytes", "MB"], [[relative_path(path) or ".", size, round(size / (1024 * 1024), 2)]]))

```

Function: shell_stat

Lines: 1699 to 1710

Description: Analyzes and executes shell_stat logic.

```

1699 | def shell_stat(path_text: str) -> None:
1700 |     path = shell_path(path_text)
1701 |     stat = path.stat()
1702 |     print_json(
1703 |         {
1704 |             "path": relative_path(path),
1705 |             "type": "directory" if path.is_dir() else "file",
1706 |             "bytes": stat.st_size,
1707 |             "modified": datetime.fromtimestamp(stat.st_mtime).isoformat(timespec="seconds"),
1708 |             "created": datetime.fromtimestamp(stat.st_ctime).isoformat(timespec="seconds"),
1709 |         }
1710 |     )

```

Function: show_status

Lines: 1713 to 1770

Description: Analyzes and executes show_status logic.

```
1713 | def show_status(json_output: bool = False) -> None:
1714 |     payload = {
1715 |         "installation": {
1716 |             "version": __version__,
1717 |             "runtime_mode": RUNTIME_MODE,
1718 |             "home_dir": str(ROOT_DIR),
1719 |             "env_override": bool(ENV_ROOT),
1720 |         },
1721 |         "datasets": summarize_all_datasets(),
1722 |         "model": read_json(MODEL_PATH),
1723 |         "latest_export": latest_export_summary(),
1724 |         "recent_runs": list_run_summaries(),
1725 |     }
1726 |     cache_artifact("status", payload)
1727 |     if json_output:
1728 |         print_json(payload)
1729 |         return
1730 |
1731 |     section("IDS Sentinel Terminal Status")
1732 |     print(
1733 |         f"Version: {payload['installation']['version']} | mode: {payload['installation']['runtime_mode']} | "
1734 |         f"home: {payload['installation']['home_dir']}"
1735 |     )
1736 |     print()
1737 |     dataset_rows = []
1738 |     for name, item in payload["datasets"].items():
1739 |         dataset_rows.append(
1740 |             [
1741 |                 name,
1742 |                 item["path"],
1743 |                 item["rows"],
1744 |                 f"{item['size_mb']} MB",
1745 |                 item["label_counts"].get("0", 0),
1746 |                 item["label_counts"].get("1", 0),
1747 |                 percent(item["label_counts"].get("1", 0), item["rows"]),
1748 |             ]
1749 |         )
1750 |     print(table(["Set", "Path", "Rows", "Size", "Normal", "Attack", "Attack Share"], dataset_rows))
1751 |
1752 |     section("Self-Learning Model")
1753 |     model = payload["model"]
1754 |     if not model:
1755 |         print("No model yet. Run: learn")
1756 |     else:
1757 |         print(f"Model: {model['model_type']} | rows learned: {model['total_rows']:,} | created: {model['created_at']}")
1758 |         print(table(["Indicator", "Separation", "Normal Mean", "Attack Mean"], [
1759 |             [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1760 |             for item in model.get("top_indicators", [])[:6]
1761 |         ]))
1762 |
1763 |     latest = payload["latest_export"]
1764 |     section("Latest Downloadable Analysis")
1765 |     if latest:
1766 |         print(f"CSV: {latest['export_csv']}")
1767 |         print(f"JSON: {latest['export_json']}")
1768 |         print(f"Rows: {latest['rows_analyzed']:,} | average risk: {latest['average_risk_score']:.4f}")
1769 |     else:
1770 |         print("No analysis export yet. Run: scan")
```

Function: show_traffic

Lines: 1773 to 1806

Description: Analyzes and executes show_traffic logic.

```
1773 | def show_traffic(json_output: bool = False) -> None:
1774 |     payload = summarize_all_datasets()
1775 |     cache_artifact("traffic", payload)
1776 |     if json_output:
1777 |         print_json(payload)
1778 |         return
1779 |
1780 |     section("Traffic Data")
1781 |     rows = []
1782 |     for name, item in payload.items():
1783 |         rows.append(
1784 |             [
1785 |                 name,
1786 |                 item["rows"],
1787 |                 f"{item['size_mb']} MB",
1788 |                 item["total_src_bytes"],
1789 |                 item["total_dst_bytes"],
1790 |                 ", ".join(f"{value}:{count}" for value, count in item["top_protocols"][:3]),
1791 |             ]
1792 |         )
1793 |     print(table(["Set", "Rows", "Size", "Source Bytes", "Dest Bytes", "Top Encoded Protocols"], rows))
1794 |
1795 |     section("Top Encoded Services And Flags")
1796 |     rows = []
1797 |     for name, item in payload.items():
1798 |         rows.append(
1799 |             [
1800 |                 name,
1801 |                 ", ".join(f"{value}:{count}" for value, count in item["top_services"][:5]),
1802 |                 ", ".join(f"{value}:{count}" for value, count in item["top_flags"][:5]),
1803 |             ]
1804 |         )
1805 |     print(table(["Set", "Services", "Flags"], rows))
1806 |     print("\nProtocol, service, and flag values are encoded IDs in these CSV files.")
```

Function: show_attacks

Lines: 1809 to 1833

Description: Analyzes and executes show_attacks logic.

```

1809 | def show_attacks(json_output: bool = False) -> None:
1810 |     datasets = summarize_all_datasets()
1811 |     model = read_json(MODEL_PATH)
1812 |     payload = {"datasets": datasets, "model_indicators": model.get("top_indicators", []) if model else []}
1813 |     cache_artifact("attacks", payload)
1814 |     if json_output:
1815 |         print_json(payload)
1816 |         return
1817 |
1818 |     section("Attack Distribution")
1819 |     rows = []
1820 |     for name, item in datasets.items():
1821 |         total = item["rows"]
1822 |         rows.append([name, "normal", item["label_counts"].get("0", 0), percent(item["label_counts"].get("0", 0), total)])
1823 |         rows.append([name, "attack", item["label_counts"].get("1", 0), percent(item["label_counts"].get("1", 0), total)])
1824 |     print(table(["Set", "Label", "Rows", "Share"], rows))
1825 |
1826 |     section("Learned Attack Indicators")
1827 |     if not model:
1828 |         print("No learned model yet. Run: learn")
1829 |         return
1830 |     print(table(["Feature", "Separation", "Normal Mean", "Attack Mean"], [
1831 |         [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1832 |         for item in model.get("top_indicators", [])[:10]
1833 |     ]))

```

Function: show_malware

Lines: 1836 to 1859

Description: Analyzes and executes show_malware logic.

```

1836 | def show_malware(json_output: bool = False, limit: int = 5000) -> None:
1837 |     model = load_or_learn_model()
1838 |     summary = analyze_csv(TEST_CSV, limit=limit, export=False, model=model)
1839 |     malware_like = summary["family_counts"].get("malware_like_activity", 0)
1840 |     privilege = summary["family_counts"].get("privilege_escalation", 0)
1841 |     payload = {
1842 |         "note": "The bundled CSVs have binary normal/attack labels, not named malware-family labels. These are behaviors.",
1843 |         "rows_analyzed": summary["rows_analyzed"],
1844 |         "malware_like_activity": malware_like,
1845 |         "privilege_escalation": privilege,
1846 |         "family_counts": summary["family_counts"],
1847 |     }
1848 |     cache_artifact("malware", payload)
1849 |     if json_output:
1850 |         print_json(payload)
1851 |         return
1852 |
1853 |     section("Malware-Like Behavior")
1854 |     print(payload["note"])
1855 |     print(table(["Indicator", "Rows", "Share"], [
1856 |         ["malware_like_activity", malware_like, percent(malware_like, summary["rows_analyzed"])],
1857 |         ["privilege_escalation", privilege, percent(privilege, summary["rows_analyzed"])],
1858 |     ]))
1859 |     print(table(["Family", "Rows"], [[name, count] for name, count in sorted(summary["family_counts"].items())]))

```

Function: show_learn

Lines: 1862 to 1878

Description: Analyzes and executes show_learn logic.

```

1862 | def show_learn(model: dict[str, Any], json_output: bool = False) -> None:
1863 |     if json_output:
1864 |         print_json(model)
1865 |         return
1866 |     section("Self-Learning Complete")
1867 |     print(f"Model: {relative_path(MODEL_PATH)}")
1868 |     print(f"Rows learned: {model['total_rows']:,}")
1869 |     print(f"Created: {model['created_at']}")
1870 |     print(table(["Source", "Rows", "Labels"], [
1871 |         [item["path"], item["rows_used"], ", ".join(f"{label}:{count}" for label, count in item["label_counts"].items())]
1872 |         for item in model["sources"]
1873 |     ]))
1874 |     print()
1875 |     print(table(["Top Indicator", "Separation", "Normal Mean", "Attack Mean"], [
1876 |         [item["feature"], item["separation"], item["normal_mean"], item["attack_mean"]]
1877 |         for item in model["top_indicators"][:8]
1878 |     ]))

```

Function: show_scan

Lines: 1881 to 1909

Description: Analyzes and executes show_scan logic.

```

1881 | def show_scan(summary: dict[str, Any], json_output: bool = False) -> None:
1882 |     if json_output:
1883 |         print_json(summary)
1884 |         return
1885 |     section("Traffic Analysis")
1886 |     print(f"Source: {summary['source_file']}")
1887 |     print(f"Rows analyzed: {summary['rows_analyzed']:,}")
1888 |     print(f"Average risk: {summary['average_risk_score']:.4f}")
1889 |     print(table(["Prediction", "Rows", "Share"], [
1890 |         [BINARY_LABELS.get(label, label), count, percent(count, summary["rows_analyzed"])]
1891 |         for label, count in sorted(summary["predicted_counts"].items())
1892 |     ]))
1893 |     print(table(["Risk", "Rows"], [[name, count] for name, count in sorted(summary["risk_counts"].items())]))
1894 |     print(table(["Family", "Rows"], [[name, count] for name, count in sorted(summary["family_counts"].items())]))
1895 |     print()
1896 |     print(table(["Accuracy", "Precision", "Recall", "F1", "TP", "TN", "FP", "FN"], [
1897 |         summary["metrics"]["accuracy"],
1898 |         summary["metrics"]["precision"],
1899 |         summary["metrics"]["recall"],
1900 |         summary["metrics"]["f1"],
1901 |         summary["metrics"]["tp"],
1902 |         summary["metrics"]["tn"],
1903 |         summary["metrics"]["fp"],
1904 |         summary["metrics"]["fn"],
1905 |     ]))
1906 |     if summary.get("export_csv"):
1907 |         print()
1908 |         print(f"Downloadable CSV: {summary['export_csv']}")
1909 |         print(f"Summary JSON: {summary['export_json']}")

```

Function: show_reports

Lines: 1912 to 1924

Description: Analyzes and executes show_reports logic.

```

1912 | def show_reports(json_output: bool = False, limit: int | None = 20) -> None:
1913 |     reports = list_reports(limit)
1914 |     cache_artifact("reports", reports)
1915 |     if json_output:
1916 |         print_json(reports)
1917 |         return
1918 |     section("Downloadable Reports")
1919 |     if not reports:
1920 |         print("No product reports yet. Run: scan")
1921 |         return
1922 |     print(table(["Name", "Path", "Size KB", "Modified"], [
1923 |         [item["name"], item["path"], item["size_kb"], item["modified"]] for item in reports
1924 |     ]))

```

Function: show_runs

Lines: 1927 to 1950

Description: Analyzes and executes show_runs logic.

```

1927 | def show_runs(json_output: bool = False, limit: int | None = 10) -> None:
1928 |     runs = list_run_summaries(limit)
1929 |     cache_artifact("runs", runs)
1930 |     if json_output:
1931 |         print_json(runs)
1932 |         return
1933 |     section("ML Training Runs")
1934 |     if not runs:
1935 |         print("No training runs found.")
1936 |         return
1937 |     rows = []
1938 |     for run in runs:
1939 |         best = (run.get("results") or [{}])[0]
1940 |         metrics = best.get("metrics", {})
1941 |         rows.append(
1942 |             [
1943 |                 run.get("run_id", "n/a"),
1944 |                 run.get("kind", "n/a"),
1945 |                 best.get("label", "n/a"),
1946 |                 metrics.get("accuracy", 0),
1947 |                 metrics.get("f1", 0),
1948 |             ]
1949 |         )
1950 |     print(table(["Run", "Kind", "Best Model", "Accuracy", "F1"], rows))

```

Function: print_shell_help

Lines: 1953 to 1985

Description: Analyzes and executes print_shell_help logic.

```

1953 | def print_shell_help() -> None:
1954 |     section("Commands")
1955 |     print(table(["Command", "Action"], [
1956 |         ["status", "product dashboard: datasets, model, latest export"],
1957 |         ["traffic", "summarize traffic volumes, services, protocols, flags"],
1958 |         ["attacks", "attack distribution and learned attack indicators"],
1959 |         ["malware [limit]", "show malware-like and privilege behavior indicators"],
1960 |         ["learn [full|quick]", "build/update the self-learning profile"],
1961 |         ["scan [path] [limit|all]", "analyze traffic and write downloadable CSV/JSON"],
1962 |         ["export [path] [limit|all]", "same as scan; defaults to all rows"],
1963 |         ["datasets", "show local and external IDS dataset catalog"],
1964 |         ["import <csv> [name]", "copy a CSV into automation/product/imports and index it"],
1965 |         ["download <url> [name]", "download a public dataset/file into imports"],
1966 |         ["index [csv] [limit|all]", "inspect columns, labels, and top values"],
1967 |         ["hunt <term> [path] [limit]", "search datasets, imports, and exported reports"],
1968 |         ["ioc list|add|remove|hunt", "store and hunt indicators of compromise"],
1969 |         ["ports [limit]", "show listening local ports and services"],
1970 |         ["netstat [limit]", "show local network connections"],
1971 |         ["port <number>", "explain a port and show local matches"],
1972 |         ["probe <host> <ports>", "authorized TCP connect probe, e.g. probe 127.0.0.1 22,80,443"],
1973 |         ["dns <host>", "resolve DNS and reverse names"],
1974 |         ["ps [limit]", "list local processes"],
1975 |         ["hash <file>", "calculate SHA256/SHA1/MD5"],
1976 |         ["filescan <file>", "hash and check built-in suspicious file strings"],
1977 |         ["pwd | cd | ls", "basic project filesystem navigation"],
1978 |         ["cat | head | tail | grep", "text inspection commands"],
1979 |         ["find | wc | du | stat", "file discovery and measurement commands"],
1980 |         ["cache [limit]", "list cached command artifacts"],
1981 |         ["reports [limit]", "list downloadable CSV/JSON reports"],
1982 |         ["runs [limit]", "list previous ML training runs"],
1983 |         ["clear", "clear the terminal"],
1984 |         ["exit", "quit"],
1985 |     ]))

```

Function: parse_limit

Lines: 1988 to 1996

Description: Analyzes and executes parse_limit logic.

```

1988 | def parse_limit(value: str | None, default: int | None) -> int | None:
1989 |     if value is None:
1990 |         return default
1991 |     if value.lower() == "all":
1992 |         return None
1993 |     parsed = int(value)
1994 |     if parsed < 0:
1995 |         raise ValueError("limit must be zero or greater")
1996 |     return parsed

```

Function: command_shell

Lines: 1999 to 2014

Description: Analyzes and executes command_shell logic.

```
1999 | def command_shell() -> None:
2000 |     ensure_product_dirs()
2001 |     print("IDS Sentinel Terminal. Type 'help' for commands, 'exit' to quit.")
2002 |     while True:
2003 |         try:
2004 |             raw = input("ids-sentinel> ").strip()
2005 |         except EOFError:
2006 |             print()
2007 |             return
2008 |         if not raw:
2009 |             continue
2010 |         try:
2011 |             if run_shell_command(raw):
2012 |                 return
2013 |         except Exception as exc:
2014 |             print(f"error: {exc}")
```

Function: split_shell_command

Lines: 2017 to 2020

Description: Analyzes and executes split_shell_command logic.

```
2017 | def split_shell_command(raw: str) -> list[str]:
2018 |     if os.name == "nt":
2019 |         raw = raw.replace("\\", "/")
2020 |     return shlex.split(raw)
```

Function: run_shell_command

Lines: 2023 to 2164

Description: Analyzes and executes run_shell_command logic.

```

2023 | def run_shell_command(raw: str) -> bool:
2024 |     SHELL_STATE["history"].append(raw)
2025 |     parts = split_shell_command(raw)
2026 |     if not parts:
2027 |         return False
2028 |     command, *args = parts
2029 |     command = command.lower()
2030 |
2031 |     if command in {"exit", "quit", "q"}:
2032 |         return True
2033 |     if command == "help":
2034 |         print_shell_help()
2035 |     elif command == "clear":
2036 |         os.system("cls" if os.name == "nt" else "clear")
2037 |     elif command == "history":
2038 |         print(table(["#", "Command"], [[index + 1, value] for index, value in enumerate(SHELL_STATE["history"][-50:
2039 |     elif command == "pwd":
2040 |         print(relative_path(Path(SHELL_STATE.get("cwd", ROOT_DIR))) or ".")
2041 |     elif command == "cd":
2042 |         shell_cd(args[0] if args else ".")
2043 |     elif command == "ls":
2044 |         all_files = "-a" in args
2045 |         path_args = [arg for arg in args if arg != "-a"]
2046 |         shell_ls(path_args[0] if path_args else ".", all_files=all_files)
2047 |     elif command == "cat":
2048 |         if not args:
2049 |             print("usage: cat <file>")
2050 |         else:
2051 |             shell_cat(args[0])
2052 |     elif command == "head":
2053 |         if not args:
2054 |             print("usage: head <file> [lines]")
2055 |         else:
2056 |             shell_head(args[0], int(args[1]) if len(args) > 1 else 20)
2057 |     elif command == "tail":
2058 |         if not args:
2059 |             print("usage: tail <file> [lines]")
2060 |         else:
2061 |             shell_tail(args[0], int(args[1]) if len(args) > 1 else 20)
2062 |     elif command == "grep":
2063 |         if not args:
2064 |             print("usage: grep <pattern> [path] [limit]")
2065 |         else:
2066 |             shell_grep(args[0], args[1] if len(args) > 1 else ".", int(args[2]) if len(args) > 2 else 50)
2067 |     elif command == "find":
2068 |         shell_find(args[0] if args else "*", args[1] if len(args) > 1 else ".", int(args[2]) if len(args) > 2 else 2
2069 |     elif command == "wc":
2070 |         if not args:
2071 |             print("usage: wc <file>")
2072 |         else:
2073 |             shell_wc(args[0])
2074 |     elif command == "du":
2075 |         shell_du(args[0] if args else ".")
2076 |     elif command == "stat":
2077 |         if not args:
2078 |             print("usage: stat <path>")
2079 |         else:
2080 |             shell_stat(args[0])
2081 |     elif command in {"status", "overview", "dashboard"}:
2082 |         show_status()
2083 |     elif command in {"traffic", "data"}:
2084 |         show_traffic()
2085 |     elif command in {"attack", "attacks"}:
2086 |         show_attacks()
2087 |     elif command in {"malware", "malwares"}:
2088 |         show_malware(limit=parse_limit(args[0], 5000) if args else 5000)
2089 |     elif command == "learn":
2090 |         mode = args[0].lower() if args else "full"

```

```

2091 |         limit = 20000 if mode == "quick" else None
2092 |         show_learn(learn_model(limit=limit, include_generated=True))
2093 |     elif command in {"scan", "analyze"}:
2094 |         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_DIR)))
2095 |         limit = parse_limit(args[1], 5000) if len(args) > 1 else 5000
2096 |         show_scan(analyze_csv(path, limit=limit, export=True))
2097 |     elif command == "export":
2098 |         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_DIR)))
2099 |         limit = parse_limit(args[1], None) if len(args) > 1 else None
2100 |         show_scan(analyze_csv(path, limit=limit, export=True))
2101 |     elif command in {"datasets", "catalog"}:
2102 |         show_dataset_catalog()
2103 |     elif command == "import":
2104 |         if not args:
2105 |             print("usage: import <csv-path> [name]")
2106 |         else:
2107 |             show_import(import_csv(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))), args[1] if args else None))
2108 |     elif command == "download":
2109 |         if not args:
2110 |             print("usage: download <url> [name]")
2111 |         else:
2112 |             downloaded = download_url(args[0], args[1] if len(args) > 1 else None)
2113 |             print(f"Downloaded: {relative_path(downloaded)}")
2114 |     elif command == "index":
2115 |         path = resolve_readable_path(args[0] if args else None, default=TEST_CSV, base=Path(SHELL_STATE.get("cwd", ROOT_DIR)))
2116 |         limit = parse_limit(args[1], 50000) if len(args) > 1 else 50000
2117 |         show_index(path, limit=limit)
2118 |     elif command == "hunt":
2119 |         if not args:
2120 |             print("usage: hunt <term> [path] [limit]")
2121 |         else:
2122 |             show_hunt(args[0], resolve_readable_path(args[1], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))) if len(args) > 1 else None, limit=parse_limit(args[2], None) if len(args) > 3 else None)
2123 |     elif command == "ioc":
2124 |         show_ioc(args)
2125 |     elif command in {"ports", "listeners"}:
2126 |         show_netstat(only_listening=True, limit=parse_limit(args[0], 40) if args else 40)
2127 |     elif command in {"netstat", "connections"}:
2128 |         show_netstat(only_listening=False, limit=parse_limit(args[0], 40) if args else 40)
2129 |     elif command == "port":
2130 |         if not args:
2131 |             print("usage: port <number>")
2132 |         else:
2133 |             show_port(int(args[0]))
2134 |     elif command == "probe":
2135 |         if len(args) < 2:
2136 |             print("usage: probe <host> <ports>")
2137 |         else:
2138 |             show_probe(args[0], args[1])
2139 |     elif command == "dns":
2140 |         if not args:
2141 |             print("usage: dns <host>")
2142 |         else:
2143 |             show_dns(args[0])
2144 |     elif command == "ps":
2145 |         show_processes(limit=parse_limit(args[0], 40) if args else 40)
2146 |     elif command == "hash":
2147 |         if not args:
2148 |             print("usage: hash <file>")
2149 |         else:
2150 |             show_hash(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))), json_output=False)
2151 |     elif command in {"filescan", "scanfile"}:
2152 |         if not args:
2153 |             print("usage: filescan <file>")
2154 |         else:
2155 |             show_file_scan(resolve_readable_path(args[0], base=Path(SHELL_STATE.get("cwd", ROOT_DIR))))
2156 |     elif command in {"reports", "downloads"}:
2157 |         show_reports(limit=parse_limit(args[0], 20) if args else 20)
2158 |     elif command == "cache":
2159 |         show_cache(limit=parse_limit(args[0], 40) if args else 40)

```

```
2160 |     elif command == "runs":
2161 |         show_runs(limit=parse_limit(args[0], 10) if args else 10)
2162 |     else:
2163 |         print(f"Unknown command: {command}. Type 'help'.")
2164 |     return False
```

Function: build_parser

Lines: 2167 to 2254

Description: Analyzes and executes build_parser logic.

```

2167 | def build_parser() -> argparse.ArgumentParser:
2168 |     parser = argparse.ArgumentParser(description="IDS Sentinel Terminal for defensive CSV traffic analysis and local
2169 |     parser.add_argument("--json", action="store_true", help="Print JSON for commands that support it.")
2170 |     parser.add_argument("--version", action="version", version=f"IDS Sentinel Terminal {__version__}")
2171 |     subparsers = parser.add_subparsers(dest="command")
2172 |
2173 |     subparsers.add_parser("shell", help="Open IDS Sentinel Terminal interactive mode.")
2174 |     subparsers.add_parser("gui", help="Open the graphical product console.")
2175 |     subparsers.add_parser("status", help="Show product status.")
2176 |     subparsers.add_parser("traffic", help="Show traffic data.")
2177 |     subparsers.add_parser("attacks", help="Show attacks and learned indicators.")
2178 |     subparsers.add_parser("datasets", help="Show local and external IDS dataset catalog.")
2179 |
2180 |     malware_parser = subparsers.add_parser("malware", help="Show malware-like behavior indicators.")
2181 |     malware_parser.add_argument("--limit", type=int, default=5000)
2182 |
2183 |     learn_parser = subparsers.add_parser("learn", help="Build/update the self-learning model.")
2184 |     learn_parser.add_argument("--quick", action="store_true", help="Use a 20,000-row sample instead of all rows.")
2185 |     learn_parser.add_argument("--full", action="store_true", help="Use all source rows. This is the default.")
2186 |     learn_parser.add_argument("--include-test", action="store_true", help="Also learn from kddtest.csv labels.")
2187 |     learn_parser.add_argument("--skip-generated", action="store_true", help="Do not learn from terminal-generated CSVs.")
2188 |
2189 |     scan_parser = subparsers.add_parser("scan", help="Analyze a CSV and export CSV/JSON results.")
2190 |     scan_parser.add_argument("path", nargs="?", default=None)
2191 |     scan_parser.add_argument("--limit", type=int, default=5000)
2192 |     scan_parser.add_argument("--all", action="store_true", help="Analyze all rows.")
2193 |     scan_parser.add_argument("--no-export", action="store_true", help="Only print summary; do not write downloadable results.")
2194 |
2195 |     export_parser = subparsers.add_parser("export", help="Analyze and export all rows by default.")
2196 |     export_parser.add_argument("path", nargs="?", default=None)
2197 |     export_parser.add_argument("--limit", type=int)
2198 |
2199 |     import_parser = subparsers.add_parser("import", help="Copy a CSV into product imports and index it.")
2200 |     import_parser.add_argument("path")
2201 |     import_parser.add_argument("--name")
2202 |
2203 |     download_parser = subparsers.add_parser("download", help="Download a public URL into product imports.")
2204 |     download_parser.add_argument("url")
2205 |     download_parser.add_argument("--name")
2206 |
2207 |     index_parser = subparsers.add_parser("index", help="Inspect a CSV file.")
2208 |     index_parser.add_argument("path", nargs="?", default=None)
2209 |     index_parser.add_argument("--limit", type=int, default=50000)
2210 |     index_parser.add_argument("--all", action="store_true")
2211 |
2212 |     hunt_parser = subparsers.add_parser("hunt", help="Search datasets, imports, and reports for text.")
2213 |     hunt_parser.add_argument("pattern")
2214 |     hunt_parser.add_argument("--path")
2215 |     hunt_parser.add_argument("--limit", type=int, default=50)
2216 |
2217 |     ioc_parser = subparsers.add_parser("ioc", help="Manage and hunt indicators of compromise.")
2218 |     ioc_parser.add_argument("ioc_args", nargs="*")
2219 |
2220 |     netstat_parser = subparsers.add_parser("netstat", help="Show local network connections.")
2221 |     netstat_parser.add_argument("--limit", type=int, default=40)
2222 |     netstat_parser.add_argument("--listening", action="store_true")
2223 |
2224 |     ports_parser = subparsers.add_parser("ports", help="Show local listening ports.")
2225 |     ports_parser.add_argument("--limit", type=int, default=40)
2226 |
2227 |     port_parser = subparsers.add_parser("port", help="Explain a port and show local matches.")
2228 |     port_parser.add_argument("number", type=int)
2229 |
2230 |     probe_parser = subparsers.add_parser("probe", help="Authorized TCP connect probe.")
2231 |     probe_parser.add_argument("host")
2232 |     probe_parser.add_argument("ports")
2233 |
2234 |     dns_parser = subparsers.add_parser("dns", help="Resolve a host.")

```

```

2235 |     dns_parser.add_argument("host")
2236 |
2237 |     ps_parser = subparsers.add_parser("ps", help="List local processes.")
2238 |     ps_parser.add_argument("--limit", type=int, default=40)
2239 |
2240 |     hash_parser = subparsers.add_parser("hash", help="Hash a file.")
2241 |     hash_parser.add_argument("path")
2242 |
2243 |     filescan_parser = subparsers.add_parser("filescan", help="Hash and triage a file.")
2244 |     filescan_parser.add_argument("path")
2245 |
2246 |     reports_parser = subparsers.add_parser("reports", help="List generated downloadable reports.")
2247 |     reports_parser.add_argument("--limit", type=int, default=20)
2248 |
2249 |     runs_parser = subparsers.add_parser("runs", help="List existing ML training runs.")
2250 |     runs_parser.add_argument("--limit", type=int, default=10)
2251 |
2252 |     cache_parser = subparsers.add_parser("cache", help="List cached command artifacts.")
2253 |     cache_parser.add_argument("--limit", type=int, default=40)
2254 |     return parser

```

Function: `normalize_global_args`

Lines: 2257 to 2264

Description: Analyzes and executes `normalize_global_args` logic.

```

2257 | def normalize_global_args(argv: list[str] | None) -> list[str] | None:
2258 |     if argv is None:
2259 |         return None
2260 |     normalized = list(argv)
2261 |     if "--json" in normalized:
2262 |         normalized = [value for value in normalized if value != "--json"]
2263 |         normalized.insert(0, "--json")
2264 |     return normalized

```

Function: `main`

Lines: 2267 to 2346

Description: Analyzes and executes `main` logic.

```

2267 | def main(argv: list[str] | None = None) -> int:
2268 |     ensure_product_dirs()
2269 |     parser = build_parser()
2270 |     args = parser.parse_args(normalize_global_args(argv))
2271 |
2272 |     try:
2273 |         if args.command is None or args.command == "shell":
2274 |             command_shell()
2275 |         elif args.command == "gui":
2276 |             from .product_gui import main as gui_main
2277 |
2278 |             return gui_main([])
2279 |         elif args.command == "status":
2280 |             show_status(args.json)
2281 |         elif args.command == "traffic":
2282 |             show_traffic(args.json)
2283 |         elif args.command == "attacks":
2284 |             show_attacks(args.json)
2285 |         elif args.command == "datasets":
2286 |             show_dataset_catalog(args.json)
2287 |         elif args.command == "malware":
2288 |             show_malware(args.json, args.limit)
2289 |         elif args.command == "learn":
2290 |             model = learn_model(
2291 |                 limit=20000 if args.quick else None,
2292 |                 include_generated=not args.skip_generated,
2293 |                 include_test=args.include_test,
2294 |             )
2295 |             show_learn(model, args.json)
2296 |         elif args.command == "scan":
2297 |             source = resolve_readable_path(args.path, default=TEST_CSV)
2298 |             summary = analyze_csv(source, limit=None if args.all else args.limit, export=not args.no_export)
2299 |             show_scan(summary, args.json)
2300 |         elif args.command == "export":
2301 |             source = resolve_readable_path(args.path, default=TEST_CSV)
2302 |             summary = analyze_csv(source, limit=args.limit, export=True)
2303 |             show_scan(summary, args.json)
2304 |         elif args.command == "import":
2305 |             show_import(import_csv(resolve_readable_path(args.path), args.name), args.json)
2306 |         elif args.command == "download":
2307 |             downloaded = download_url(args.url, args.name)
2308 |             payload = {"downloaded": relative_path(downloaded)}
2309 |             print_json(payload) if args.json else print(f"Downloaded: {payload['downloaded']}")
2310 |         elif args.command == "index":
2311 |             show_index(resolve_readable_path(args.path, default=TEST_CSV), args.json, limit=None if args.all else args.limit)
2312 |         elif args.command == "hunt":
2313 |             show_hunt(args.pattern, resolve_readable_path(args.path) if args.path else None, args.json, args.limit)
2314 |         elif args.command == "ioc":
2315 |             show_ioc(args.ioc_args, args.json)
2316 |         elif args.command == "netstat":
2317 |             show_netstat(args.json, only_listening=args.listening, limit=args.limit)
2318 |         elif args.command == "ports":
2319 |             show_netstat(args.json, only_listening=True, limit=args.limit)
2320 |         elif args.command == "port":
2321 |             show_port(args.number, args.json)
2322 |         elif args.command == "probe":
2323 |             show_probe(args.host, args.ports, args.json)
2324 |         elif args.command == "dns":
2325 |             show_dns(args.host, args.json)
2326 |         elif args.command == "ps":
2327 |             show_processes(args.json, args.limit)
2328 |         elif args.command == "hash":
2329 |             show_hash(resolve_readable_path(args.path), args.json)
2330 |         elif args.command == "filescan":
2331 |             show_file_scan(resolve_readable_path(args.path), args.json)
2332 |         elif args.command == "reports":
2333 |             show_reports(args.json, args.limit)
2334 |         elif args.command == "runs":

```

```

2335 |         show_runs(args.json, args.limit)
2336 |     elif args.command == "cache":
2337 |         show_cache(args.json, args.limit)
2338 |     else:
2339 |         parser.error(f"Unknown command: {args.command}")
2340 | except KeyboardInterrupt:
2341 |     print("\nInterrupted.")
2342 |     return 130
2343 | except Exception as exc:
2344 |     print(f"error: {exc}", file=sys.stderr)
2345 |     return 1
2346 |     return 0

```

Function: update

Lines: 440 to 447

Description: Analyzes and executes update logic.

```

0440 |     def update(self, value: float) -> None:
0441 |         self.count += 1
0442 |         delta = value - self.mean
0443 |         self.mean += delta / self.count
0444 |         delta2 = value - self.mean
0445 |         self.m2 += delta * delta2
0446 |         self.min_value = min(self.min_value, value)
0447 |         self.max_value = max(self.max_value, value)

```

Function: to_json

Lines: 449 to 457

Description: Analyzes and executes to_json logic.

```

0449 |     def to_json(self) -> dict[str, float | int]:
0450 |         variance = self.m2 / max(self.count - 1, 1)
0451 |         return {
0452 |             "count": self.count,
0453 |             "mean": round(self.mean, 8),
0454 |             "variance": round(max(variance, 1e-9), 8),
0455 |             "min": round(self.min_value if self.count else 0.0, 8),
0456 |             "max": round(self.max_value if self.count else 0.0, 8),
0457 |         }

```

Module: ./ids_app/storage.py

Overview

Total Lines: 50

Function: ensure_directories

Lines: 10 to 12

Description: Analyzes and executes ensure_directories logic.

```

0010 | def ensure_directories() -> None:
0011 |     for path in (AUTOMATION_DIR, JOBS_DIR, RUNS_DIR, LEGACY_DIR):
0012 |         path.mkdir(parents=True, exist_ok=True)

```

Function: read_json

Lines: 15 to 19

Description: Analyzes and executes read_json logic.

```
0015 | def read_json(path: Path, default: Any = None) -> Any:
0016 |     if not path.exists():
0017 |         return default
0018 |     with path.open("r", encoding="utf-8") as handle:
0019 |         return json.load(handle)
```

Function: write_json

Lines: 22 to 25

Description: Analyzes and executes write_json logic.

```
0022 | def write_json(path: Path, payload: Any) -> None:
0023 |     path.parent.mkdir(parents=True, exist_ok=True)
0024 |     with path.open("w", encoding="utf-8") as handle:
0025 |         json.dump(payload, handle, indent=2, sort_keys=False)
```

Function: job_path

Lines: 28 to 29

Description: Analyzes and executes job_path logic.

```
0028 | def job_path(job_id: str) -> Path:
0029 |     return JOBS_DIR / f"{job_id}.json"
```

Function: run_dir

Lines: 32 to 33

Description: Analyzes and executes run_dir logic.

```
0032 | def run_dir(run_id: str) -> Path:
0033 |     return RUNS_DIR / run_id
```

Function: run_summary_path

Lines: 36 to 37

Description: Analyzes and executes run_summary_path logic.

```
0036 | def run_summary_path(run_id: str) -> Path:
0037 |     return run_dir(run_id) / "summary.json"
```

Function: list_run_summaries

Lines: 40 to 49

Description: Analyzes and executes list_run_summaries logic.

```

0040 | def list_run_summaries(limit: int = 20) -> list[dict[str, Any]]:
0041 |     ensure_directories()
0042 |     summaries: list[dict[str, Any]] = []
0043 |     for path in sorted(RUNS_DIR.glob("*/summary.json"), key=lambda item: item.stat().st_mtime, reverse=True):
0044 |         payload = read_json(path)
0045 |         if payload:
0046 |             summaries.append(payload)
0047 |         if len(summaries) >= limit:
0048 |             break
0049 |     return summaries

```

Module: `./ids_app/terminal.py`

Overview

Total Lines: 646

Function: `_format_number`

Lines: 88 to 93

Description: Analyzes and executes `_format_number` logic.

```

0088 | def _format_number(value: Any) -> str:
0089 |     if isinstance(value, float):
0090 |         return f"{value:.4f}"
0091 |     if isinstance(value, int):
0092 |         return f"{value:,}"
0093 |     return str(value)

```

Function: `_percent`

Lines: 96 to 97

Description: Analyzes and executes `_percent` logic.

```

0096 | def _percent(value: float | int) -> str:
0097 |     return f"{float(value) * 100:.2f}%"

```

Function: `_table`

Lines: 100 to 109

Description: Analyzes and executes `_table` logic.

```

0100 | def _table(headers: list[str], rows: list[list[Any]]) -> str:
0101 |     text_rows = [_format_number(cell) for cell in row] for row in rows]
0102 |     widths = [
0103 |         max(len(header), *(len(row[index]) for row in text_rows)) if text_rows else len(header)
0104 |         for index, header in enumerate(headers)
0105 |     ]
0106 |     line = " ".join(header.ljust(widths[index]) for index, header in enumerate(headers))
0107 |     rule = " ".join("-" * width for width in widths)
0108 |     body = [" ".join(row[index].ljust(widths[index]) for index in range(len(headers))) for row in text_rows]
0109 |     return "\n".join([line, rule, *body])

```

Function: `_section`

Lines: 112 to 114

Description: Analyzes and executes `_section` logic.

```
0112 | def _section(title: str) -> None:
0113 |     print(f"\n{title}")
0114 |     print("=" * len(title))
```

Function: `_print_json`

Lines: 117 to 118

Description: Analyzes and executes `_print_json` logic.

```
0117 | def _print_json(payload: Any) -> None:
0118 |     print(json.dumps(payload, indent=2, sort_keys=False))
```

Function: `show_datasets`

Lines: 121 to 136

Description: Analyzes and executes `show_datasets` logic.

```
0121 | def show_datasets(json_output: bool = False) -> None:
0122 |     payload = dataset_summary()
0123 |     if json_output:
0124 |         _print_json(payload)
0125 |         return
0126 |
0127 |     _section("Datasets")
0128 |     rows = []
0129 |     for key in ("classical_train", "classical_test", "dnn_train", "dnn_test"):
0130 |         item = payload[key]
0131 |         labels = ", ".join(f"{label}:{count}," for label, count in item["label_counts"].items())
0132 |         rows.append([item["path"], item["rows"], item["columns"], f"{item['size_mb']} MB", labels])
0133 |     print(_table(["Path", "Rows", "Cols", "Size", "Labels"], rows))
0134 |     print()
0135 |     print(f"Train files match: {payload['duplicates']['train_files_match']}")
0136 |     print(f"Test files match: {payload['duplicates']['test_files_match']}")
```

Function: `show_attacks`

Lines: 139 to 169

Description: Analyzes and executes `show_attacks` logic.

```

0139 | def show_attacks(json_output: bool = False) -> None:
0140 |     payload = dataset_summary()
0141 |     rows = []
0142 |     for key in ("classical_train", "classical_test"):
0143 |         item = payload[key]
0144 |         total = item["rows"]
0145 |         for label, count in sorted(item["label_counts"].items()):
0146 |             rows.append(
0147 |                 [
0148 |                     item["path"],
0149 |                     label,
0150 |                     BINARY_LABELS.get(label, "unknown"),
0151 |                     count,
0152 |                     f"{{(count / total) * 100:.2f}}%",
0153 |                 ]
0154 |             )
0155 |
0156 |     if json_output:
0157 |         _print_json(
0158 |             {
0159 |                 "label_meaning": BINARY_LABELS,
0160 |                 "note": "This dataset is binary encoded. Named attack families are not present in the CSV files.",
0161 |                 "rows": rows,
0162 |             }
0163 |         )
0164 |     return
0165 |
0166 |     _section("Attack Label Distribution")
0167 |     print(_table(["Dataset", "Label", "Meaning", "Rows", "Share"], rows))
0168 |     print()
0169 |     print("The CSV files only contain binary labels here. The app treats 0 as normal and 1 as attack.")

```

Function: show_features

Lines: 172 to 179

Description: Analyzes and executes show_features logic.

```

0172 | def show_features(json_output: bool = False) -> None:
0173 |     rows = [[0, "label", "binary target: 0=normal, 1=attack"]]
0174 |     rows.extend([[index, name, "numeric/coded IDS feature"] for index, name in enumerate(FEATURE_NAMES, start=1)])
0175 |     if json_output:
0176 |         _print_json({"columns": rows})
0177 |         return
0178 |     _section("Feature Columns")
0179 |     print(_table(["Column", "Name", "Meaning"], rows))

```

Function: show_legacy

Lines: 182 to 213

Description: Analyzes and executes show_legacy logic.

```

0182 | def show_legacy(json_output: bool = False) -> None:
0183 |     payload = evaluate_legacy_predictions()
0184 |     if json_output:
0185 |         _print_json(payload)
0186 |         return
0187 |
0188 |     _section("Legacy Classical Results")
0189 |     classical_rows = [
0190 |         [
0191 |             item["label"],
0192 |             _percent(item["metrics"]["accuracy"]),
0193 |             _percent(item["metrics"]["precision"]),
0194 |             _percent(item["metrics"]["recall"]),
0195 |             _percent(item["metrics"]["f1"]),
0196 |         ]
0197 |         for item in payload["classical"]
0198 |     ]
0199 |     print(_table(["Model", "Accuracy", "Precision", "Recall", "F1"], classical_rows))
0200 |
0201 |     _section("Legacy DNN Results")
0202 |     dnn_rows = [
0203 |         [
0204 |             item["label"],
0205 |             _percent(item["metrics"]["accuracy"]),
0206 |             _percent(item["metrics"]["precision"]),
0207 |             _percent(item["metrics"]["recall"]),
0208 |             _percent(item["metrics"]["f1"]),
0209 |             item["history"]["epochs_logged"] if item.get("history") else "n/a",
0210 |         ]
0211 |         for item in payload["dnn"]
0212 |     ]
0213 |     print(_table(["Model", "Accuracy", "Precision", "Recall", "F1", "Epochs"], dnn_rows))

```

Function: show_runs

Lines: 216 to 242

Description: Analyzes and executes show_runs logic.

```

0216 | def show_runs(json_output: bool = False, limit: int = 20) -> None:
0217 |     payload = list_run_summaries(limit=limit)
0218 |     if json_output:
0219 |         _print_json(payload)
0220 |         return
0221 |
0222 |     _section("Completed Runs")
0223 |     if not payload:
0224 |         print("No completed runs yet.")
0225 |         return
0226 |
0227 |     rows = []
0228 |     for run in payload:
0229 |         best = run.get("results", [{}])[0]
0230 |         metrics = best.get("metrics", {})
0231 |         rows.append(
0232 |             [
0233 |                 run.get("run_id"),
0234 |                 run.get("kind"),
0235 |                 best.get("label", "n/a"),
0236 |                 _percent(metrics.get("accuracy", 0)),
0237 |                 _percent(metrics.get("f1", 0)),
0238 |                 run.get("dataset", {}).get("train_rows", 0),
0239 |                 run.get("dataset", {}).get("test_rows", 0),
0240 |             ]
0241 |         )
0242 |     print(_table(["Run", "Kind", "Best Model", "Accuracy", "F1", "Train", "Test"], rows))

```

Function: show_reports

Lines: 245 to 266

Description: Analyzes and executes show_reports logic.

```

0245 | def show_reports(json_output: bool = False) -> None:
0246 |     reports = []
0247 |     if REPORTS_DIR.exists():
0248 |         for path in sorted(REPORTS_DIR.glob("*")):
0249 |             if path.is_file():
0250 |                 reports.append(
0251 |                     {
0252 |                         "name": path.name,
0253 |                         "path": str(path.relative_to(ROOT_DIR)),
0254 |                         "size_mb": round(path.stat().st_size / (1024 * 1024), 2),
0255 |                     }
0256 |                 )
0257 |
0258 |     if json_output:
0259 |         _print_json(reports)
0260 |         return
0261 |
0262 |     _section("Reports")
0263 |     if not reports:
0264 |         print("No report files found.")
0265 |         return
0266 |     print(_table(["Name", "Path", "Size"], [[item["name"], item["path"], f"{item['size_mb']} MB"] for item in reports]))

```

Function: show_best

Lines: 269 to 299

Description: Analyzes and executes show_best logic.

```

0269 | def show_best(json_output: bool = False) -> None:
0270 |     legacy = evaluate_legacy_predictions()
0271 |     runs = list_run_summaries(limit=50)
0272 |     best_legacy_classical = legacy["classical"][0] if legacy["classical"] else None
0273 |     best_legacy_dnn = legacy["dnn"][0] if legacy["dnn"] else None
0274 |     best_run = None
0275 |     for run in runs:
0276 |         if not run.get("results"):
0277 |             continue
0278 |         candidate = {"run_id": run["run_id"], "kind": run["kind"], **run["results"][0]}
0279 |         if best_run is None or candidate["metrics"]["f1"] > best_run["metrics"]["f1"]:
0280 |             best_run = candidate
0281 |
0282 |     payload = {
0283 |         "legacy_classical": best_legacy_classical,
0284 |         "legacy_dnn": best_legacy_dnn,
0285 |         "automation_run": best_run,
0286 |     }
0287 |     if json_output:
0288 |         _print_json(payload)
0289 |         return
0290 |
0291 |     _section("Best Known Results")
0292 |     rows = []
0293 |     if best_legacy_classical:
0294 |         rows.append(["Legacy Classical", best_legacy_classical["label"], _percent(best_legacy_classical["metrics"])]
0295 |     if best_legacy_dnn:
0296 |         rows.append(["Legacy DNN", best_legacy_dnn["label"], _percent(best_legacy_dnn["metrics"])]
0297 |     if best_run:
0298 |         rows.append([f"Run {best_run['run_id']}", best_run["label"], _percent(best_run["metrics"])]
0299 |     print(_table(["Source", "Model", "Accuracy", "F1"], rows))

```

Function: show_head

Lines: 302 to 315

Description: Analyzes and executes show_head logic.

```

0302 | def show_head(path_text: str, lines: int = 5) -> None:
0303 |     path = _resolve_repo_path(path_text)
0304 |     if not path.exists() or not path.is_file():
0305 |         print(f"Not a file: {path_text}")
0306 |         return
0307 |     if path.stat().st_size > 100 * 1024 * 1024:
0308 |         print("Refusing to print a very large file. Use a smaller file or a sampled CSV command.")
0309 |         return
0310 |     _section(f"Head: {path.relative_to(ROOT_DIR)}")
0311 |     with path.open("r", encoding="utf-8", errors="ignore", newline="") as handle:
0312 |         for index, line in enumerate(handle):
0313 |             if index >= lines:
0314 |                 break
0315 |             print(line.rstrip())

```

Function: show_overview

Lines: 318 to 332

Description: Analyzes and executes show_overview logic.

```

0318 | def show_overview(json_output: bool = False) -> None:
0319 |     payload = {
0320 |         "datasets": dataset_summary(),
0321 |         "legacy": evaluate_legacy_predictions(),
0322 |         "runs": list_run_summaries(limit=8),
0323 |         "profiles": {"classical": CLASSICAL_PROFILES, "dnn": DNN_PROFILES},
0324 |     }
0325 |     if json_output:
0326 |         _print_json(payload)
0327 |         return
0328 |     show_datasets()
0329 |     show_attacks()
0330 |     show_legacy()
0331 |     show_runs(limit=8)
0332 |     show_reports()

```

Function: train_classical

Lines: 335 to 346

Description: Analyzes and executes train_classical logic.

```

0335 | def train_classical(args: argparse.Namespace) -> None:
0336 |     config: dict[str, Any] = {"profile": args.profile, "random_state": args.random_state}
0337 |     if args.train_sample is not None:
0338 |         config["train_sample"] = args.train_sample
0339 |     if args.test_sample is not None:
0340 |         config["test_sample"] = args.test_sample
0341 |     if args.models:
0342 |         config["models"] = args.models
0343 |
0344 |     _section("Running Classical Train/Test")
0345 |     summary = train_classical_suite("terminal", config)
0346 |     _print_train_summary(summary)

```

Function: train_dnn

Lines: 349 to 366

Description: Analyzes and executes train_dnn logic.

```

0349 | def train_dnn(args: argparse.Namespace) -> None:
0350 |     from .dnn import train_dnn_suite
0351 |
0352 |     config: dict[str, Any] = {"profile": args.profile, "random_state": args.random_state}
0353 |     if args.architectures:
0354 |         config["architectures"] = args.architectures
0355 |     if args.epochs is not None:
0356 |         config["epochs"] = args.epochs
0357 |     if args.batch_size is not None:
0358 |         config["batch_size"] = args.batch_size
0359 |     if args.train_sample is not None:
0360 |         config["train_sample"] = args.train_sample
0361 |     if args.test_sample is not None:
0362 |         config["test_sample"] = args.test_sample
0363 |
0364 |     _section("Running DNN Train/Test")
0365 |     summary = train_dnn_suite("terminal", config)
0366 |     _print_train_summary(summary)

```

Function: auto_train

Lines: 369 to 385

Description: Analyzes and executes auto_train logic.

```
0369 | def auto_train(args: argparse.Namespace) -> None:
0370 |     profile_name = args.auto_profile
0371 |     config = AUTO_TRAIN_PROFILES[profile_name]
0372 |     _section(f"Auto Train: {profile_name}")
0373 |     print("Running classical suite...")
0374 |     classical_summary = train_classical_suite("terminal-auto", dict(config["classical"]))
0375 |     _print_train_summary(classical_summary)
0376 |
0377 |     if args.skip_dnn:
0378 |         return
0379 |
0380 |     print()
0381 |     print("Running DNN suite...")
0382 |     from .dnn import train_dnn_suite
0383 |
0384 |     dnn_summary = train_dnn_suite("terminal-auto", dict(config["dnn"]))
0385 |     _print_train_summary(dnn_summary)
```

Function: _print_train_summary

Lines: 388 to 407

Description: Analyzes and executes _print_train_summary logic.

```
0388 | def _print_train_summary(summary: dict[str, Any]) -> None:
0389 |     print(f"Run ID: {summary['run_id']}")
0390 |     print(f"Train rows: {summary['dataset']['train_rows']:,}")
0391 |     print(f"Test rows: {summary['dataset']['test_rows']:,}")
0392 |     rows = []
0393 |     for item in summary["results"]:
0394 |         rows.append(
0395 |             [
0396 |                 item["label"],
0397 |                 f"{item['training_seconds']}s",
0398 |                 _percent(item["metrics"]["accuracy"]),
0399 |                 _percent(item["metrics"]["precision"]),
0400 |                 _percent(item["metrics"]["recall"]),
0401 |                 _percent(item["metrics"]["f1"]),
0402 |             ]
0403 |         )
0404 |     print()
0405 |     print(_table(["Model", "Time", "Accuracy", "Precision", "Recall", "F1"], rows))
0406 |     print()
0407 |     print(f"Saved summary: automation/runs/{summary['run_id']}/summary.json")
```

Function: command_shell

Lines: 410 to 427

Description: Analyzes and executes command_shell logic.

```
0410 | def command_shell() -> None:
0411 |     ensure_directories()
0412 |     print("IDS command console. Type 'help' for commands, 'exit' to quit.")
0413 |     while True:
0414 |         try:
0415 |             raw = input("ids> ").strip()
0416 |         except EOFError:
0417 |             print()
0418 |             return
0419 |         if not raw:
0420 |             continue
0421 |         try:
0422 |             should_exit = run_shell_command(raw)
0423 |         except Exception as exc:
0424 |             print(f"error: {exc}")
0425 |             continue
0426 |         if should_exit:
0427 |             return
```

Function: run_shell_command

Lines: 430 to 472

Description: Analyzes and executes run_shell_command logic.

```

0430 | def run_shell_command(raw: str) -> bool:
0431 |     parts = shlex.split(raw)
0432 |     if not parts:
0433 |         return False
0434 |     command, *args = parts
0435 |     command = command.lower()
0436 |
0437 |     if command in {"exit", "quit", "q"}:
0438 |         return True
0439 |     if command == "help":
0440 |         print_shell_help()
0441 |     elif command == "clear":
0442 |         os.system("clear")
0443 |     elif command == "pwd":
0444 |         print(ROOT_DIR)
0445 |     elif command == "ls":
0446 |         list_path(args[0] if args else ".")
0447 |     elif command == "head":
0448 |         if not args:
0449 |             print("usage: head <path> [lines]")
0450 |         else:
0451 |             show_head(args[0], int(args[1]) if len(args) > 1 else 5)
0452 |     elif command in {"overview", "status"}:
0453 |         show_overview()
0454 |     elif command in {"data", "datasets"}:
0455 |         show_datasets()
0456 |     elif command in {"attack", "attacks"}:
0457 |         show_attacks()
0458 |     elif command in {"feature", "features", "schema"}:
0459 |         show_features()
0460 |     elif command == "legacy":
0461 |         show_legacy()
0462 |     elif command == "reports":
0463 |         show_reports()
0464 |     elif command == "runs":
0465 |         show_runs(limit=int(args[0]) if args else 20)
0466 |     elif command == "best":
0467 |         show_best()
0468 |     elif command == "train":
0469 |         run_train_command(args)
0470 |     else:
0471 |         print(f"Unknown command: {command}. Type 'help'.")
0472 |     return False

```

Function: print_shell_help

Lines: 475 to 495

Description: Analyzes and executes print_shell_help logic.

```
0475 | def print_shell_help() -> None:
0476 |     _section("Commands")
0477 |     rows = [
0478 |         ["help", "show this command list"],
0479 |         ["overview | status", "show datasets, attacks, legacy metrics, runs, reports"],
0480 |         ["datasets | data", "show CSV sizes, labels, duplicate checks"],
0481 |         ["attacks", "show attack/normal label distribution"],
0482 |         ["features | schema", "show the 42 dataset columns"],
0483 |         ["legacy", "show saved legacy prediction metrics"],
0484 |         ["runs [limit]", "show completed train/test runs"],
0485 |         ["best", "show best legacy and automation results"],
0486 |         ["reports", "list report PDFs"],
0487 |         ["ls [path]", "list repo files"],
0488 |         ["head <path> [lines]", "print first lines of a text/CSV file"],
0489 |         ["train auto [quick|standard|full]", "train classical and DNN automatically"],
0490 |         ["train classical [quick|fast|balanced|full]", "run classical train/test"],
0491 |         ["train dnn [quick|fast|balanced|full]", "run DNN train/test"],
0492 |         ["clear", "clear terminal"],
0493 |         ["exit", "quit"],
0494 |     ]
0495 |     print(_table(["Command", "Action"], rows))
```

Function: run_train_command

Lines: 498 to 536

Description: Analyzes and executes run_train_command logic.

```

0498 | def run_train_command(args: list[str]) -> None:
0499 |     if not args:
0500 |         print("usage: train <auto|classical|dnn> [quick|fast|standard|balanced|full]")
0501 |         return
0502 |     kind = args[0].lower()
0503 |     profile = args[1].lower() if len(args) > 1 else "quick"
0504 |
0505 |     if kind == "auto":
0506 |         if profile == "fast":
0507 |             profile = "quick"
0508 |         if profile not in AUTO_TRAIN_PROFILES:
0509 |             print("profiles: quick, standard, full")
0510 |             return
0511 |         namespace = argparse.Namespace(auto_profile=profile, skip_dnn=False)
0512 |         auto_train(namespace)
0513 |     elif kind == "classical":
0514 |         if profile == "quick":
0515 |             namespace = argparse.Namespace(profile="fast", train_sample=5000, test_sample=2000, models=None, random=None)
0516 |         else:
0517 |             if profile == "standard":
0518 |                 profile = "fast"
0519 |             if profile not in CLASSICAL_PROFILES:
0520 |                 print("profiles: quick, fast, balanced, full")
0521 |                 return
0522 |             namespace = argparse.Namespace(profile=profile, train_sample=None, test_sample=None, models=None, random=None)
0523 |             train_classical(namespace)
0524 |     elif kind == "dnn":
0525 |         if profile == "quick":
0526 |             namespace = argparse.Namespace(profile="fast", architectures=[1], epochs=1, batch_size=128, train_sample=None, test_sample=None, models=None, random=None)
0527 |         else:
0528 |             if profile == "standard":
0529 |                 profile = "fast"
0530 |             if profile not in DNN_PROFILES:
0531 |                 print("profiles: quick, fast, balanced, full")
0532 |                 return
0533 |             namespace = argparse.Namespace(profile=profile, architectures=None, epochs=None, batch_size=None, train_sample=None, test_sample=None, models=None, random=None)
0534 |             train_dnn(namespace)
0535 |     else:
0536 |         print("train kinds: auto, classical, dnn")

```

Function: list_path

Lines: 539 to 551

Description: Analyzes and executes list_path logic.

```

0539 | def list_path(path_text: str) -> None:
0540 |     path = _resolve_repo_path(path_text)
0541 |     if not path.exists():
0542 |         print(f"Not found: {path_text}")
0543 |         return
0544 |     if path.is_file():
0545 |         print(path.relative_to(ROOT_DIR))
0546 |         return
0547 |     entries = []
0548 |     for child in sorted(path.iterdir(), key=lambda item: (item.is_file(), item.name.lower())):
0549 |         marker = "/" if child.is_dir() else ""
0550 |         entries.append([child.name + marker, f"{child.stat().st_size:,"} if child.is_file() else "dir"])
0551 |     print(_table(["Name", "Size"], entries))

```

Function: _resolve_repo_path

Lines: 554 to 563

Description: Analyzes and executes `_resolve_repo_path` logic.

```
0554 | def _resolve_repo_path(path_text: str) -> Path:
0555 |     path = Path(path_text)
0556 |     if not path.is_absolute():
0557 |         path = ROOT_DIR / path
0558 |     resolved = path.resolve()
0559 |     try:
0560 |         resolved.relative_to(ROOT_DIR)
0561 |     except ValueError:
0562 |         raise ValueError("path must stay inside the project directory")
0563 |     return resolved
```

Function: `build_parser`

Lines: 566 to 602

Description: Analyzes and executes `build_parser` logic.

```
0566 | def build_parser() -> argparse.ArgumentParser:
0567 |     parser = argparse.ArgumentParser(description="Terminal-only IDS automation console.")
0568 |     parser.add_argument("--json", action="store_true", help="Print JSON for commands that support it.")
0569 |     subparsers = parser.add_subparsers(dest="command")
0570 |
0571 |     subparsers.add_parser("overview", help="Show datasets, legacy metrics, runs, and reports.")
0572 |     subparsers.add_parser("datasets", help="Show CSV file sizes, labels, hashes, and duplicate checks.")
0573 |     subparsers.add_parser("attacks", help="Show attack/normal label distribution.")
0574 |     subparsers.add_parser("features", help="Show target and feature columns.")
0575 |     subparsers.add_parser("legacy", help="Show metrics for existing saved prediction files.")
0576 |     subparsers.add_parser("best", help="Show best known legacy and generated results.")
0577 |     subparsers.add_parser("shell", help="Open the interactive IDS command shell.")
0578 |     runs_parser = subparsers.add_parser("runs", help="Show completed automation runs.")
0579 |     runs_parser.add_argument("--limit", type=int, default=20)
0580 |     subparsers.add_parser("reports", help="List PDF reports available in the repo.")
0581 |
0582 |     auto_parser = subparsers.add_parser("auto-train", help="Train classical and DNN suites automatically.")
0583 |     auto_parser.add_argument("--auto-profile", choices=AUTO_TRAIN_PROFILES.keys(), default="quick")
0584 |     auto_parser.add_argument("--skip-dnn", action="store_true")
0585 |
0586 |     classical_parser = subparsers.add_parser("train-classical", help="Run a classical ML train/test suite.")
0587 |     classical_parser.add_argument("--profile", choices=CLASSICAL_PROFILES.keys(), default="fast")
0588 |     classical_parser.add_argument("--train-sample", type=int)
0589 |     classical_parser.add_argument("--test-sample", type=int)
0590 |     classical_parser.add_argument("--models", nargs="+")
0591 |     classical_parser.add_argument("--random-state", type=int, default=42)
0592 |
0593 |     dnn_parser = subparsers.add_parser("train-dnn", help="Run a DNN train/test suite.")
0594 |     dnn_parser.add_argument("--profile", choices=DNN_PROFILES.keys(), default="fast")
0595 |     dnn_parser.add_argument("--architectures", nargs="+", type=int)
0596 |     dnn_parser.add_argument("--epochs", type=int)
0597 |     dnn_parser.add_argument("--batch-size", type=int)
0598 |     dnn_parser.add_argument("--train-sample", type=int)
0599 |     dnn_parser.add_argument("--test-sample", type=int)
0600 |     dnn_parser.add_argument("--random-state", type=int, default=42)
0601 |
0602 |     return parser
```

Function: `main`

Lines: 605 to 642

Description: Analyzes and executes `main` logic.

```

0605 | def main(argv: list[str] | None = None) -> int:
0606 |     ensure_directories()
0607 |     parser = build_parser()
0608 |     args = parser.parse_args(argv)
0609 |
0610 |     try:
0611 |         if args.command is None:
0612 |             command_shell()
0613 |         elif args.command == "shell":
0614 |             command_shell()
0615 |         elif args.command == "overview":
0616 |             show_overview(args.json)
0617 |         elif args.command == "datasets":
0618 |             show_datasets(args.json)
0619 |         elif args.command == "attacks":
0620 |             show_attacks(args.json)
0621 |         elif args.command == "features":
0622 |             show_features(args.json)
0623 |         elif args.command == "legacy":
0624 |             show_legacy(args.json)
0625 |         elif args.command == "best":
0626 |             show_best(args.json)
0627 |         elif args.command == "runs":
0628 |             show_runs(args.json, args.limit)
0629 |         elif args.command == "reports":
0630 |             show_reports(args.json)
0631 |         elif args.command == "auto-train":
0632 |             auto_train(args)
0633 |         elif args.command == "train-classical":
0634 |             train_classical(args)
0635 |         elif args.command == "train-dnn":
0636 |             train_dnn(args)
0637 |         else:
0638 |             parser.error(f"Unknown command: {args.command}")
0639 |     except KeyboardInterrupt:
0640 |         print("\nInterrupted.")
0641 |         return 130
0642 |     return 0

```

Module: ./research_report/build_report.py

Overview

Total Lines: 1324

Class: SourceNote

Lines: 62 to 69

Description: Analyzes and executes SourceNote logic.

```

0062 | class SourceNote:
0063 |     source_id: str
0064 |     title: str
0065 |     authors: str
0066 |     year: str
0067 |     url: str
0068 |     access_note: str
0069 |     relevance: str

```

Class: FunctionInfo

Lines: 73 to 77

Description: Analyzes and executes FunctionInfo logic.

```
0073 | class FunctionInfo:
0074 |     name: str
0075 |     start: int
0076 |     end: int
0077 |     kind: str
```

Class: ModuleInfo

Lines: 81 to 87

Description: Analyzes and executes ModuleInfo logic.

```
0081 | class ModuleInfo:
0082 |     path: str
0083 |     line_count: int
0084 |     function_count: int
0085 |     class_count: int
0086 |     functions: list[FunctionInfo]
0087 |     classes: list[FunctionInfo]
```

Class: ListingSpec

Lines: 91 to 96

Description: Analyzes and executes ListingSpec logic.

```
0091 | class ListingSpec:
0092 |     path: str
0093 |     start: int
0094 |     end: int
0095 |     title: str
0096 |     explanation: str
```

Class: NumberedCanvas

Lines: 99 to 125

Description: Analyzes and executes NumberedCanvas logic.

```

0099 | class NumberedCanvas(canvas.Canvas):
0100 |     last_page_count = 0
0101 |
0102 |     def __init__(self, *args: Any, **kwargs: Any) -> None:
0103 |         super().__init__(*args, **kwargs)
0104 |         self._saved_page_states: list[dict[str, Any]] = []
0105 |
0106 |     def showPage(self) -> None:
0107 |         self._saved_page_states.append(dict(self.__dict__))
0108 |         self._startPage()
0109 |
0110 |     def save(self) -> None:
0111 |         page_count = len(self._saved_page_states)
0112 |         type(self).last_page_count = page_count
0113 |         for state in self._saved_page_states:
0114 |             self.__dict__.update(state)
0115 |             self.draw_footer(page_count)
0116 |             super().showPage()
0117 |         super().save()
0118 |
0119 |     def draw_footer(self, page_count: int) -> None:
0120 |         self.setStrokeColor(colors.HexColor("#D9D9D9"))
0121 |         self.line(0.75 * inch, 0.62 * inch, 7.75 * inch, 0.62 * inch)
0122 |         self.setFont(BODY_FONT, 8.5)
0123 |         self.setFillColor(MUTED)
0124 |         self.drawString(0.78 * inch, 0.4 * inch, "IDS Sentinel Terminal Research Report")
0125 |         self.drawRightString(7.72 * inch, 0.4 * inch, f"Page {self._pageNumber} of {page_count}")

```

Function: ensure_dirs

Lines: 128 to 131

Description: Analyzes and executes ensure_dirs logic.

```

0128 | def ensure_dirs() -> None:
0129 |     OUT_DIR.mkdir(parents=True, exist_ok=True)
0130 |     ASSET_DIR.mkdir(parents=True, exist_ok=True)
0131 |     RAW_ASSET_DIR.mkdir(parents=True, exist_ok=True)

```

Function: styles

Lines: 134 to 203

Description: Analyzes and executes styles logic.

```

0134 | def styles() -> dict[str, ParagraphStyle]:
0135 |     sample = getSampleStyleSheet()
0136 |     sample["Title"].fontName = BODY_BOLD
0137 |     sample["Title"].fontSize = 24
0138 |     sample["Title"].leading = 30
0139 |     sample["Title"].textColor = ACCENT
0140 |     sample["Title"].spaceAfter = 16
0141 |     sample["Heading1"].fontName = BODY_BOLD
0142 |     sample["Heading1"].fontSize = 17
0143 |     sample["Heading1"].leading = 22
0144 |     sample["Heading1"].spaceBefore = 10
0145 |     sample["Heading1"].spaceAfter = 10
0146 |     sample["Heading1"].textColor = ACCENT
0147 |     sample["Heading2"].fontName = BODY_BOLD
0148 |     sample["Heading2"].fontSize = 13
0149 |     sample["Heading2"].leading = 17
0150 |     sample["Heading2"].spaceBefore = 8
0151 |     sample["Heading2"].spaceAfter = 6
0152 |     sample["Heading2"].textColor = colors.HexColor("#203864")
0153 |     sample["BodyText"].fontName = BODY_FONT
0154 |     sample["BodyText"].fontSize = 10.5
0155 |     sample["BodyText"].leading = 15
0156 |     sample["BodyText"].textColor = TEXT
0157 |     sample["BodyText"].spaceAfter = 8
0158 |     sample.add(
0159 |         ParagraphStyle(
0160 |             name="Small",
0161 |             parent=sample["BodyText"],
0162 |             fontSize=9,
0163 |             leading=12,
0164 |             textColor=MUTED,
0165 |             spaceAfter=6,
0166 |         )
0167 |     )
0168 |     sample.add(
0169 |         ParagraphStyle(
0170 |             name="Caption",
0171 |             parent=sample["BodyText"],
0172 |             fontSize=8.5,
0173 |             leading=11,
0174 |             textColor=MUTED,
0175 |             spaceBefore=4,
0176 |             spaceAfter=10,
0177 |         )
0178 |     )
0179 |     sample.add(
0180 |         ParagraphStyle(
0181 |             name="BulletBody",
0182 |             parent=sample["BodyText"],
0183 |             leftIndent=14,
0184 |             firstLineIndent=-8,
0185 |             spaceBefore=0,
0186 |             spaceAfter=4,
0187 |         )
0188 |     )
0189 |     sample.add(
0190 |         ParagraphStyle(
0191 |             name="CodeCommentary",
0192 |             parent=sample["BodyText"],
0193 |             fontSize=9.5,
0194 |             leading=13.5,
0195 |             textColor=TEXT,
0196 |             backColor=colors.HexColor("#FAFAFA"),
0197 |             borderPadding=6,
0198 |             borderColor=GRID,
0199 |             borderWidth=0.4,
0200 |             borderRadius=2,
0201 |         )

```

```
0202 |     )
0203 |     return sample
```

Function: paragraph

Lines: 206 to 207

Description: Analyzes and executes paragraph logic.

```
0206 | def paragraph(text: str, style: ParagraphStyle) -> Paragraph:
0207 |     return Paragraph(escape(text), style)
```

Function: bullet

Lines: 210 to 211

Description: Analyzes and executes bullet logic.

```
0210 | def bullet(text: str, style: ParagraphStyle) -> Paragraph:
0211 |     return Paragraph(escape(f"• {text}"), style)
```

Function: table_style

Lines: 214 to 229

Description: Analyzes and executes table_style logic.

```
0214 | def table_style(header_bg: colors.Color = ACCENT_LIGHT) -> TableStyle:
0215 |     return TableStyle(
0216 |         [
0217 |             ("FONTNAME", (0, 0), (-1, 0), BODY_BOLD),
0218 |             ("BACKGROUND", (0, 0), (-1, 0), header_bg),
0219 |             ("TEXTCOLOR", (0, 0), (-1, 0), colors.HexColor("#18324C")),
0220 |             ("GRID", (0, 0), (-1, -1), 0.35, GRID),
0221 |             ("VALIGN", (0, 0), (-1, -1), "TOP"),
0222 |             ("FONTSIZE", (0, 0), (-1, -1), 8.3),
0223 |             ("LEADING", (0, 0), (-1, -1), 10),
0224 |             ("LEFTPADDING", (0, 0), (-1, -1), 4),
0225 |             ("RIGHTPADDING", (0, 0), (-1, -1), 4),
0226 |             ("TOPPADDING", (0, 0), (-1, -1), 3),
0227 |             ("BOTTOMPADDING", (0, 0), (-1, -1), 3),
0228 |         ]
0229 |     )
```

Function: load_json_command

Lines: 232 to 242

Description: Analyzes and executes load_json_command logic.

```

0232 | def load_json_command(args: list[str]) -> Any:
0233 |     result = subprocess.run(
0234 |         ["python", "-m", "ids_app.product_app", *args, "--json"],
0235 |         cwd=ROOT,
0236 |         capture_output=True,
0237 |         text=True,
0238 |         check=False,
0239 |     )
0240 |     if result.returncode != 0:
0241 |         raise RuntimeError(result.stderr.strip() or "command failed")
0242 |     return json.loads(result.stdout)

```

Function: run_text_command

Lines: 245 to 253

Description: Analyzes and executes run_text_command logic.

```

0245 | def run_text_command(args: list[str]) -> str:
0246 |     result = subprocess.run(
0247 |         ["python", "-m", "ids_app.product_app", *args],
0248 |         cwd=ROOT,
0249 |         capture_output=True,
0250 |         text=True,
0251 |         check=False,
0252 |     )
0253 |     return (result.stdout + ("\n" + result.stderr if result.stderr else "")).strip()

```

Function: try_import

Lines: 256 to 260

Description: Analyzes and executes try_import logic.

```

0256 | def try_import(name: str) -> bool:
0257 |     with contextlib.suppress(Exception):
0258 |         __import__(name)
0259 |     return True
0260 |     return False

```

Function: repo_counts

Lines: 263 to 271

Description: Analyzes and executes repo_counts logic.

```

0263 | def repo_counts() -> dict[str, int]:
0264 |     files = [p for p in ROOT.rglob("*") if p.is_file()]
0265 |     return {
0266 |         "files": len(files),
0267 |         "py_files": sum(1 for p in files if p.suffix == ".py"),
0268 |         "csv_files": sum(1 for p in files if p.suffix.lower() == ".csv"),
0269 |         "json_files": sum(1 for p in files if p.suffix.lower() == ".json"),
0270 |         "md_files": sum(1 for p in files if p.suffix.lower() == ".md"),
0271 |     }

```

Function: parse_module

Lines: 274 to 291

Description: Analyzes and executes parse_module logic.

```
0274 | def parse_module(path: Path) -> ModuleInfo:
0275 |     text = path.read_text(encoding="utf-8", errors="ignore")
0276 |     tree = ast.parse(text)
0277 |     funcs: list[FunctionInfo] = []
0278 |     classes: list[FunctionInfo] = []
0279 |     for node in tree.body:
0280 |         if isinstance(node, ast.ClassDef):
0281 |             classes.append(FunctionInfo(node.name, node.lineno, node.end_lineno or node.lineno, "class"))
0282 |         elif isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
0283 |             funcs.append(FunctionInfo(node.name, node.lineno, node.end_lineno or node.lineno, "function"))
0284 |     return ModuleInfo(
0285 |         path=str(path.relative_to(ROOT)).replace("\\", "/"),
0286 |         line_count=len(text.splitlines()),
0287 |         function_count=len(funcs),
0288 |         class_count=len(classes),
0289 |         functions=funcs,
0290 |         classes=classes,
0291 |     )
```

Function: module_inventory

Lines: 294 to 296

Description: Analyzes and executes module_inventory logic.

```
0294 | def module_inventory() -> list[ModuleInfo]:
0295 |     targets = sorted((ROOT / "ids_app").glob("*.py")) + sorted((ROOT / "scripts").glob("*.py"))
0296 |     return [parse_module(path) for path in targets]
```

Function: resolve_font

Lines: 299 to 308

Description: Analyzes and executes resolve_font logic.

```
0299 | def resolve_font(size: int) -> ImageFont.ImageFont:
0300 |     candidates = [
0301 |         Path(os.environ.get("SystemRoot", "C:\\Windows")) / "Fonts" / "consola.ttf",
0302 |         Path(os.environ.get("SystemRoot", "C:\\Windows")) / "Fonts" / "cour.ttf",
0303 |     ]
0304 |     for candidate in candidates:
0305 |         if candidate.exists():
0306 |             with contextlib.suppress(Exception):
0307 |                 return ImageFont.truetype(str(candidate), size)
0308 |     return ImageFont.load_default()
```

Function: render_terminal_image

Lines: 311 to 338

Description: Analyzes and executes render_terminal_image logic.

```

0311 | def render_terminal_image(source_text: str, dest_path: Path, title: str, max_lines: int = 28) -> Path:
0312 |     lines = source_text.splitlines()[:max_lines]
0313 |     if not lines:
0314 |         lines = ["<no output>"]
0315 |     font = resolve_font(18)
0316 |     small = resolve_font(16)
0317 |     measure = Image.new("RGB", (10, 10), "white")
0318 |     draw = ImageDraw.Draw(measure)
0319 |     line_height = 26
0320 |     widths = [int(draw.textlength(line, font=font)) for line in lines]
0321 |     width = max(1080, max(widths) + 90)
0322 |     height = 96 + len(lines) * line_height + 30
0323 |     image = Image.new("RGB", (width, height), "#0B1220")
0324 |     draw = ImageDraw.Draw(image)
0325 |     draw.rounded_rectangle((12, 12, width - 12, height - 12), radius=18, fill="#0F172A", outline="#2D436A", width=2)
0326 |     draw.rounded_rectangle((34, 34, width - 34, height - 34), radius=14, fill="#101B34", outline="#314B75", width=1)
0327 |     draw.ellipse((52, 48, 66, 62), fill="#FF5F56")
0328 |     draw.ellipse((74, 48, 88, 62), fill="#FFBD2E")
0329 |     draw.ellipse((96, 48, 110, 62), fill="#27C93F")
0330 |     draw.text((128, 44), title, font=small, fill="#D8E3F7")
0331 |     y = 84
0332 |     for index, line in enumerate(lines, start=1):
0333 |         prefix = f"{index:02d} "
0334 |         draw.text((56, y), prefix, font=font, fill="#5BA7FF")
0335 |         draw.text((98, y), line, font=font, fill="#E5EEF9")
0336 |         y += line_height
0337 |     image.save(dest_path)
0338 |     return dest_path

```

Function: capture_gui

Lines: 341 to 357

Description: Analyzes and executes capture_gui logic.

```

0341 | def capture_gui(path: Path) -> Path:
0342 |     if path.exists():
0343 |         return path
0344 |     root = tk.Tk()
0345 |     app = IDSPProductGUI(root)
0346 |     app.command_var.set("traffic")
0347 |     app.run_freeform_command()
0348 |
0349 |     def _capture() -> None:
0350 |         root.update_idletasks()
0351 |         image = ImageGrab.grab(window=root.winfo_id())
0352 |         image.save(path)
0353 |         root.destroy()
0354 |
0355 |     root.after(2200, _capture)
0356 |     root.mainloop()
0357 |     return path

```

Function: ensure_assets

Lines: 360 to 372

Description: Analyzes and executes ensure_assets logic.

```
0360 | def ensure_assets() -> dict[str, Path]:
0361 |     ensure_dirs()
0362 |     assets: dict[str, Path] = {}
0363 |     gui = capture_gui(RAW_ASSET_DIR / "gui_screenshot_window.png")
0364 |     assets["gui"] = gui
0365 |     text_outputs = {
0366 |         "status_terminal.png": run_text_command(["status"]),
0367 |         "attacks_terminal.png": run_text_command(["attacks"]),
0368 |         "ports_terminal.png": run_text_command(["ports", "--limit", "20"]),
0369 |     }
0370 |     for name, text in text_outputs.items():
0371 |         assets[name] = render_terminal_image(text, ASSET_DIR / name, title=name.replace("_", " ").replace(".png", ""))
0372 |     return assets
```

Function: source_notes

Lines: 375 to 530

Description: Analyzes and executes source_notes logic.

```

0375 | def source_notes() -> list[SourceNote]:
0376 |     return [
0377 |         SourceNote(
0378 |             "NIST-800-94",
0379 |             "Guide to Intrusion Detection and Prevention Systems (IDPS)",
0380 |             "Karen Scarfone and Peter Mell",
0381 |             "2007",
0382 |             "https://csrc.nist.gov/pubs/sp/800/94/final",
0383 |             "Accessed May 15, 2026.",
0384 |             "Operational foundation for how IDS and IPS technologies should be understood, designed, monitored, and
0385 |         ),
0386 |         SourceNote(
0387 |             "NIST-800-61R2",
0388 |             "Computer Security Incident Handling Guide",
0389 |             "Paul Cichonski, Thomas Millar, Tim Grance, Karen Scarfone",
0390 |             "2012",
0391 |             "https://csrc.nist.gov/pubs/sp/800/61/r2/final",
0392 |             "Accessed May 15, 2026; page notes the document was withdrawn on April 3, 2025 and superseded by Rev. 3
0393 |             "Used to frame the product as part of an incident handling workflow rather than as a standalone algorithm
0394 |         ),
0395 |         SourceNote(
0396 |             "NIST-800-83R1",
0397 |             "Guide to Malware Incident Prevention and Handling for Desktops and Laptops",
0398 |             "Murugiah Souppaya and Karen Scarfone",
0399 |             "2013",
0400 |             "https://csrc.nist.gov/pubs/sp/800/83/r1/final",
0401 |             "Accessed May 15, 2026.",
0402 |             "Supports the report's discussion of file triage, malware-like indicators, and incident response preparat
0403 |         ),
0404 |         SourceNote(
0405 |             "NIST-800-137",
0406 |             "Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations",
0407 |             "Kelley Dempsey et al.",
0408 |             "2011",
0409 |             "https://csrc.nist.gov/pubs/sp/800/137/final",
0410 |             "Accessed May 15, 2026.",
0411 |             "Provides the conceptual frame for continuous monitoring and why persistent telemetry matters.",
0412 |         ),
0413 |         SourceNote(
0414 |             "KDD-1999",
0415 |             "KDD Cup 1999: Computer Network Intrusion Detection",
0416 |             "SIGKDD Cup organizers",
0417 |             "1999",
0418 |             "https://www.kdd.org/kdd-cup/view/kdd-cup-1999/Data",
0419 |             "Accessed May 15, 2026.",
0420 |             "Primary source for the benchmark dataset bundled with this repository.",
0421 |         ),
0422 |         SourceNote(
0423 |             "UCI-KDD99",
0424 |             "KDD Cup 1999 Data",
0425 |             "Salvatore Stolfo, Wei Fan, Wenke Lee, Andreas Prodromidis, Philip Chan",
0426 |             "1999",
0427 |             "https://archive.ics.uci.edu/dataset/130/kdd+cup+1999+data",
0428 |             "Accessed May 15, 2026.",
0429 |             "Used for formal citation metadata, instance counts, and licensing context.",
0430 |         ),
0431 |         SourceNote(
0432 |             "CIC-IDS2017",
0433 |             "Intrusion Detection Evaluation Dataset (CIC-IDS2017)",
0434 |             "Canadian Institute for Cybersecurity, University of New Brunswick",
0435 |             "2017",
0436 |             "https://www.unb.ca/cic/datasets/ids-2017.html",
0437 |             "Accessed May 15, 2026.",
0438 |             "Authoritative dataset description for modern attacks, five-day collection schedule, and flow feature g
0439 |         ),
0440 |         SourceNote(
0441 |             "UNSW-NB15",
0442 |             "The UNSW-NB15 Dataset",

```

```

0443 |         "UNSW Canberra at ADFA",
0444 |         "2015-2024 page snapshot",
0445 |         "https://research.unsw.edu.au/projects/unsw-nb15-dataset",
0446 |         "Accessed May 15, 2026.",
0447 |         "Source for the dataset composition, attack families, record counts, and split sizes.",
0448 |     ),
0449 |     SourceNote(
0450 |         "DENNING-1987",
0451 |         "An Intrusion-Detection Model",
0452 |         "Dorothy E. Denning",
0453 |         "1987",
0454 |         "https://doi.org/10.1109/TSE.1987.232894",
0455 |         "Accessed via DOI and search metadata on May 15, 2026.",
0456 |         "Seminal conceptual basis for anomaly-oriented intrusion detection built around profiles and deviations",
0457 |     ),
0458 |     SourceNote(
0459 |         "LEE-1999",
0460 |         "A Data Mining Framework for Building Intrusion Detection Models",
0461 |         "Wenke Lee, Salvatore Stolfo, Kui Mok",
0462 |         "1999",
0463 |         "https://doi.org/10.1109/SECPRI.1999.766909",
0464 |         "Accessed via DOI-linked search metadata on May 15, 2026.",
0465 |         "Seminal bridge from audit data toward data-mining-based intrusion detection workflows.",
0466 |     ),
0467 |     SourceNote(
0468 |         "LIU-LANG-2019",
0469 |         "Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey",
0470 |         "Hongyu Liu and Bo Lang",
0471 |         "2019",
0472 |         "https://www.mdpi.com/2076-3417/9/20/4396",
0473 |         "Accessed May 15, 2026.",
0474 |         "Broad survey used to situate this repository inside the ML and DL IDS landscape.",
0475 |     ),
0476 |     SourceNote(
0477 |         "CANTONE-2024",
0478 |         "On the Cross-Dataset Generalization of Machine Learning for Network Intrusion Detection",
0479 |         "Marco Cantone, Claudio Marrocco, Alessandro Bria",
0480 |         "2024",
0481 |         "https://arxiv.org/abs/2402.10974",
0482 |         "Accessed May 15, 2026.",
0483 |         "Modern evidence that same-dataset accuracy can mask serious generalization failure across network environments.",
0484 |     ),
0485 |     SourceNote(
0486 |         "MITRE-ATTACK",
0487 |         "MITRE ATT&CK",
0488 |         "MITRE",
0489 |         "Living knowledge base",
0490 |         "https://attack.mitre.org/index.html",
0491 |         "Accessed May 15, 2026; landing page advertised ATT&CK v19.",
0492 |         "Used to connect the repository's hunt and IOC features to contemporary detection engineering language.",
0493 |     ),
0494 |     SourceNote(
0495 |         "PY-ZIPAPP",
0496 |         "zipapp - Manage executable Python zip archives",
0497 |         "Python Software Foundation",
0498 |         "2026 documentation snapshot",
0499 |         "https://docs.python.org/3.14/library/zipapp.html",
0500 |         "Accessed May 15, 2026.",
0501 |         "Supports the report's packaging discussion around distributable Python applications.",
0502 |     ),
0503 |     SourceNote(
0504 |         "PY-TKINTER",
0505 |         "tkinter - Python interface to Tcl/Tk",
0506 |         "Python Software Foundation",
0507 |         "2026 documentation snapshot",
0508 |         "https://docs.python.org/3/library/tkinter.html",
0509 |         "Accessed May 15, 2026.",
0510 |         "Used to contextualize the GUI layer and its cross-platform baseline.",
0511 |     ),

```

```

0512 |         SourceNote(
0513 |             "GH-OIDC-PYPI",
0514 |             "Configuring OpenID Connect in PyPI",
0515 |             "GitHub Docs",
0516 |             "2026",
0517 |             "https://docs.github.com/en/actions/how-to-secure-your-work/security-harden-deployments/oidc-in-pypi",
0518 |             "Accessed May 15, 2026.",
0519 |             "Supports the release automation analysis for trusted package publishing.",
0520 |         ),
0521 |         SourceNote(
0522 |             "PYPI-TRUSTED-PUBLISHER",
0523 |             "Creating a PyPI project with a Trusted Publisher",
0524 |             "PyPI Documentation",
0525 |             "2026",
0526 |             "https://docs.pypi.org/trusted-publishers/creating-a-project-through-oidc/",
0527 |             "Accessed May 15, 2026.",
0528 |             "Used in the packaging chapter to describe zero-token project bootstrapping.",
0529 |         ),
0530 |     ]

```

Function: feature_descriptions

Lines: 533 to 576

Description: Analyzes and executes feature_descriptions logic.

```

0533 | def feature_descriptions() -> dict[str, str]:
0534 |     return {
0535 |         "duration": "Connection duration in seconds. Short, repetitive sessions often characterize scanning and flood attacks.",
0536 |         "protocol_type": "Encoded transport protocol. Even in encoded form, the field captures major shifts among TCP, UDP, and ICMP.",
0537 |         "service": "Encoded service indicator. It condenses port-and-protocol usage into an application-oriented signature.",
0538 |         "flag": "Encoded TCP flag state. This is especially informative for incomplete handshakes, resets, and other anomalies.",
0539 |         "src_bytes": "Bytes sent from source to destination. Useful for distinguishing sparse probes from bulk transfers.",
0540 |         "dst_bytes": "Bytes sent from destination to source. Combined with source bytes, this exposes asymmetric communication patterns.",
0541 |         "land": "Boolean marker for source and destination equivalence. Historically useful for detecting malformed packets.",
0542 |         "wrong_fragment": "Count of malformed or misplaced fragments. Elevated values are uncommon in healthy traffic.",
0543 |         "urgent": "Urgent packet count. Rare in ordinary enterprise activity, so spikes deserve explanation.",
0544 |         "hot": "Count of suspicious content indicators derived from the original dataset schema. It acts as a coarse-grained anomaly signal.",
0545 |         "num_failed_logins": "Failed authentication attempts. Directly relevant to brute-force and credential abuse.",
0546 |         "logged_in": "Binary success marker for authenticated sessions. The repository's learned model treats this as a high-value signal.",
0547 |         "num_compromised": "Number of compromised conditions reported in the source schema. High values are strong indicators of compromise.",
0548 |         "root_shell": "Binary marker for root shell acquisition. This is a direct privilege escalation cue.",
0549 |         "su_attempted": "Whether privilege switching was attempted. Even low frequency events matter because they are rare.",
0550 |         "num_root": "Count of root-level operations or accesses. A useful escalation strength signal.",
0551 |         "num_file_creations": "File creation count within the session. Helpful when reasoning about payload droppers or malware.",
0552 |         "num_shells": "Shell spawns associated with the session. Repeated shell creation is highly suspicious in most contexts.",
0553 |         "num_access_files": "Sensitive file access count. Supports coarse inference about reconnaissance or data collection.",
0554 |         "num_outbound_cmds": "Outbound command count. The field is almost always zero in the bundled data, which indicates a lack of command execution.",
0555 |         "is_host_login": "Binary flag for privileged local host login. Rare and context-sensitive.",
0556 |         "is_guest_login": "Binary flag for guest access. Can signal low-trust account activity or misconfiguration.",
0557 |         "count": "Connections to the same host in a short window. This is one of the strongest attack separators in the dataset.",
0558 |         "srv_count": "Connections to the same service in a short window. Useful for burst-style probes and service-specific attacks.",
0559 |         "error_rate": "SYN error rate over the short window. High values often reveal handshake-heavy denial or probe attacks.",
0560 |         "srv_error_rate": "Service-scoped SYN error rate. A tighter lens on service-targeted failures.",
0561 |         "reset_rate": "Reset error rate over the window. Helpful for failed access attempts and aggressive scanning.",
0562 |         "srv_reset_rate": "Service-scoped reset error rate. Useful when a single application endpoint is under stress.",
0563 |         "same_srv_rate": "Fraction of recent connections hitting the same service. Stable benign workflows and attack patterns.",
0564 |         "diff_srv_rate": "Fraction of recent connections hitting different services. Elevated values can indicate host-hopping.",
0565 |         "srv_diff_host_rate": "How widely a service is spread across hosts in the recent window. Strong for spotting service-specific activity.",
0566 |         "dst_host_count": "Historical count of connections to the destination host. In this repository it is one of the most powerful features.",
0567 |         "dst_host_srv_count": "Historical count of connections to the destination host and service pair.",
0568 |         "dst_host_same_srv_rate": "Fraction of destination-host connections using the same service. A useful stability signal.",
0569 |         "dst_host_diff_srv_rate": "Fraction of destination-host connections using different services. Supports service-level analysis.",
0570 |         "dst_host_same_src_port_rate": "Fraction of connections to the destination host sharing the same source port.",
0571 |         "dst_host_srv_diff_host_rate": "Fraction of a destination service's traffic arriving from different hosts. Useful for identifying targeted attacks.",
0572 |         "dst_host_error_rate": "Host-level SYN error rate. One of the most class-separating host-centric fields in the dataset.",
0573 |         "dst_host_srv_error_rate": "Host-and-service SYN error rate. Important for service-targeted denial or misconfigurations.",
0574 |         "dst_host_reset_rate": "Host-level reset error rate.",
0575 |         "dst_host_srv_reset_rate": "Host-and-service reset error rate. This often complements the SYN error fields.",
0576 |     }

```

Function: command_descriptions

Lines: 579 to 607

Description: Analyzes and executes command_descriptions logic.

```

0579 | def command_descriptions() -> dict[str, str]:
0580 |     return {
0581 |         "shell": "Starts the interactive IDS shell that exposes the product's own command grammar rather than the h
0582 |         "gui": "Opens the Tkinter-based graphical console for status review, scans, hunting, network probing, and f
0583 |         "status": "Summarizes installation mode, dataset volumes, learned model indicators, cached outputs, and his
0584 |         "traffic": "Reports encoded traffic distribution from the bundled train and test CSV files.",
0585 |         "attacks": "Shows attack share statistics together with the most separating learned indicators.",
0586 |         "datasets": "Lists local sources and the hard-coded external dataset catalog.",
0587 |         "malware": "Infers malware-like behavior categories from CSV-derived features rather than from malware fami
0588 |         "learn": "Builds or refreshes the repository's pure-Python Gaussian profile using bundled and generated CSV
0589 |         "scan": "Analyzes a CSV file row-by-row, scores risk, classifies behavior, and exports results by default.",
0590 |         "export": "Convenience alias for full-result CSV and JSON export production.",
0591 |         "import": "Copies an external CSV into the product import area so it can be indexed and analyzed consistent
0592 |         "download": "Fetches a public URL into the import area; useful for bringing in external benchmark material.
0593 |         "index": "Inspects schema-like properties of a CSV file, including column counts and sample rows.",
0594 |         "hunt": "Searches datasets, imports, and reports for a textual pattern.",
0595 |         "ioc": "Manages and hunts indicators of compromise stored in the product IOC file.",
0596 |         "netstat": "Parses local network connections from the host's `netstat -ano` output.",
0597 |         "ports": "Shows local listening ports and UDP endpoints.",
0598 |         "port": "Explains a single port using the built-in service and risk knowledge base.",
0599 |         "probe": "Performs an authorized TCP connect probe against a host and bounded set of ports.",
0600 |         "dns": "Resolves a hostname and attempts reverse lookups where possible.",
0601 |         "ps": "Lists local processes for lightweight host triage.",
0602 |         "hash": "Calculates file hashes for later comparison and response workflows.",
0603 |         "filescan": "Hashes a file and applies simple suspicious-pattern triage logic.",
0604 |         "reports": "Lists generated downloadable analysis exports.",
0605 |         "runs": "Lists historical classical and DNN training runs from `automation/runs`.",
0606 |         "cache": "Enumerates cached command artifacts stored by the product.",
0607 |     }

```

Function: listing_specs

Lines: 610 to 633

Description: Analyzes and executes listing_specs logic.

```

0610 | def listing_specs() -> list[ListingSpec]:
0611 |     return [
0612 |         ListingSpec("ids_app/product_terminal.py", 1, 130, "Listing 1. Runtime bootstrap and packaged asset staging",
0613 |         ListingSpec("ids_app/product_terminal.py", 403, 563, "Listing 2. Streaming row ingestion and pure-Python mo
0614 |         ListingSpec("ids_app/product_terminal.py", 573, 662, "Listing 3. Probabilistic scoring and behavioral class
0615 |         ListingSpec("ids_app/product_terminal.py", 801, 941, "Listing 4. CSV analysis and export pipeline", "The an
0616 |         ListingSpec("ids_app/product_terminal.py", 1114, 1309, "Listing 5. Local service knowledge, netstat parsing,
0617 |         ListingSpec("ids_app/product_terminal.py", 1402, 1578, "Listing 6. File triage, IOC handling, and text hunt
0618 |         ListingSpec("ids_app/product_terminal.py", 1587, 1709, "Listing 7. Embedded shell command layer", "Instead
0619 |         ListingSpec("ids_app/product_terminal.py", 1713, 1928, "Listing 8. Status, traffic, attacks, malware, and ru
0620 |         ListingSpec("ids_app/product_terminal.py", 2023, 2350, "Listing 9. Command dispatch, parser construction, a
0621 |         ListingSpec("ids_app/product_gui.py", 1, 205, "Listing 10. GUI theme, layout, and dark-mode composition", "
0622 |         ListingSpec("ids_app/product_gui.py", 206, 348, "Listing 11. GUI command execution and output coloring", "T
0623 |         ListingSpec("ids_app/classical.py", 1, 113, "Listing 12. Classical training suite", "This file packages a co
0624 |         ListingSpec("ids_app/dnn.py", 1, 145, "Listing 13. DNN training suite", "The DNN path is deliberately thinne
0625 |         ListingSpec("ids_app/data.py", 1, 123, "Listing 14. Dataset loading and split management", "All higher-leve
0626 |         ListingSpec("ids_app/storage.py", 1, 50, "Listing 15. Run and job storage primitives", "The storage helper
0627 |         ListingSpec("ids_app/api.py", 1, 106, "Listing 16. API endpoints", "This module exposes FastAPI endpoints
0628 |         ListingSpec("ids_app/terminal.py", 1, 220, "Listing 17. Legacy terminal interface overview", "The legacy te
0629 |         ListingSpec("ids_app/terminal.py", 320, 646, "Listing 18. Legacy train-and-shell orchestration", "This exce
0630 |         ListingSpec("pyproject.toml", 1, 60, "Listing 19. Installable package metadata and endpoints", "The projec
0631 |         ListingSpec(".github/workflows/release.yml", 1, 80, "Listing 20. Release and PyPI workflow", "The workflow
0632 |         ListingSpec("scripts/build_python_package.py", 1, 120, "Listing 21. Wheel and source distribution helper",
0633 |     ]

```

Function: extract_code

Lines: 636 to 640

Description: Analyzes and executes extract_code logic.

```
0636 | def extract_code(path: str, start: int, end: int) -> str:
0637 |     text = (ROOT / path).read_text(encoding="utf-8", errors="ignore").splitlines()
0638 |     width = len(str(end))
0639 |     numbered = [f"{index:{width}d}: {line}" for index, line in enumerate(text[start - 1:end], start=start)]
0640 |     return "\n".join(numbered)
```

Function: environment_findings

Lines: 643 to 654

Description: Analyzes and executes environment_findings logic.

```
0643 | def environment_findings() -> list[tuple[str, str]]:
0644 |     return [
0645 |         ("Python runtime", subprocess.run(["python", "--version"], capture_output=True, text=True, check=False).stdout),
0646 |         ("reportlab", "available" if try_import("reportlab") else "missing"),
0647 |         ("Pillow", "available" if try_import("PIL") else "missing"),
0648 |         ("tkinter", "available" if try_import("tkinter") else "missing"),
0649 |         ("numpy", "available" if try_import("numpy") else "missing"),
0650 |         ("joblib", "available" if try_import("joblib") else "missing"),
0651 |         ("fastapi", "available" if try_import("fastapi") else "missing"),
0652 |         ("pydantic", "available" if try_import("pydantic") else "missing"),
0653 |         ("tensorflow", "available" if try_import("tensorflow") else "missing"),
0654 |     ]
```

Function: legacy_terminal_failure

Lines: 657 to 665

Description: Analyzes and executes legacy_terminal_failure logic.

```
0657 | def legacy_terminal_failure() -> str:
0658 |     result = subprocess.run(
0659 |         ["python", "-m", "ids_app.terminal", "--help"],
0660 |         cwd=ROOT,
0661 |         capture_output=True,
0662 |         text=True,
0663 |         check=False,
0664 |     )
0665 |     return (result.stderr or result.stdout).strip()
```

Function: build_context

Lines: 668 to 693

Description: Analyzes and executes build_context logic.

```

0668 | def build_context() -> dict[str, Any]:
0669 |     assets = ensure_assets()
0670 |     status = load_json_command(["status"])
0671 |     attacks = load_json_command(["attacks"])
0672 |     datasets = load_json_command(["datasets"])
0673 |     runs = load_json_command(["runs"])
0674 |     ports = load_json_command(["ports"])
0675 |     counts = repo_counts()
0676 |     modules = module_inventory()
0677 |     subcommands = list(command_descriptions().items())
0678 |     legacy_failure = legacy_terminal_failure()
0679 |     return {
0680 |         "assets": assets,
0681 |         "status": status,
0682 |         "attacks": attacks,
0683 |         "datasets": datasets,
0684 |         "runs": runs,
0685 |         "ports": ports,
0686 |         "counts": counts,
0687 |         "modules": modules,
0688 |         "subcommands": subcommands,
0689 |         "sources": source_notes(),
0690 |         "feature_descriptions": feature_descriptions(),
0691 |         "legacy_failure": legacy_failure,
0692 |         "environment": environment_findings(),
0693 |     }

```

Function: module_overview_table

Lines: 696 to 720

Description: Analyzes and executes module_overview_table logic.

```

0696 | def module_overview_table(context: dict[str, Any]) -> LongTable:
0697 |     rows = ["Module", "Lines", "Functions", "Classes", "Dominant Role"]
0698 |     for module in context["modules"]:
0699 |         if module.path.endswith("product_terminal.py"):
0700 |             role = "Primary CLI, analysis core, caching, IOC, network triage, parser"
0701 |         elif module.path.endswith("product_gui.py"):
0702 |             role = "Dark-mode Tk GUI for the product console"
0703 |         elif module.path.endswith("classical.py"):
0704 |             role = "Classical supervised model training suite"
0705 |         elif module.path.endswith("dnn.py"):
0706 |             role = "TensorFlow-based deep learning suite"
0707 |         elif module.path.endswith("api.py"):
0708 |             role = "FastAPI interface for runs and jobs"
0709 |         elif module.path.endswith("data.py"):
0710 |             role = "Dataset loading, sampling, and summary helpers"
0711 |         elif module.path.endswith("storage.py"):
0712 |             role = "Run and job persistence helpers"
0713 |         elif module.path.endswith("terminal.py"):
0714 |             role = "Legacy command console and training workflow"
0715 |         else:
0716 |             role = "Support module"
0717 |         rows.append([module.path, f"{module.line_count:}", module.function_count, module.class_count, role])
0718 |     table = LongTable(rows, repeatRows=1, colWidths=[2.05 * inch, 0.7 * inch, 0.8 * inch, 0.7 * inch, 2.55 * inch])
0719 |     table.setStyle(table_style())
0720 |     return table

```

Function: runs_table

Lines: 723 to 741

Description: Analyzes and executes runs_table logic.

```

0723 | def runs_table(context: dict[str, Any]) -> LongTable:
0724 |     rows = ["Run ID", "Kind", "Train rows", "Test rows", "Best model", "Accuracy"]
0725 |     for run in context["runs"]:
0726 |         best_model = run.get("best_model", "-")
0727 |         best_accuracy = max((item.get("metrics", {}).get("accuracy", 0.0) for item in run.get("results", [])), default=0.0)
0728 |         dataset = run.get("dataset", {})
0729 |         rows.append(
0730 |             [
0731 |                 run.get("run_id", "-"),
0732 |                 run.get("kind", "-"),
0733 |                 f"{dataset.get('train_rows', 0):,}",
0734 |                 f"{dataset.get('test_rows', 0):,}",
0735 |                 best_model,
0736 |                 f"{best_accuracy:.4f}",
0737 |             ]
0738 |         )
0739 |     table = LongTable(rows, repeatRows=1, colWidths=[1.8 * inch, 1.05 * inch, 0.8 * inch, 0.8 * inch, 1.65 * inch, 0.8 * inch])
0740 |     table.setStyle(table_style())
0741 |     return table

```

Function: dataset_comparison_table

Lines: 744 to 780

Description: Analyzes and executes dataset_comparison_table logic.

```

0744 | def dataset_comparison_table(context: dict[str, Any]) -> LongTable:
0745 |     train = context["status"]["datasets"]["train"]
0746 |     test = context["status"]["datasets"]["test"]
0747 |     rows = [
0748 |         ["Dataset", "Origin", "Rows", "Features/Columns", "Attack coverage note"],
0749 |         [
0750 |             "Bundled KDD train",
0751 |             "Local repo asset",
0752 |             f"{train['rows']:,}",
0753 |             str(train["columns"]),
0754 |             "Binary normal/attack labels from the KDD99 derivative used by the project.",
0755 |         ],
0756 |         [
0757 |             "Bundled KDD test",
0758 |             "Local repo asset",
0759 |             f"{test['rows']:,}",
0760 |             str(test["columns"]),
0761 |             "Held-out companion split with a similarly high attack share.",
0762 |         ],
0763 |         [
0764 |             "CIC-IDS2017",
0765 |             "UNB CIC official page",
0766 |             "5 days of PCAP/CSV flow capture",
0767 |             "80+ flow features",
0768 |             "FTP/SSH brute force, DoS, DDoS, Heartbleed, web attacks, infiltration, botnet, port scan.",
0769 |         ],
0770 |         [
0771 |             "UNSW-NB15",
0772 |             "UNSW official page",
0773 |             "2,540,044 total records; 175,341 train / 82,332 test split listed",
0774 |             "49 features plus class",
0775 |             "Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms.",
0776 |         ],
0777 |     ]
0778 |     table = LongTable(rows, repeatRows=1, colWidths=[1.25 * inch, 1.25 * inch, 1.25 * inch, 1.1 * inch, 2.45 * inch])
0779 |     table.setStyle(table_style())
0780 |     return table

```

Function: environment_table

Lines: 783 to 788

Description: Analyzes and executes environment_table logic.

```
0783 | def environment_table(context: dict[str, Any]) -> Table:
0784 |     rows = [["Component", "Observed state"]]
0785 |     rows.extend(context["environment"])
0786 |     table = Table(rows, colWidths=[1.8 * inch, 4.9 * inch])
0787 |     table.setStyle(table_style())
0788 |     return table
```

Function: make_bar_chart

Lines: 791 to 807

Description: Analyzes and executes make_bar_chart logic.

```
0791 | def make_bar_chart(title: str, items: list[tuple[str, float]], width: float = 6.5 * inch, height: float = 2.2 * inch):
0792 |     drawing = Drawing(width, height)
0793 |     drawing.add(Rect(0, 0, width, height, fillColor=colors.white, strokeColor=GRID, strokeWidth=0.5))
0794 |     drawing.add(String(12, height - 16, title, fontName=BODY_BOLD, fontSize=11, fillColor=ACCENT))
0795 |     max_value = max((value for _, value in items), default=1.0)
0796 |     bar_area_top = height - 34
0797 |     bar_height = 18
0798 |     gap = 12
0799 |     start_y = bar_area_top - bar_height
0800 |     for index, (label, value) in enumerate(items):
0801 |         y = start_y - index * (bar_height + gap)
0802 |         bar_width = 0 if max_value == 0 else (value / max_value) * (width - 170)
0803 |         drawing.add(String(12, y + 4, label, fontName=BODY_FONT, fontSize=8.5, fillColor=TEXT))
0804 |         drawing.add(Rect(118, y, width - 150, bar_height, fillColor=colors.HexColor("#F3F6FA"), strokeColor=GRID, strokeWidth=0.5))
0805 |         drawing.add(Rect(118, y, bar_width, bar_height, fillColor=colors.HexColor("#4F81BD"), strokeColor=colors.HexColor("#F3F6FA"), strokeWidth=0.5))
0806 |         drawing.add(String(width - 26, y + 4, f"{value:,.2f}", fontName=BODY_FONT, fontSize=8.2, fillColor=TEXT, textAngle=90))
0807 |     return drawing
```

Function: architecture_diagram

Lines: 810 to 840

Description: Analyzes and executes architecture_diagram logic.

```

0810 | def architecture_diagram() -> Drawing:
0811 |     width = 6.8 * inch
0812 |     height = 3.6 * inch
0813 |     drawing = Drawing(width, height)
0814 |     drawing.add(Rect(0, 0, width, height, fillColor=colors.white, strokeColor=GRID, strokeWidth=0.5))
0815 |     boxes = [
0816 |         (24, 250, 138, 44, "Entry Points", "product_app.py / pyproject"),
0817 |         (188, 250, 176, 44, "Core CLI", "product_terminal.py"),
0818 |         (390, 250, 140, 44, "GUI", "product_gui.py"),
0819 |         (188, 184, 176, 44, "Analytics", "learn / scan / export"),
0820 |         (24, 118, 138, 44, "Support", "data.py / storage.py"),
0821 |         (188, 118, 176, 44, "Optional ML", "classical.py / dnn.py"),
0822 |         (390, 118, 140, 44, "API", "api.py"),
0823 |         (188, 52, 176, 44, "Artifacts", "automation/runs + cache"),
0824 |     ]
0825 |     for x, y, w, h, title, subtitle in boxes:
0826 |         drawing.add(Rect(x, y, w, h, fillColor=ACCENT_LIGHT, strokeColor=colors.HexColor("#7A9CC6"), strokeWidth=0.5))
0827 |         drawing.add(String(x + 8, y + h - 16, title, fontName=BODY_BOLD, fontSize=10, fillColor=ACCENT))
0828 |         drawing.add(String(x + 8, y + 12, subtitle, fontName=BODY_FONT, fontSize=8.3, fillColor=TEXT))
0829 |     lines = [
0830 |         (162, 272, 188, 272),
0831 |         (364, 272, 390, 272),
0832 |         (276, 250, 276, 228),
0833 |         (276, 184, 276, 162),
0834 |         (162, 140, 188, 140),
0835 |         (364, 140, 390, 140),
0836 |         (276, 118, 276, 96),
0837 |     ]
0838 |     for x1, y1, x2, y2 in lines:
0839 |         drawing.add(Line(x1, y1, x2, y2, strokeColor=colors.HexColor("#7A7A7A"), strokeWidth=1))
0840 |     return drawing

```

Function: append_story_intro

Lines: 843 to 862

Description: Analyzes and executes append_story_intro logic.

```

0843 | def append_story_intro(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0844 |     train = context["status"]["datasets"]["train"]
0845 |     test = context["status"]["datasets"]["test"]
0846 |     story.append(Spacer(1, 0.6 * inch))
0847 |     story.append(Paragraph("IDS Sentinel Terminal", s["Title"]))
0848 |     story.append(Paragraph("A research-style technical assessment of a productized intrusion-analysis CLI and GUI",
0849 |     story.append(Paragraph(f"Prepared from repository evidence and web research on {ACCESS_DATE}", s["Small"])))
0850 |     story.append(Spacer(1, 0.3 * inch))
0851 |     summary = (
0852 |         f"This report examines the IDS Sentinel Terminal repository as both a software product and a research artifact",
0853 |         f"The current codebase bundles two KDD-derived CSV assets with {train['rows']:,} training rows and {test['rows']:,} test rows",
0854 |         f"ships a pure-Python self-learning model, exposes a Tk-based GUI, and records multiple historical classical analysis artifacts",
0855 |         f"The report also situates the project against NIST guidance, MITRE ATT&CK, and more modern benchmark datasets",
0856 |     )
0857 |     story.append(paragraph(summary, s["BodyText"]))
0858 |     story.append(Spacer(1, 0.25 * inch))
0859 |     story.append(bullet("Purpose: determine what this repository already delivers as an analyst-facing tool and when it was last updated"))
0860 |     story.append(bullet("Method: combine local repo inspection, executable command evidence, live GUI capture, and a manual review of artifacts"))
0861 |     story.append(bullet("Scope: architecture, datasets, scoring logic, training artifacts, operational commands, packaging, and documentation"))
0862 |     story.append(PageBreak())

```

Function: append_abstract

Lines: 865 to 880

Description: Analyzes and executes append_abstract logic.

```

0865 | def append_abstract(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0866 |     attacks = context["attacks"]["model_indicators"]
0867 |     story.append(Paragraph("Abstract", s["Heading1"]))
0868 |     text = (
0869 |         "IDS Sentinel Terminal is an attempt to recast a machine-learning-oriented intrusion detection repository in
0870 |         "Instead of stopping at model training scripts, the project now exposes an installable command-line interface
0871 |         "This report studies the repository in the manner of a software-centric research paper: first by grounding
0872 |         f"Two observations dominate the evidence. First, the repository's own learned profile finds high separation
0873 |         "The report therefore concludes that IDS Sentinel Terminal is already useful as a local analysis shell and
0874 |     )
0875 |     story.append(paragraph(text, s["BodyText"]))
0876 |     story.append(Paragraph("Research Questions", s["Heading2"]))
0877 |     story.append(bullet("How closely does the repository's current design align with established IDS operational gu
0878 |     story.append(bullet("What can be verified directly from the code and historical artifacts about the product's ca
0879 |     story.append(bullet("Which defects or architectural pressures most urgently limit the project's credibility as a
0880 |     story.append(PageBreak())

```

Function: append_chapter_introduction

Lines: 883 to 896

Description: Analyzes and executes append_chapter_introduction logic.

```

0883 | def append_chapter_introduction(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0884 |     counts = context["counts"]
0885 |     story.append(Paragraph("1. Introduction and Research Framing", s["Heading1"]))
0886 |     paragraphs = [
0887 |         "The repository under study no longer behaves like a bare academic notebook or a one-off training script co
0888 |         f"A quick structural count highlights that transition. The working tree presently contains {counts['files']}
0889 |         "This report treats the repository as a combined software engineering and applied security artifact. It asks
0890 |         "Methodologically, the report uses four evidence streams. First, it analyzes the repository itself, includin
0891 |         "A useful way to read the remainder of the document is to keep three layers in mind. The first layer is open
0892 |     ]
0893 |     for text in paragraphs:
0894 |         story.append(paragraph(text, s["BodyText"]))
0895 |     story.append(architecture_diagram())
0896 |     story.append(Paragraph("Figure 1. High-level repository architecture reconstructed from the codebase.", s["Capt

```

Function: append_chapter_foundations

Lines: 899 to 911

Description: Analyzes and executes append_chapter_foundations logic.

```

0899 | def append_chapter_foundations(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0900 |     story.append(Paragraph("2. Standards and Literature Foundation", s["Heading1"]))
0901 |     paragraphs = [
0902 |         "The conceptual ancestry of this repository is easy to locate. Denning's 1987 model framed intrusion detect
0903 |         "NIST's SP 800-94 remains a useful operational anchor because it reminds us that IDS technology is not mere
0904 |         "SP 800-61 and SP 800-83 broaden that interpretation. Incident handling guidance emphasizes that effective
0905 |         "MITRE ATT&CK adds a contemporary vocabulary for talking about what the product can and cannot currently sup
0906 |         "Modern survey work also sharpens the evaluation standard. The 2019 survey by Liu and Lang shows how broad
0907 |     ]
0908 |     for text in paragraphs:
0909 |         story.append(paragraph(text, s["BodyText"]))
0910 |     story.append(dataset_comparison_table(context))
0911 |     story.append(Paragraph("Table 1. Local benchmark assets versus two modern external datasets cited by the product

```

Function: append_chapter_datasets

Lines: 914 to 933

Description: Analyzes and executes `append_chapter_datasets` logic.

```
0914 | def append_chapter_datasets(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0915 |     train = context["status"]["datasets"]["train"]
0916 |     test = context["status"]["datasets"]["test"]
0917 |     story.append(Paragraph("3. Dataset Baseline and Threat Realism", s["Heading1"]))
0918 |     paragraphs = [
0919 |         f"The repository's built-in operating baseline is firmly centered on a KDD-derived connection dataset. The
0920 |         "The feature schema is also historically revealing. KDD-style fields such as `count`, `srv_count`, `serror_
0921 |         "The external dataset catalog hard-coded in the product is therefore a sign of architectural maturity even l
0922 |         "At the same time, the product has not yet operationalized that catalog into an end-to-end evaluation harne
0923 |         "A second realism issue concerns labeling granularity. The bundled assets are binary normal-versus-attack d
0924 |     ]
0925 |     for text in paragraphs:
0926 |         story.append(paragraph(text, s["BodyText"]))
0927 |     drawing = make_bar_chart(
0928 |         "Top learned indicator separations",
0929 |         [(item["feature"], float(item["separation"])) for item in context["attacks"]["model_indicators"][:8]],
0930 |         height=2.8 * inch,
0931 |     )
0932 |     story.append(drawing)
0933 |     story.append(Paragraph("Figure 2. The local Gaussian profile finds the strongest class separation in count-like
```

Function: `append_chapter_architecture`

Lines: 936 to 947

Description: Analyzes and executes `append_chapter_architecture` logic.

```
0936 | def append_chapter_architecture(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0937 |     story.append(Paragraph("4. Repository Architecture and Product Shape", s["Heading1"]))
0938 |     paragraphs = [
0939 |         "From a software engineering perspective, the clearest structural fact is that `ids_app/product_terminal.py
0940 |         "The surrounding modules partly offset that concentration. `product_gui.py` translates the command layer int
0941 |         "This asymmetry also explains why the product is pleasant to use in its successful paths. Because the GUI de
0942 |         "However, the asymmetry creates maintenance pressure. Adding a new command often means editing parser logic.
0943 |     ]
0944 |     for text in paragraphs:
0945 |         story.append(paragraph(text, s["BodyText"]))
0946 |     story.append(module_overview_table(context))
0947 |     story.append(Paragraph("Table 2. Module inventory for the executable Python surfaces in the repository.", s["Cap
```

Function: `append_chapter_detection`

Lines: 950 to 963

Description: Analyzes and executes `append_chapter_detection` logic.

```
0950 | def append_chapter_detection(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0951 |     model = context["status"]["model"]
0952 |     story.append(Paragraph("5. Detection and Analytics Pipeline", s["Heading1"]))
0953 |     paragraphs = [
0954 |         f"The pure-Python learned profile is one of the repository's most pragmatic design choices. According to the
0955 |         "The learning process itself is intentionally transparent. The code builds per-label running statistics acro
0956 |         "That narrative layer is especially visible in `classify_behavior`. The function uses feature combinations i
0957 |         "The export path further strengthens the operational value of the pipeline. The scan routine produces CSV an
0958 |         "The key caveat is that the pipeline's semantic richness exceeds the semantic richness of the source labels
0959 |     ]
0960 |     for text in paragraphs:
0961 |         story.append(paragraph(text, s["BodyText"]))
0962 |     story.append(RLImage(str(context["assets"]["status_terminal.png"]), width=6.8 * inch, height=4.4 * inch))
0963 |     story.append(Paragraph("Figure 3. Terminal-styled rendering of the live `status` command used as primary local c
```

Function: append_chapter_ml

Lines: 966 to 977

Description: Analyzes and executes append_chapter_ml logic.

```
0966 | def append_chapter_ml(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0967 |     story.append(Paragraph("6. Supervised Training Subsystems and Historical Results", s["Heading1"]))
0968 |     paragraphs = [
0969 |         "Although the primary packaged product surface now emphasizes the lightweight analyzer, the repository still
0970 |         "The run artifacts are locally persuasive but scientifically limited. Several classical runs report best acc
0971 |         "The engineering story is even more interesting than the metric story. In the current Python environment use
0972 |         "There is a deeper lesson here. By keeping the pure-Python analyzer separate from the heavier training path
0973 |     ]
0974 |     for text in paragraphs:
0975 |         story.append(paragraph(text, s["BodyText"]))
0976 |     story.append(runs_table(context))
0977 |     story.append(Paragraph("Table 3. Historical run summaries recorded under `automation/runs`.", s["Caption"]))
```

Function: append_chapter_operations

Lines: 980 to 993

Description: Analyzes and executes append_chapter_operations logic.

```
0980 | def append_chapter_operations(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0981 |     story.append(Paragraph("7. Operational Tool Surface", s["Heading1"]))
0982 |     paragraphs = [
0983 |         "A notable strength of IDS Sentinel Terminal is that it does not confine itself to one analytic gesture. The
0984 |         "The GUI reinforces that interpretation. The live window captured for this report demonstrates a dark-mode,
0985 |         "The command set is also disciplined. Where many internal tools would expose raw shell execution or broad re
0986 |         "The main operational weakness in this layer is platform realism. The netstat parsing routine clearly assum
0987 |     ]
0988 |     for text in paragraphs:
0989 |         story.append(paragraph(text, s["BodyText"]))
0990 |     story.append(RLImage(str(context["assets"]["gui"]), width=6.85 * inch, height=4.42 * inch))
0991 |     story.append(Paragraph("Figure 4. Live window capture of the Tk-based GUI running the traffic workflow.", s["Caption"]))
0992 |     story.append(RLImage(str(context["assets"]["ports_terminal.png"]), width=6.8 * inch, height=4.2 * inch))
0993 |     story.append(Paragraph("Figure 5. Terminal-styled rendering of the local port inspection output.", s["Caption"]))
```

Function: append_chapter_distribution

Lines: 996 to 1007

Description: Analyzes and executes append_chapter_distribution logic.

```
0996 | def append_chapter_distribution(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
0997 |     story.append(Paragraph("8. Distribution Engineering and Release Posture", s["Heading1"]))
0998 |     paragraphs = [
0999 |         "Packaging is where this repository most clearly departs from the academic-project stereotype. The `pyprojec
1000 |         "The supporting documentation consulted for this report aligns well with the implementation direction. The
1001 |         "The weakness is again one of truthfulness rather than ambition. The package metadata currently exposes a C
1002 |         "This distinction matters even more because the product now aspires to `pipx`-style consumption. Users who
1003 |     ]
1004 |     for text in paragraphs:
1005 |         story.append(paragraph(text, s["BodyText"]))
1006 |     story.append(environment_table(context))
1007 |     story.append(Paragraph("Table 4. Dependency observations from the exact Python environment used to produce this"))
```

Function: append_chapter_risks

Lines: 1010 to 1051

Description: Analyzes and executes `append_chapter_risks` logic.

```
1010 | def append_chapter_risks(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1011 |     story.append(Paragraph("9. Critical Findings, Bugs, and Optimization Priorities", s["Heading1"]))
1012 |     findings = [
1013 |         "Undeclared dependencies are the most important verified product issue. The installable metadata does not L",
1014 |         "The legacy terminal fails immediately in the observed environment with `ModuleNotFoundError: No module name",
1015 |         "The API module imports FastAPI and Pydantic at module import time. Because these packages are absent, the ",
1016 |         "Network inspection behavior is Windows-centric. The code parses `netstat -ano` output and consults the Wind",
1017 |         "The core orchestration file is large and multi-concern. This is not an immediate runtime defect, but it is",
1018 |         "There is still a naming mismatch between the repo's historical local folder name and the newer `IDS-Sentin",
1019 |         "The scientific evaluation story remains bound to KDD-style data for local evidence. Without integrated cro",
1020 |         "The release workflow's smoke test is modest. It proves wheel installability and basic CLI health but does n",
1021 |     ]
1022 |     story.append(paragraph("The report's bug audit is intentionally conservative: only issues that were directly sup",
1023 |     for item in findings:
1024 |         story.append(bullet(item, s["BulletBody"]))
1025 |     story.append(Paragraph("Observed failure evidence", s["Heading2"]))
1026 |     story.append(
1027 |         Preformatted(
1028 |             textwrap.shorten(context["legacy_failure"], width=1200, placeholder=" ..."),
1029 |             ParagraphStyle(
1030 |                 "FailureBlock",
1031 |                 fontName=MONO,
1032 |                 fontSize=7.6,
1033 |                 leading=9.1,
1034 |                 textColor=colors.HexColor("#7A1C1C"),
1035 |                 backColor=colors.HexColor("#FFF2F2"),
1036 |                 borderPadding=6,
1037 |                 borderWidth=0.4,
1038 |                 borderColor=colors.HexColor("#E3B9B9"),
1039 |             ),
1040 |         )
1041 |     )
1042 |     roadmaps = [
1043 |         "Refactor `product_terminal.py` into command modules grouped by concern: data, analytics, IOC, host triage,",
1044 |         "Declare an explicit base dependency set for the operational console and separate extras such as `[api]`, `[",
1045 |         "Add a platform abstraction for local host inspection instead of hard-coding Windows netstat assumptions.",
1046 |         "Build an integrated evaluation harness for CIC-IDS2017 and UNSW-NB15 so that the external catalog becomes a",
1047 |         "Add contract tests for `status`, `scan`, export generation, cache listing, and parser behavior, plus smoke",
1048 |     ]
1049 |     story.append(Paragraph("Priority optimization roadmap", s["Heading2"]))
1050 |     for item in roadmaps:
1051 |         story.append(bullet(item, s["BulletBody"]))
```

Function: `append_conclusion`

Lines: 1054 to 1063

Description: Analyzes and executes `append_conclusion` logic.

```
1054 | def append_conclusion(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1055 |     story.append(Paragraph("10. Conclusion", s["Heading1"]))
1056 |     paragraphs = [
1057 |         "IDS Sentinel Terminal has crossed an important threshold. It is no longer just a collection of intrusion-d",
1058 |         "Yet the report's central conclusion is deliberately balanced. The product is operationally ahead of its me",
1059 |         "If the repository continues along its current trajectory, the most promising direction is not simply to ad",
1060 |     ]
1061 |     for text in paragraphs:
1062 |         story.append(paragraph(text, s["BodyText"]))
1063 |     story.append(PageBreak())
```

Function: append_feature_glossary

Lines: 1066 to 1076

Description: Analyzes and executes append_feature_glossary logic.

```
1066 | def append_feature_glossary(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1067 |     story.append(Paragraph("Appendix A. Feature Glossary and Analytical Interpretation", s["Heading1"]))
1068 |     descriptions = context["feature_descriptions"]
1069 |     for name in product_terminal.FEATURE_NAMES:
1070 |         story.append(Paragraph(name, s["Heading2"]))
1071 |         base = descriptions.get(name, "No local description was supplied.")
1072 |         extra = (
1073 |             f"In this repository, the feature participates in a binary normal-versus-attack setting and may also fe
1074 |             f"Analysts should therefore read it both as a raw benchmark field and as an ingredient in the product's
1075 |         )
1076 |         story.append(paragraph(f"{base} {extra}", s["BodyText"]))
```

Function: append_command_reference

Lines: 1079 to 1085

Description: Analyzes and executes append_command_reference logic.

```
1079 | def append_command_reference(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1080 |     story.append(Paragraph("Appendix B. Command Reference", s["Heading1"]))
1081 |     descriptions = command_descriptions()
1082 |     for name, desc in context["subcommands"]:
1083 |         story.append(Paragraph(name, s["Heading2"]))
1084 |         story.append(paragraph(descriptions[name], s["BodyText"]))
1085 |         story.append(paragraph(f"Observed parser usage: {desc}", s["Small"]))
```

Function: append_module_appendix

Lines: 1088 to 1105

Description: Analyzes and executes append_module_appendix logic.

```
1088 | def append_module_appendix(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1089 |     story.append(Paragraph("Appendix C. Module-by-Module Symbol Inventory", s["Heading1"]))
1090 |     for module in context["modules"]:
1091 |         story.append(Paragraph(module.path, s["Heading2"]))
1092 |         intro = (
1093 |             f"This module contains {module.line_count:,} lines, {module.function_count} top-level function(s), and
1094 |             f"The table below is generated directly from the Python AST and therefore reflects the actual structure
1095 |         )
1096 |         story.append(paragraph(intro, s["BodyText"]))
1097 |         rows = [["Symbol", "Kind", "Start", "End", "Approximate responsibility"]]
1098 |         for class_info in module.classes:
1099 |             rows.append([class_info.name, "class", class_info.start, class_info.end, "Encapsulates a focused behavior"])
1100 |         for func in module.functions:
1101 |             rows.append([func.name, "function", func.start, func.end, infer_function_role(func.name)])
1102 |         table = LongTable(rows, repeatRows=1, colWidths=[1.65 * inch, 0.65 * inch, 0.5 * inch, 0.5 * inch, 4.0 * inch])
1103 |         table.setStyle(table_style())
1104 |         story.append(table)
1105 |         story.append(Spacer(1, 0.1 * inch))
```

Function: infer_function_role

Lines: 1108 to 1128

Description: Analyzes and executes infer_function_role logic.

```

1108 | def infer_function_role(name: str) -> str:
1109 |     mapping = {
1110 |         "show": "Formats and presents operator-facing output.",
1111 |         "load": "Loads data or configuration from storage or the environment.",
1112 |         "read": "Reads a persisted artifact.",
1113 |         "write": "Writes a persisted artifact.",
1114 |         "parse": "Interprets text or user input into structured values.",
1115 |         "run": "Executes a workflow, command, or worker path.",
1116 |         "build": "Constructs a parser, model, or object graph.",
1117 |         "hash": "Computes a file digest for triage.",
1118 |         "scan": "Performs analysis over a file or dataset.",
1119 |         "learn": "Builds or refreshes a learned profile.",
1120 |         "classify": "Converts signals into a risk or behavior label.",
1121 |         "list": "Enumerates available artifacts or resources.",
1122 |         "probe": "Actively tests a local or remote endpoint.",
1123 |         "resolve": "Normalizes config or paths into concrete objects.",
1124 |     }
1125 |     for prefix, role in mapping.items():
1126 |         if name.startswith(prefix):
1127 |             return role
1128 |     return "Support routine in the module's local workflow."

```

Function: `append_code_listings`

Lines: 1131 to 1184

Description: Analyzes and executes `append_code_listings` logic.

```

1131 | def append_code_listings(story: list[Any], s: dict[str, ParagraphStyle], extra_chunks: int) -> None:
1132 |     story.append(Paragraph("Appendix D. Annotated Code Listings", s["Heading1"]))
1133 |     for spec in listing_specs():
1134 |         story.append(Paragraph(spec.title, s["Heading2"]))
1135 |         story.append(paragraph(spec.explanation, s["CodeCommentary"]))
1136 |         code = extract_code(spec.path, spec.start, spec.end)
1137 |         story.append(
1138 |             Preformatted(
1139 |                 code,
1140 |                 ParagraphStyle(
1141 |                     "Code",
1142 |                     fontName=MONO,
1143 |                     fontSize=7.3,
1144 |                     leading=8.7,
1145 |                     backColor=colors.HexColor("#F6F8FA"),
1146 |                     borderPadding=5,
1147 |                     borderColor=GRID,
1148 |                     borderWidth=0.35,
1149 |                 ),
1150 |             )
1151 |         )
1152 |         story.append(Spacer(1, 0.1 * inch))
1153 |
1154 |     if extra_chunks > 0:
1155 |         story.append(Paragraph("Appendix E. Extended Core Listings", s["Heading1"]))
1156 |         chunk_size = 72
1157 |         start_line = 1
1158 |         for chunk_index in range(extra_chunks):
1159 |             chunk_start = start_line + chunk_index * chunk_size
1160 |             chunk_end = chunk_start + chunk_size - 1
1161 |             story.append(Paragraph(f"Extended core chunk {chunk_index + 1}: `product_terminal.py` lines {chunk_start} to {chunk_end}"))
1162 |             story.append(
1163 |                 paragraph(
1164 |                     "These extended chunks are included because the repository's primary orchestration logic is too large to fit in the main body of the document.",
1165 |                     "They allow a reader to inspect the surrounding implementation context directly.",
1166 |                     s["CodeCommentary"],
1167 |                 )
1168 |             )
1169 |             code = extract_code("ids_app/product_terminal.py", chunk_start, chunk_end)
1170 |             story.append(
1171 |                 Preformatted(
1172 |                     code,
1173 |                     ParagraphStyle(
1174 |                         "CodeExtended",
1175 |                         fontName=MONO,
1176 |                         fontSize=7.2,
1177 |                         leading=8.5,
1178 |                         backColor=colors.HexColor("#FBFBFB"),
1179 |                         borderPadding=5,
1180 |                         borderColor=GRID,
1181 |                         borderWidth=0.35,
1182 |                     ),
1183 |                 )
1184 |             )

```

Function: append_output_appendix

Lines: 1187 to 1197

Description: Analyzes and executes append_output_appendix logic.

```

1187 | def append_output_appendix(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1188 |     story.append(Paragraph("Appendix F. Live Output Evidence", s["Heading1"]))
1189 |     figures = [
1190 |         ("Status command output", context["assets"]["status_terminal.png"]),
1191 |         ("Attacks command output", context["assets"]["attacks_terminal.png"]),
1192 |         ("Ports command output", context["assets"]["ports_terminal.png"]),
1193 |     ]
1194 |     for title, image_path in figures:
1195 |         story.append(Paragraph(title, s["Heading2"]))
1196 |         story.append(RLImage(str(image_path), width=6.8 * inch, height=4.15 * inch))
1197 |         story.append(Paragraph(f"Figure. {title} rendered from a live command invocation in the report environment."))

```

Function: numbered_text_block

Lines: 1200 to 1203

Description: Analyzes and executes numbered_text_block logic.

```

1200 | def numbered_text_block(text: str) -> str:
1201 |     lines = text.splitlines()
1202 |     width = len(str(len(lines) or 1))
1203 |     return "\n".join(f"{index:{width}d}: {line}" for index, line in enumerate(lines, start=1))

```

Function: append_raw_evidence

Lines: 1206 to 1232

Description: Analyzes and executes append_raw_evidence logic.

```

1206 | def append_raw_evidence(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1207 |     story.append(Paragraph("Appendix G. Primary JSON Evidence Dumps", s["Heading1"]))
1208 |     evidence = [
1209 |         ("status --json", json.dumps(context["status"], indent=2)),
1210 |         ("attacks --json", json.dumps(context["attacks"], indent=2)),
1211 |         ("datasets --json", json.dumps(context["datasets"], indent=2)),
1212 |         ("runs --json", json.dumps(context["runs"], indent=2)),
1213 |     ]
1214 |     style = ParagraphStyle(
1215 |         "EvidenceCode",
1216 |         fontName=MONO,
1217 |         fontSize=6.6,
1218 |         leading=7.8,
1219 |         backColor=colors.HexColor("#F8FAFC"),
1220 |         borderPadding=5,
1221 |         borderColor=GRID,
1222 |         borderWidth=0.35,
1223 |     )
1224 |     for title, payload in evidence:
1225 |         story.append(Paragraph(title, s["Heading2"]))
1226 |         story.append(
1227 |             paragraph(
1228 |                 "This dump is included verbatim from a live command invocation so that readers can inspect the exact output.",
1229 |                 s["CodeCommentary"],
1230 |             )
1231 |         )
1232 |         story.append(Preformatted(numbered_text_block(payload), style))

```

Function: append_bibliography

Lines: 1235 to 1241

Description: Analyzes and executes append_bibliography logic.

```

1235 | def append_bibliography(story: list[Any], s: dict[str, ParagraphStyle], context: dict[str, Any]) -> None:
1236 |     story.append(Paragraph("Appendix H. Annotated Bibliography", s["Heading1"]))
1237 |     for source in context["sources"]:
1238 |         story.append(Paragraph(f"[{source.source_id}] {source.title}", s["Heading2"]))
1239 |         entry = f"{source.authors}. {source.year}. {source.url}"
1240 |         story.append(paragraph(entry, s["Small"]))
1241 |         story.append(paragraph(f"{source.access_note} {source.relevance}", s["BodyText"]))

```

Function: build_story

Lines: 1244 to 1266

Description: Analyzes and executes build_story logic.

```

1244 | def build_story(context: dict[str, Any], extra_chunks: int) -> list[Any]:
1245 |     s = styles()
1246 |     story: list[Any] = []
1247 |     append_story_intro(story, s, context)
1248 |     append_abstract(story, s, context)
1249 |     append_chapter_introduction(story, s, context)
1250 |     append_chapter_foundations(story, s, context)
1251 |     append_chapter_datasets(story, s, context)
1252 |     append_chapter_architecture(story, s, context)
1253 |     append_chapter_detection(story, s, context)
1254 |     append_chapter_ml(story, s, context)
1255 |     append_chapter_operations(story, s, context)
1256 |     append_chapter_distribution(story, s, context)
1257 |     append_chapter_risks(story, s, context)
1258 |     append_conclusion(story, s, context)
1259 |     append_feature_glossary(story, s, context)
1260 |     append_command_reference(story, s, context)
1261 |     append_module_appendix(story, s, context)
1262 |     append_code_listings(story, s, extra_chunks)
1263 |     append_output_appendix(story, s, context)
1264 |     append_raw_evidence(story, s, context)
1265 |     append_bibliography(story, s, context)
1266 |     return story

```

Function: build_pdf

Lines: 1269 to 1282

Description: Analyzes and executes build_pdf logic.

```

1269 | def build_pdf(context: dict[str, Any], extra_chunks: int) -> int:
1270 |     doc = SimpleDocTemplate(
1271 |         str(REPORT_PATH),
1272 |         pagesize=letter,
1273 |         leftMargin=0.75 * inch,
1274 |         rightMargin=0.75 * inch,
1275 |         topMargin=0.8 * inch,
1276 |         bottomMargin=0.8 * inch,
1277 |         title="IDS Sentinel Terminal Research Report",
1278 |         author="OpenAI Codex",
1279 |     )
1280 |     story = build_story(context, extra_chunks)
1281 |     doc.build(story, canvasmaker=NumberedCanvas)
1282 |     return NumberedCanvas.last_page_count

```

Function: choose_chunk_count

Lines: 1285 to 1295

Description: Analyzes and executes choose_chunk_count logic.

```
1285 | def choose_chunk_count(context: dict[str, Any], min_pages: int, max_pages: int) -> int:
1286 |     extra_chunks = 12
1287 |     for _ in range(6):
1288 |         page_count = build_pdf(context, extra_chunks)
1289 |         if min_pages <= page_count <= max_pages:
1290 |             return extra_chunks
1291 |         if page_count < min_pages:
1292 |             extra_chunks += max(1, math.ceil((min_pages - page_count) / 2))
1293 |         else:
1294 |             extra_chunks = max(0, extra_chunks - max(1, math.ceil((page_count - max_pages) / 2)))
1295 |     return extra_chunks
```

Function: build_report

Lines: 1298 to 1302

Description: Analyzes and executes build_report logic.

```
1298 | def build_report(min_pages: int, max_pages: int) -> tuple[Path, int]:
1299 |     context = build_context()
1300 |     extra_chunks = choose_chunk_count(context, min_pages, max_pages)
1301 |     page_count = build_pdf(context, extra_chunks)
1302 |     return REPORT_PATH, page_count
```

Function: parse_args

Lines: 1305 to 1309

Description: Analyzes and executes parse_args logic.

```
1305 | def parse_args() -> argparse.Namespace:
1306 |     parser = argparse.ArgumentParser(description="Build the IDS Sentinel Terminal research report PDF.")
1307 |     parser.add_argument("--min-pages", type=int, default=130)
1308 |     parser.add_argument("--max-pages", type=int, default=150)
1309 |     return parser.parse_args()
```

Function: main

Lines: 1312 to 1320

Description: Analyzes and executes main logic.

```
1312 | def main() -> int:
1313 |     args = parse_args()
1314 |     ensure_dirs()
1315 |     report_path, page_count = build_report(args.min_pages, args.max_pages)
1316 |     print(report_path.relative_to(ROOT))
1317 |     print(f"pages={page_count}")
1318 |     if not (args.min_pages <= page_count <= args.max_pages):
1319 |         print(f"warning: page count is outside the requested range {args.min_pages}-{args.max_pages}")
1320 |     return 0
```

Function: __init__

Lines: 102 to 104

Description: Analyzes and executes init logic.

```
0102 |     def __init__(self, *args: Any, **kwargs: Any) -> None:
0103 |         super().__init__(*args, **kwargs)
0104 |         self._saved_page_states: list[dict[str, Any]] = []
```

Function: showPage

Lines: 106 to 108

Description: Analyzes and executes showPage logic.

```
0106 |     def showPage(self) -> None:
0107 |         self._saved_page_states.append(dict(self.__dict__))
0108 |         self._startPage()
```

Function: save

Lines: 110 to 117

Description: Analyzes and executes save logic.

```
0110 |     def save(self) -> None:
0111 |         page_count = len(self._saved_page_states)
0112 |         type(self).last_page_count = page_count
0113 |         for state in self._saved_page_states:
0114 |             self.__dict__.update(state)
0115 |             self.draw_footer(page_count)
0116 |             super().showPage()
0117 |             super().save()
```

Function: draw_footer

Lines: 119 to 125

Description: Analyzes and executes draw_footer logic.

```
0119 |     def draw_footer(self, page_count: int) -> None:
0120 |         self.setStrokeColor(colors.HexColor("#D9D9D9"))
0121 |         self.line(0.75 * inch, 0.62 * inch, 7.75 * inch, 0.62 * inch)
0122 |         self.setFont(BODY_FONT, 8.5)
0123 |         self.setFillColor(MUTED)
0124 |         self.drawString(0.78 * inch, 0.4 * inch, "IDS Sentinel Terminal Research Report")
0125 |         self.drawRightString(7.72 * inch, 0.4 * inch, f"Page {self._pageNumber} of {page_count}")
```

Function: _capture

Lines: 349 to 353

Description: Analyzes and executes _capture logic.

```
0349 |     def _capture() -> None:
0350 |         root.update_idletasks()
0351 |         image = ImageGrab.grab(window=root.wininfo_id())
0352 |         image.save(path)
0353 |         root.destroy()
```

Module: ./scripts/build_distributions.py

Overview

Total Lines: 198

Function: copy_tree

Lines: 24 to 31

Description: Analyzes and executes copy_tree logic.

```
0024 | def copy_tree(source: Path, target: Path) -> None:
0025 |     if target.exists():
0026 |         shutil.rmtree(target)
0027 |     shutil.copytree(
0028 |         source,
0029 |         target,
0030 |         ignore=shutil.ignore_patterns("__pycache__", "*.pyc", ".pytest_cache"),
0031 |     )
```

Function: write_text

Lines: 34 to 38

Description: Analyzes and executes write_text logic.

```
0034 | def write_text(path: Path, text: str, executable: bool = False) -> None:
0035 |     path.parent.mkdir(parents=True, exist_ok=True)
0036 |     path.write_text(text, encoding="utf-8", newline="\n")
0037 |     if executable:
0038 |         path.chmod(path.stat().st_mode | stat.S_IXUSR | stat.S_IXGRP | stat.S_IXOTH)
```

Function: prepare_stage

Lines: 41 to 67

Description: Analyzes and executes prepare_stage logic.

```

0041 | def prepare_stage(include_exports: bool = False) -> Path:
0042 |     if BUILD_DIR.exists():
0043 |         shutil.rmtree(BUILD_DIR)
0044 |     BUILD_DIR.mkdir(parents=True)
0045 |
0046 |     app_src = BUILD_DIR / "pyz_src"
0047 |     copy_tree(ROOT / "ids_app", app_src / "ids_app")
0048 |     zipapp.create_archive(app_src, BUILD_DIR / APP_PYZ, main="ids_app.product_app:main", interpreter="/usr/bin/env p
0049 |
0050 |     for filename in ("README.md", "INSTALL_FOR_PITCH.md", "DOWNLOAD_TOOL.md"):
0051 |         source = ROOT / filename
0052 |         if source.exists():
0053 |             shutil.copy2(source, BUILD_DIR / filename)
0054 |
0055 |     product_dir = BUILD_DIR / "automation" / "product"
0056 |     (product_dir / "exports").mkdir(parents=True, exist_ok=True)
0057 |     (product_dir / "imports").mkdir(parents=True, exist_ok=True)
0058 |     (product_dir / "cache" / "indexes").mkdir(parents=True, exist_ok=True)
0059 |     (product_dir / "cache" / "commands").mkdir(parents=True, exist_ok=True)
0060 |
0061 |     if include_exports and (ROOT / "automation" / "product" / "exports").exists():
0062 |         copy_tree(ROOT / "automation" / "product" / "exports", product_dir / "exports")
0063 |
0064 |     write_launchers(BUILD_DIR)
0065 |     write_text(BUILD_DIR / "VERSION.txt", f"build_time={datetime.now().isoformat(timespec='seconds')}\n")
0066 |     shutil.rmtree(app_src)
0067 |     return BUILD_DIR

```

Function: write_launchers

Lines: 70 to 136

Description: Analyzes and executes write_launchers logic.

```

0070 | def write_launchers(stage: Path) -> None:
0071 |     write_text(
0072 |         stage / f"{CLI_NAME}.cmd",
0073 |         """@echo off
0074 | setlocal
0075 | cd /d "%~dp0"
0076 | set "IDS_PRODUCT_HOME=%CD%"
0077 | where py >nul 2>nul
0078 | if %ERRORLEVEL% EQU 0 (
0079 |     py -3 "%~dp0ids-sentinel-terminal.pyz" %*
0080 |     exit /b %ERRORLEVEL%
0081 | )
0082 | where python >nul 2>nul
0083 | if %ERRORLEVEL% EQU 0 (
0084 |     python "%~dp0ids-sentinel-terminal.pyz" %*
0085 |     exit /b %ERRORLEVEL%
0086 | )
0087 | echo Python 3 was not found. Install Python 3 and rerun this command. 1>&2
0088 | exit /b 1
0089 | """
0090 | )
0091 |     write_text(
0092 |         stage / f"{GUI_NAME}.cmd",
0093 |         """@echo off
0094 | setlocal
0095 | cd /d "%~dp0"
0096 | call "%~dp0ids-sentinel-terminal.cmd" gui
0097 | """
0098 | )
0099 |     write_text(
0100 |         stage / CLI_NAME,
0101 |         """#!/usr/bin/env sh
0102 | set -eu
0103 | DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
0104 | export IDS_PRODUCT_HOME="$DIR"
0105 | exec python3 "$DIR/ids-sentinel-terminal.pyz" "$@"
0106 | """
0107 |         executable=True,
0108 |     )
0109 |     write_text(
0110 |         stage / GUI_NAME,
0111 |         """#!/usr/bin/env sh
0112 | set -eu
0113 | DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
0114 | export IDS_PRODUCT_HOME="$DIR"
0115 | exec python3 "$DIR/ids-sentinel-terminal.pyz" gui "$@"
0116 | """
0117 |         executable=True,
0118 |     )
0119 |     write_text(
0120 |         stage / "INSTALL.txt",
0121 |         """IDS Sentinel Terminal
0122 |
0123 | This portable build contains the packaged app plus bundled seed datasets.
0124 | On first run it will initialize the working home inside this folder.
0125 |
0126 | Windows:
0127 |     ids-sentinel-terminal.cmd status
0128 |     ids-sentinel-terminal-gui.cmd
0129 |
0130 | macOS/Linux:
0131 |     ./ids-sentinel-terminal status
0132 |     ./ids-sentinel-terminal-gui
0133 |
0134 | Read README.md for the full manual.
0135 | """
0136 | )

```

Function: make_zip

Lines: 139 to 144

Description: Analyzes and executes make_zip logic.

```
0139 | def make_zip(stage: Path, target: Path) -> None:
0140 |     if target.exists():
0141 |         target.unlink()
0142 |     with zipfile.ZipFile(target, "w", compression=zipfile.ZIP_DEFLATED) as archive:
0143 |         for path in stage.rglob("**"):
0144 |             archive.write(path, path.relative_to(stage.parent))
```

Function: make_tar

Lines: 147 to 151

Description: Analyzes and executes make_tar logic.

```
0147 | def make_tar(stage: Path, target: Path) -> None:
0148 |     if target.exists():
0149 |         target.unlink()
0150 |     with tarfile.open(target, "w:gz") as archive:
0151 |         archive.add(stage, arcname=stage.name)
```

Function: build_archives

Lines: 154 to 167

Description: Analyzes and executes build_archives logic.

```
0154 | def build_archives(include_exports: bool = False) -> list[Path]:
0155 |     stage = prepare_stage(include_exports=include_exports)
0156 |     DIST_DIR.mkdir(parents=True, exist_ok=True)
0157 |     targets = [
0158 |         DIST_DIR / f"{PACKAGE_NAME}-windows.zip",
0159 |         DIST_DIR / f"{PACKAGE_NAME}-macos.tar.gz",
0160 |         DIST_DIR / f"{PACKAGE_NAME}-linux.tar.gz",
0161 |         DIST_DIR / f"{PACKAGE_NAME}-portable.zip",
0162 |     ]
0163 |     make_zip(stage, targets[0])
0164 |     make_tar(stage, targets[1])
0165 |     make_tar(stage, targets[2])
0166 |     make_zip(stage, targets[3])
0167 |     return targets
```

Function: build_python_package

Lines: 170 to 180

Description: Analyzes and executes build_python_package logic.

```

0170 | def build_python_package() -> list[Path]:
0171 |     DIST_DIR.mkdir(parents=True, exist_ok=True)
0172 |     subprocess.run(
0173 |         [sys.executable, str(ROOT / "scripts" / "build_python_package.py)],
0174 |         cwd=ROOT,
0175 |         check=True,
0176 |     )
0177 |     targets = []
0178 |     for pattern in ("ids_sentinel_terminal-*.whl", "ids_sentinel_terminal-*.tar.gz", "ids-sentinel-terminal-*.tar.g
0179 |         targets.extend(sorted(DIST_DIR.glob(pattern)))
0180 |     return targets

```

Function: main

Lines: 183 to 194

Description: Analyzes and executes main logic.

```

0183 | def main() -> int:
0184 |     parser = argparse.ArgumentParser(description="Build cross-platform IDS Sentinel Terminal archives.")
0185 |     parser.add_argument("--include-exports", action="store_true", help="Bundle generated analysis reports too.")
0186 |     parser.add_argument("--python-package", action="store_true", help="Also build wheel and sdist via python -m bui
0187 |     args = parser.parse_args()
0188 |     targets = build_archives(include_exports=args.include_exports)
0189 |     if args.python_package:
0190 |         targets.extend(build_python_package())
0191 |     for target in targets:
0192 |         size_mb = target.stat().st_size / (1024 * 1024)
0193 |         print(f"{target.relative_to(ROOT)} ({size_mb:.2f} MB)")
0194 |     return 0

```

Module: ./scripts/build_python_package.py

Overview

Total Lines: 85

Function: is_python_package_artifact

Lines: 14 to 20

Description: Analyzes and executes is_python_package_artifact logic.

```

0014 | def is_python_package_artifact(path: Path) -> bool:
0015 |     if path.suffix == ".whl" and path.name.startswith("ids_sentinel_terminal-"):
0016 |         return True
0017 |     if path.name.startswith(("ids_sentinel_terminal-", "ids-sentinel-terminal-")) and path.name.endswith(".tar.gz")
0018 |         suffix = path.name.split("ids_sentinel_terminal-", 1)[-1] if path.name.startswith("ids_sentinel_terminal-")
0019 |         return bool(suffix) and suffix[0].isdigit()
0020 |     return False

```

Function: clean_previous_outputs

Lines: 23 to 33

Description: Analyzes and executes clean_previous_outputs logic.

```

0023 | def clean_previous_outputs() -> None:
0024 |     if BUILD_SRC_DIR.exists():
0025 |         shutil.rmtree(BUILD_SRC_DIR)
0026 |     DIST_DIR.mkdir(parents=True, exist_ok=True)
0027 |     for path in DIST_DIR.iterdir():
0028 |         if path.is_dir():
0029 |             continue
0030 |         if not path.name.endswith((".whl", ".tar.gz")):
0031 |             continue
0032 |         if is_python_package_artifact(path):
0033 |             path.unlink()

```

Function: stage_sources

Lines: 36 to 52

Description: Analyzes and executes stage_sources logic.

```

0036 | def stage_sources() -> Path:
0037 |     BUILD_SRC_DIR.mkdir(parents=True, exist_ok=True)
0038 |     shutil.copy2(ROOT / "pyproject.toml", BUILD_SRC_DIR / "pyproject.toml")
0039 |     shutil.copy2(ROOT / "README.md", BUILD_SRC_DIR / "README.md")
0040 |     if (ROOT / "INSTALL_FOR_PITCH.md").exists():
0041 |         shutil.copy2(ROOT / "INSTALL_FOR_PITCH.md", BUILD_SRC_DIR / "INSTALL_FOR_PITCH.md")
0042 |     if (ROOT / "DOWNLOAD_TOOL.md").exists():
0043 |         shutil.copy2(ROOT / "DOWNLOAD_TOOL.md", BUILD_SRC_DIR / "DOWNLOAD_TOOL.md")
0044 |     if (ROOT / "MANIFEST.in").exists():
0045 |         shutil.copy2(ROOT / "MANIFEST.in", BUILD_SRC_DIR / "MANIFEST.in")
0046 |     shutil.copypath(
0047 |         ROOT / "ids_app",
0048 |         BUILD_SRC_DIR / "ids_app",
0049 |         ignore=shutil.ignore_patterns("__pycache__", "*.pyc", ".pytest_cache"),
0050 |     )
0051 |     (BUILD_SRC_DIR / "setup.py").write_text("from setuptools import setup\nsetup()\n", encoding="utf-8", newline="\n")
0052 |     return BUILD_SRC_DIR

```

Function: ensure_build_tools

Lines: 55 to 60

Description: Analyzes and executes ensure_build_tools logic.

```

0055 | def ensure_build_tools() -> None:
0056 |     try:
0057 |         import setuptools # noqa: F401
0058 |         import wheel # noqa: F401
0059 |     except ImportError:
0060 |         subprocess.run([sys.executable, "-m", "pip", "install", "setuptools", "wheel"], cwd=ROOT, check=True)

```

Function: run_setup

Lines: 63 to 68

Description: Analyzes and executes run_setup logic.

```
0063 | def run_setup(stage_dir: Path, command: str) -> None:
0064 |     subprocess.run(
0065 |         [sys.executable, "setup.py", command, "--dist-dir", str(DIST_DIR)],
0066 |         cwd=stage_dir,
0067 |         check=True,
0068 |     )
```

Function: main

Lines: 71 to 81

Description: Analyzes and executes main logic.

```
0071 | def main() -> int:
0072 |     clean_previous_outputs()
0073 |     ensure_build_tools()
0074 |     stage_dir = stage_sources()
0075 |     run_setup(stage_dir, "bdist_wheel")
0076 |     run_setup(stage_dir, "sdist")
0077 |     for path in sorted(DIST_DIR.iterdir()):
0078 |         if path.is_file() and is_python_package_artifact(path):
0079 |             size_mb = path.stat().st_size / (1024 * 1024)
0080 |             print(f"{path.relative_to(ROOT)} ({size_mb:.2f} MB)")
0081 |     return 0
```

Module: ./temp1.py

Overview

Total Lines: 76